

# Open-FDD Documentation

Generated by scripts/build\_docs\_pdf.py

June 16, 2026

- 1 Home
- 2 Open-FDD
  - 2.1 Who it is for
  - 2.2 Start here
  - 2.3 v3 edge stack
  - 2.4 Distribution
  - 2.5 License
- 3 Open-FDD Central / Portfolio / RCx reports
- 4 Open-FDD Central / Portfolio / RCx reports
  - 4.1 Boundary
  - 4.2 Incremental build order
  - 4.3 RCx report content (normalized roles)
  - 4.4 Prerequisites
  - 4.5 Safety
- 5 Run with Docker images
- 6 Run with Docker images
  - 6.1 Images
  - 6.2 1. Install and validate Docker
  - 6.3 2. Bootstrap the site (one script)
    - 6.3.1 Manual start (optional)
  - 6.4 Long-term operation
    - 6.4.1 Survive power cycles
    - 6.4.2 Start / stop / restart
    - 6.4.3 LAN access
  - 6.5 Deployment choices
  - 6.6 Next steps
- 7 First login and health check
- 8 First login and health check
  - 8.1 Public health
  - 8.2 Login
  - 8.3 Smoke checklist
  - 8.4 Stack detail (authenticated)
  - 8.5 Troubleshooting
- 9 Quick Start
- 10 Quick Start
- 11 Updating the stack

- 12 Updating the stack
  - 12.1 Before you start
  - 12.2 Fetch helper scripts (no git clone)
  - 12.3 One-command upgrade (recommended)
  - 12.4 Step by step
    - 12.4.1 1. Backup site state
    - 12.4.2 2. Verify images (optional)
    - 12.4.3 3. Pull and recreate
    - 12.4.4 4. Verify
  - 12.5 Dashboard UI updates
  - 12.6 Rollback
  - 12.7 Safe cleanup
  - 12.8 Next steps
- 13 Local development
- 14 Local development
  - 14.1 Clone and auth
  - 14.2 Start stack
  - 14.3 BACnet bench overlay (optional)
  - 14.4 Production-like local stack (Caddy, LAN scans)
  - 14.5 Docs preview
- 15 Build Docker images
- 16 Build Docker images
  - 16.1 Local build
  - 16.2 Publish to GHCR (maintainers)
- 17 Developer Guide
- 18 Developer Guide
- 19 Tests and CI
- 20 Tests and CI
  - 20.1 Local tests
  - 20.2 CI (GitHub Actions)
  - 20.3 Docs build (same as CI)
  - 20.4 Security-related CI
- 21 DataFusion SQL rule recipes
- 22 DataFusion SQL recipes
  - 22.1 Temperature high
  - 22.2 Humidity high
  - 22.3 Occupied + temperature
  - 22.4 With fault confirmation
- 23 Contributing
- 24 Contributing
  - 24.1 Expectations
  - 24.2 Pull request checklist
  - 24.3 Repository layout (short)
  - 24.4 Docs links (GitHub Pages)
  - 24.5 Security issues
  - 24.6 Agent-assisted development
- 25 Health checks (developer)
- 26 Health checks (developer)
  - 26.1 Clone and local stack
  - 26.2 Stack health script
  - 26.3 Compose overlays
  - 26.4 BACnet poll pipeline

- 26.5 pytest (bridge security / BACnet)
- 26.6 Pentest production stack (LAN ZAP)
- 26.7 Post-deploy insurance (inventory hosts)
- 27 Security testing cycle
- 28 Security testing cycle
  - 28.1 Who owns what
  - 28.2 Per-revision workflow
    - 28.2.1 1. Local functional tests
    - 28.2.2 2. Pentest production stack (on Open-FDD host)
    - 28.2.3 3. Host + exposure checks (on edge VM)
    - 28.2.4 4. LAN security scan (from laptop)
    - 28.2.5 5. Pull request + automated review
    - 28.2.6 6. Release
    - 28.2.7 7. Lab edge validation
    - 28.2.8 8. Next cycle
  - 28.3 What each scan layer covers
  - 28.4 Open-source contributors
- 29 Python recipes (full Arrow library)
- 30 Python recipes — full Arrow library
  - 30.1 Shared helpers (paste once per rule file)
  - 30.2 GL36 AHU Rules A-M
    - 30.2.1 Rule A — duct static low at full fan speed (AHU-A)
    - 30.2.2 Rules B & C — blended air outside OAT/RAT band (AHU-D)
    - 30.2.3 Rule D — discharge cold when heating commanded (AHU-B)
    - 30.2.4 Rule E — SAT too low with full heating (AHU-C)
    - 30.2.5 Rule F — SAT/MAT mismatch in economizer mode (AHU-E)
    - 30.2.6 Rule G — ambient too warm for free cooling (AHU-E)
    - 30.2.7 Rule H — OAT/MAT mismatch (econ + mech cooling) (AHU-E)
    - 30.2.8 Rule I — OAT/MAT mismatch (economizer only) (AHU-E)
    - 30.2.9 Rule J — discharge above blended in cooling (AHU-B)
    - 30.2.10 Rule K — discharge above setpoint in full cooling (AHU-C)
    - 30.2.11 Rule L — cooling coil  $\Delta T$  when inactive (CH-C)
    - 30.2.12 Rule M — heating coil  $\Delta T$  when inactive (AHU-B)
  - 30.3 Starter pack (VAV / AHU baseline)
    - 30.3.1 01\_vav\_zone\_temp\_bounds\_occupied (VAV-C)
    - 30.3.2 02\_vav\_zone\_temp\_flatline\_occupied (VAV-C)
    - 30.3.3 03\_vav\_damper\_command\_extreme\_flatline (VAV-D)
    - 30.3.4 04\_ahu\_runtime\_outside\_schedule (BLD-C)
    - 30.3.5 05 / 06 / 07 — see Rules A, sensor\_bounds\_mask on sa-t / sensor\_flatline\_mask on sa-t
  - 30.4 Economizer starters
    - 30.4.1 ahu\_econ\_100oa\_temp\_tracking\_fault (AHU-E)
    - 30.4.2 ahu\_mech\_cooling\_when\_free\_cooling\_available (AHU-E)
    - 30.4.3 ahu\_oa\_damper\_excess\_open\_extreme\_ambient (AHU-E)
  - 30.5 VAV zones
    - 30.5.1 zone\_reheat\_warm\_ambient (VAV-A)
    - 30.5.2 zone\_damper\_valve\_full\_open (VAV-D)

- 30.5.3 Zone / IAQ bounds
- 30.6 Central plant
  - 30.6.1 dp\_below\_sp\_pump\_max (CH-E)
  - 30.6.2 flow\_high\_pump\_max (CH-B)
  - 30.6.3 plant\_supply\_temp\_deadband (CH-D)
  - 30.6.4 chiller\_excessive\_runtime (CH-F)
- 30.7 Heat pumps
  - 30.7.1 hp\_discharge\_cold\_when\_heating (HP-D)
- 30.8 Opportunistic / ventilation
  - 30.8.1 econ\_active\_warm\_ambient (AHU-E)
  - 30.8.2 mech\_cool\_when\_econ\_available (AHU-E)
  - 30.8.3 low\_oa\_fraction\_estimated (VAV-B)
  - 30.8.4 preheat\_excess\_temp (AHU-B)
- 30.9 Weather station
  - 30.9.1 weather\_temp\_spike (BLD-B)
  - 30.9.2 weather\_gust\_lt\_wind (BLD-B)
  - 30.9.3 RH bounds
- 30.10 Schedule + weather gating (occupied hours)
- 30.11 FC4 — PID hunting (legacy GL36)
- 30.12 Next steps
- 31 System overview
- 32 System overview
  - 32.1 Docker edge stack (production)
  - 32.2 Components
  - 32.3 Deployment
  - 32.4 Optional PyPI-only path
- 33 Deployment modes
- 34 Deployment modes
  - 34.1 Mode summary
  - 34.2 v3 container stack (all edge modes)
  - 34.3 Caddy HTTP (typical edge)
  - 34.4 Caddy TLS (OT LAN)
  - 34.5 BACnet data path
  - 34.6 What not to do on production edges
  - 34.7 Related
- 35 Containers
- 36 Containers
  - 36.1 GHCR images
  - 36.2 Stack diagram
  - 36.3 Responsibilities
  - 36.4 Network and ports (typical edge)
  - 36.5 Long-lived deployment
  - 36.6 Persistent data
  - 36.7 Optional host services
- 37 Data flow
- 38 Data flow
  - 38.1 Telemetry ingest
    - 38.1.1 BACnet
    - 38.1.2 Modbus TCP
    - 38.1.3 JSON API (HTTP/HTTPS)
  - 38.2 Rule evaluation
  - 38.3 Model sync

- 38.4 Retention
- 39 Architecture
- 40 Architecture
- 41 Fault confirmation / minimum duration
- 42 Fault confirmation
  - 42.1 Config fields
  - 42.2 Example
  - 42.3 Paired smoke default (5 minutes)
- 43 Rule Cookbook
- 44 Rule Cookbook
- 45 Windowing & debugging
- 46 Windowing & debugging
  - 46.1 Lookback vs rolling window
  - 46.2 Rolling windows (Arrow)
  - 46.3 Rule Lab console
- 47 Expression cookbook (Arrow-native)
- 48 Expression cookbook (Arrow-native)
  - 48.1 Legacy → modern translation
  - 48.2 How Arrow rules are structured
    - 48.2.1 Command scaling (0-1 vs 0-100 %)
    - 48.2.2 Occupied hours (schedule gating)
  - 48.3 Sensor validation (bounds, flatline, rate of change)
    - 48.3.1 Bounds (out of range) — oob\_rolling
    - 48.3.2 Flatline (stuck sensor) — flatline\_1h
    - 48.3.3 Rate of change (spike) — rate\_of\_change
    - 48.3.4 Mixing envelope (MAT vs OAT/RAT) — mixing\_envelope
  - 48.4 GL36-inspired AHU rules (legacy expression → fault code)
  - 48.5 Starter pack (VAV / AHU baseline)
    - 48.5.1 Economizer starters
  - 48.6 VAV, central plant, heat pump
  - 48.7 Opportunistic / ventilation
  - 48.8 Data quality vs equipment faults
  - 48.9 Metric (°C) equivalents
  - 48.10 Binding rules to the fault catalog
  - 48.11 Pandas YAML → Arrow checklist (commissioning)
- 49 GL36 algorithm stubs
- 50 GL36 algorithm stubs (doc-only)
  - 50.1 Shared lookback helper
  - 50.2 VAV zone cooling requests (0-3)
  - 50.3 AHU duct static trim & respond (advisory fault)
  - 50.4 Chiller plant enable (valve request counting)
  - 50.5 Hot water plant HWST trim & respond (advisory)
  - 50.6 Chilled water plant (DP + CHWST reset advisory)
  - 50.7 Testing later
- 51 Central plants (CHW / CTW / BLR)
- 52 Central plant Arrow recipes
  - 52.1 Starter fault codes
  - 52.2 Required point roles (CHW)
  - 52.3 Synthetic test pattern
- 53 Dashboard overview
- 54 Dashboard overview
  - 54.1 Primary areas

- 54.2 Live updates
- 54.3 Roles
- 55 Auth and roles
- 56 Auth and roles
  - 56.1 Production (LAN / edge)
  - 56.2 Localhost dev
- 57 Rule Lab
- 58 Rule Lab
  - 58.1 Equipment-scoped test and export
  - 58.2 Export all rules
  - 58.3 Workflow
  - 58.4 Dev kit zip (download)
  - 58.5 Lookback windows
  - 58.6 Algorithms (supervisory — coming soon)
  - 58.7 Related
- 59 JSON API driver
- 60 JSON API driver
  - 60.1 Supported features
  - 60.2 Secrets: `json_api.env.local`
  - 60.3 OpenWeatherMap showcase (recommended demo)
    - 60.3.1 1. Configure env
    - 60.3.2 2. Register from the UI
    - 60.3.3 3. Headless / script
    - 60.3.4 4. FDD: local OA-T vs web weather
  - 60.4 Commissioning (UI)
    - 60.4.1 API for agents
    - 60.4.2 Bench preset
  - 60.5 Authentication examples
    - 60.5.1 Query-string API key via env (OpenWeather)
    - 60.5.2 Bearer token (API key)
    - 60.5.3 HTTP Basic
    - 60.5.4 POST with JSON body
  - 60.6 REST API
  - 60.7 Typical OT URLs
  - 60.8 Limitations
  - 60.9 Disable polling (save API quota)
  - 60.10 Smoke test
- 61 Algorithms
- 62 Algorithms (coming soon)
  - 62.1 GL36 Trim & Respond reference
  - 62.2 Planned workflow (same kit shape as Rule Lab)
  - 62.3 Lookback windows
  - 62.4 AHU supervisory checks (doc-only stubs)
    - 62.4.1 Duct static pressure too high
    - 62.4.2 Supply air temperature too cold (cooling coil over-delivering)
  - 62.5 Plant supervisory checks (doc-only)
  - 62.6 Related
- 63 Model workflow
- 64 Model workflow
  - 64.1 Brick model
  - 64.2 Typical integrator flow

- 65 Driver framework
- 66 Driver framework
  - 66.1 Drivers
  - 66.2 Data layout
  - 66.3 Historian and FDD
  - 66.4 Rule Lab (Arrow upload/download)
  - 66.5 Smoke validation
  - 66.6 Related docs
- 67 Operator Bridge
- 68 Operator Bridge
- 69 Ollama and analytics
- 70 Ollama and analytics
  - 70.1 What runs without login
  - 70.2 Ollama setup
  - 70.3 Automatic analytics (deterministic, always on)
  - 70.4 When Ollama is used
  - 70.5 AI Agent (authenticated)
  - 70.6 Code map
- 71 FDD and assignments
- 72 FDD and assignments
  - 72.1 What the product actually does
    - 72.1.1 BRICK modeling (integrator + AI-assisted)
    - 72.1.2 Rule assignment (three binding kinds)
    - 72.1.3 CLASS vs device
  - 72.2 Commissioning JSON shape
    - 72.2.1 Rule names vs ids (Rule Lab alignment)
  - 72.3 Related
- 73 Network setup
- 74 Network setup
  - 74.1 Bind address
  - 74.2 Discovery range
  - 74.3 Verify
- 75 Discover and read
- 76 Discover and read
  - 76.1 Operator workflow
- 77 Polling
- 78 Polling
  - 78.1 Commission poll loop (default)
  - 78.2 Bridge ingest worker
  - 78.3 Health
  - 78.4 Poll strategy (edge)
  - 78.5 Async driver and single-stack I/O
  - 78.6 Operator override scans
- 79 Operator override scans
- 80 Operator override scans (P8)
  - 80.1 Schedule
  - 80.2 What each scan does
  - 80.3 BACnet I/O priority
  - 80.4 Manual trigger
  - 80.5 Health checks
  - 80.6 Data paths
- 81 Write safety

- 82 Write safety
  - 82.1 Defaults
  - 82.2 Enable writes (commission role)
  - 82.3 Operator rules
- 83 BACnet
- 84 BACnet
- 85 Convention
- 86 Fault code convention
  - 86.1 Format (Grade-A, 3.0.18+)
  - 86.2 Categories (Grade-A)
  - 86.3 Severity
  - 86.4 Linking rules
  - 86.5 Cookbook mapping
- 87 Live site update
- 88 Live site update
  - 88.1 Layout on the edge host
  - 88.2 Concepts
  - 88.3 1. Verify new images on GHCR
  - 88.4 2. Update dashboard UI (when release changed React)
  - 88.5 3. Pull and recreate (edge host)
  - 88.6 4. Ansible image-only (control machine)
  - 88.7 5. Validate
  - 88.8 Rollback
  - 88.9 Related
- 89 Central collection
- 90 OpenFDD RCx Central — collection
  - 90.1 Interval summary schema
  - 90.2 Edge export formats
  - 90.3 Collect today (rollup API)
  - 90.4 Tuning proposals
- 91 AHU faults
- 92 AHU faults
- 93 Standards crosswalk
- 94 Standards crosswalk
  - 94.1 Example
- 95 Backup and restore
- 96 Backup and restore
  - 96.1 Quick backup (edge host)
  - 96.2 Manual backup (SSH)
  - 96.3 What to protect
  - 96.4 Restore after a bad upgrade
  - 96.5 Safe cleanup
  - 96.6 Related
- 97 OpenFDD RCx Central API
- 98 OpenFDD RCx Central API
  - 98.1 Start
  - 98.2 Edge registry (browser auth)
  - 98.3 Analytics & RCx
  - 98.4 Validation (legacy collect workflow)
  - 98.5 RCx report body
- 99 VAV faults
- 100 VAV faults



- 101 Docker images (GHCR)
- 102 Docker images (GHCR)
  - 102.1 Images
  - 102.2 Tags
  - 102.3 Publish
  - 102.4 vs PyPI
- 103 RTU faults
- 104 RTU faults
- 105 Deployment validation
- 106 Deployment validation
  - 106.1 Edge host smoke (5 minutes)
  - 106.2 Browser checklist
  - 106.3 Control-machine insurance check
  - 106.4 LAN security smoke (optional)
  - 106.5 Arrow / FDD rules (3.0+)
  - 106.6 Related
- 107 Sensor and data quality
- 108 Sensor and data quality
  - 108.1 Catalog codes
  - 108.2 Default bounds (imperial, occupied)
  - 108.3 Communication quality
  - 108.4 Operator workflow
- 109 Logging and audit
- 110 Logging and audit
  - 110.1 Log types
  - 110.2 Audit record schema
    - 110.2.1 Auth events (industry-standard login trail)
    - 110.2.2 OT / commissioning events
    - 110.2.3 Secret redaction
  - 110.3 Integrator API (read logs in the UI stack)
  - 110.4 Retention and rotation (defaults)
  - 110.5 Docker edge: json-file caps (not journald for app audit)
  - 110.6 Local dev (run\_local.sh)
  - 110.7 Quick checks
  - 110.8 Comparison to typical web apps
  - 110.9 Related
- 111 GHCR image retention
- 112 GHCR image retention
  - 112.1 Policy (conservative)
  - 112.2 Dry-run first
  - 112.3 Protect Acme rollback tags
  - 112.4 Run locally
  - 112.5 GitHub Actions workflow
  - 112.6 Permissions
  - 112.7 Verify images still pull
  - 112.8 Rollback example
  - 112.9 Related issues
- 113 Fault Codes
- 114 Fault Codes
- 115 Operations
- 116 Operations
- 117 Portfolio

- 118 Portfolio (central desk)
- 119 Acme VAV/AHU rule guidance
- 120 Acme VAV/AHU FDD rule bundle
  - 120.1 Mixed manufacturers and units
  - 120.2 Rule matrix
    - 120.2.1 Phase 2+ (not enabled by default)
    - 120.2.2 Phase 3 rollups (future)
  - 120.3 Data model checks
  - 120.4 Tuning notes
  - 120.5 JSON API weather + BACnet OAT
  - 120.6 BACnet override scans
  - 120.7 Docker backup / recovery
- 121 Bench 5007 dual-source smoke
- 122 Bench 5007 dual-source smoke
  - 122.1 Run
  - 122.2 Paired semantic points
- 123 Bench 5007 long FDD smoke
- 124 Bench 5007 long FDD smoke
  - 124.1 Commands
  - 124.2 Key environment
  - 124.3 Fault confirmation window
  - 124.4 Artifacts
  - 124.5 Pass / fail
  - 124.6 Related
- 125 ACME live validation
- 126 ACME live validation
  - 126.1 Model export / import
  - 126.2 Run FDD (read-only)
  - 126.3 Reports
  - 126.4 Fault confirmation
  - 126.5 Safe rules on ACME
  - 126.6 Related
- 127 AI agent context
- 128 AI agent context
  - 128.1 1. Open-FDD in one paragraph
  - 128.2 2. Deployment mental model
  - 128.3 3. Source / driver modes
  - 128.4 4. Data model contract
  - 128.5 5. Rule execution contract
  - 128.6 6. Fault confirmation window
  - 128.7 7. Validation modes
  - 128.8 8. Security rules
  - 128.9 9. Where to look
- 129 LAN hardening
- 130 LAN hardening
  - 130.1 Auth and bind
  - 130.2 Network exposure
  - 130.3 Rule Lab
  - 130.4 TLS and LAN security scans
  - 130.5 Ollama on the edge
- 131 ZAP baseline (local bench)

- 132 ZAP baseline (local bench)
  - 132.1 Run
  - 132.2 Expected results (HTTP bench mode)
  - 132.3 Header ownership
  - 132.4 Authenticated scan (later)
  - 132.5 After fixes
- 133 TLS and certificates
- 134 TLS and certificates
  - 134.1 Ownership
  - 134.2 Generate bench certs (local)
  - 134.3 ZAP / Nmap with TLS bench
  - 134.4 Renewal
  - 134.5 Enterprise CA on Caddy (edge)
  - 134.6 Caddy internal CA mode (lab)
  - 134.7 Secrets
- 135 Authenticated scanning (roadmap)
- 136 Authenticated scanning (roadmap)
  - 136.1 Current state (3.0.x)
  - 136.2 Why baseline is not enough
  - 136.3 Planned approach (contributors welcome)
  - 136.4 Manual authenticated ZAP today
  - 136.5 Related
- 137 Secrets and auth
- 138 Secrets and auth
  - 138.1 Files (gitignored)
  - 138.2 Required variables (production)
  - 138.3 GHCR pulls
  - 138.4 Backup
- 139 BACnet write allowlists
- 140 BACnet write allowlists
- 141 Linux host hardening
- 142 Linux host hardening
  - 142.1 Scope
  - 142.2 Quick operator workflow
  - 142.3 Ubuntu version and EOL
  - 142.4 Kernel and package updates
  - 142.5 Host Python (pip, wheel, pyOpenSSL)
  - 142.6 Ollama — no LAN exposure
  - 142.7 OpenSSH Terrapin (CVE-2023-48795)
  - 142.8 ICMP timestamp (Low)
  - 142.9 TLS and certificate scans
  - 142.10 Maintainer security audits (CI parity)
- 143 Tenable / Nessus remediation
- 144 Tenable / Nessus remediation
  - 144.1 Prioritized action plan
  - 144.2 Finding classification table
  - 144.3 Operator commands (edge VM)
    - 144.3.1 Validate (read-only)
    - 144.3.2 Remediate host (maintenance window)
    - 144.3.3 Remediate Open-FDD containers
    - 144.3.4 Rescan
  - 144.4 Ollama unauthenticated access (Critical)

- 144.5 pyOpenSSL and pip in containers
- 144.6 TLS — expected vs production
- 144.7 Residual findings after remediation
- 144.8 Notes for IT / security team
- 144.9 Repo-owned fixes (this document's release)
- 145 AI agents
- 146 AI agents and MCP
- 147 Security
- 148 Security
- 149 API routes
- 150 API routes
  - 150.1 Health & audit
  - 150.2 Auth
  - 150.3 Rules & FDD
  - 150.4 Rule Lab playground
  - 150.5 BRICK / data model
  - 150.6 Rule Lab (dashboard)
  - 150.7 Bench validation (read-only)
  - 150.8 Niagara baskStream
  - 150.9 BACnet
  - 150.10 Faults (check-engine)
  - 150.11 Timeseries & sites
  - 150.12 Building & host
  - 150.13 Local agent (Ollama)
  - 150.14 Modbus
  - 150.15 JSON API (HTTP/HTTPS)
  - 150.16 PyPI package (separate)
  - 150.17 Errors
- 151 Python package
- 152 Python package (open-fdd)
  - 152.1 Modules
  - 152.2 When to use package-only
  - 152.3 Versioning
  - 152.4 Offline Arrow example
- 153 Configuration reference
- 154 Configuration reference
  - 154.1 Bridge / auth
  - 154.2 BACnet
  - 154.3 Docker edge
- 155 Glossary
- 156 Glossary
- 157 Workspace env files
- 158 Workspace env files
  - 158.1 Ansible / edge extras
  - 158.2 Related docs
- 159 Appendix
- 160 Appendix
- 161 GL36 lab site (Acme)
- 162 GL36 lab site (Acme)
  - 162.1 BACnet poll scope
  - 162.2 Example FDD rules (Arrow)
    - 162.2.1 Duct static pressure high (AHU)

- 162.2.2 SAT too cold vs setpoint
  - 162.2.3 Site rule stubs in the repo
  - 162.2.4 Local OA-T vs web weather (JSON API)
- 162.3 Deploy / upgrade (Ansible, this lab only)
- 162.4 AI-assisted data modeling (lab workflow)
- 163 Examples & lab notes
- 164 Examples & lab notes
- 165 Paired FDD smoke (bench + Acme)
- 166 Paired FDD smoke
  - 166.1 What is tested
  - 166.2 Fault confirmation (5 minutes)
  - 166.3 Modes (hardcoded)
  - 166.4 Run
  - 166.5 Reports
  - 166.6 Pass criteria
- 167 Arrow-native runtime
- 168 Arrow-native runtime
  - 168.1 Authoring contract
  - 168.2 PyArrow-only policy (edge + Rule Lab)
  - 168.3 Package layout
- 169 Release process
- 170 Release process
  - 170.1 1. Prepare version
  - 170.2 2. Merge and tag
  - 170.3 3. PyPI Trusted Publishing setup (one-time)
  - 170.4 4. Verify release
  - 170.5 5. Manual workflow\_dispatch
  - 170.6 Package vs Docker
- 171 PyPI package
- 172 PyPI package (open-fdd)
  - 172.1 What it is
  - 172.2 What it is not
  - 172.3 Install
  - 172.4 CLI
  - 172.5 Rule contract
  - 172.6 Modules shipped on PyPI
  - 172.7 Examples
  - 172.8 Local validation
- 173 MCP server
- 174 Open-FDD MCP server
  - 174.1 Modes
  - 174.2 Transport
  - 174.3 Human approval
  - 174.4 Client examples
  - 174.5 Site registry example
  - 174.6 RCx Central (not MCP)
  - 174.7 Fault code labels (agents)
  - 174.8 Doc RAG refresh
  - 174.9 Module layout
- 175 Adr Datafusion Rust Ready Rule Backend
- 176 ADR: DataFusion as optional Rust-ready rule backend
  - 176.1 Status

- 176.2 Context
- 176.3 Decision
- 176.4 Why
- 176.5 Non-goals (this milestone)
- 176.6 Future path
- 176.7 Consequences
- 177 Adr Rust Ready Arrow Fdd Contract
- 178 ADR: Rust-ready Arrow FDD contract
  - 178.1 Status
  - 178.2 Context
  - 178.3 Decision
  - 178.4 Consequences
- 179 Bench Validation Agent
- 180 Bench validation agent (local benserver)
  - 180.1 Topology
  - 180.2 One-command validation
  - 180.3 API tools (bridge must be running)
  - 180.4 MCP / agent notes
  - 180.5 Overnight
- 181 Rcx Central Dash Agent
- 182 RCx Central Dash — agent guide
  - 182.1 Runtime stack (benserver / analyst workstation)
  - 182.2 Dash UI (2026-06)
  - 182.3 Edge parity (Data Model tab)
  - 182.4 BRICK equipment typing (common bug)
  - 182.5 Fault codes (stable labels)
  - 182.6 Central API routes (agent smoke)
  - 182.7 Tests
  - 182.8 PRs (typical split)
  - 182.9 Related
- 183 Rcx Central Overnight Patch Cycle
- 184 RCx Central overnight patch-cycle (agent runbook)
  - 184.1 Safety (non-negotiable)
  - 184.2 One patch cycle
  - 184.3 GH Actions
  - 184.4 RCx Central local
  - 184.5 Live ACME (gated)
  - 184.6 Try-out vs overnight
  - 184.7 Rev bump / deploy
  - 184.8 Branch cleanup after merge
- 185 Bench Bacnet Vs Niagara
- 186 BACnet direct vs Niagara bench validation
  - 186.1 Topology
  - 186.2 Quick start
  - 186.3 Mapping config
  - 186.4 Brick model
  - 186.5 Normalized data model
  - 186.6 Poll intervals
  - 186.7 Overnight smoke
  - 186.8 Read-only rules
- 187 Dashboard Analytics

- 188 Dashboard analytics
  - 188.1 Pages
  - 188.2 API (read-only)
  - 188.3 Fault hours
  - 188.4 Theme
  - 188.5 Tests
- 189 Datafusion Sql Rules
- 190 DataFusion SQL rules (optional)
  - 190.1 When to use SQL vs PyArrow
  - 190.2 Rule metadata
  - 190.3 Safety
  - 190.4 Rules Lab
- 191 Dashboard Rcx Makeover Inventory
- 192 Dashboard + RCx Makeover — Inventory
  - 192.1 What exists
  - 192.2 What partially exists
  - 192.3 What is missing (this branch targets)
  - 192.4 Files inspected
  - 192.5 Recommended implementation path
  - 192.6 Known risks
- 193 Rcx Central Current State
- 194 OpenFDD RCx Central — current state
  - 194.1 Product names
  - 194.2 Entrypoints
  - 194.3 Central API
  - 194.4 Edge REST (read-only, shared with Data Model tab)
  - 194.5 Dash UI tabs
  - 194.6 Pre-Docker owner checklist (benserver :8050)
  - 194.7 BRICK model notes (agents)
  - 194.8 Fault codes
  - 194.9 Agent docs
  - 194.10 Tests
  - 194.11 Remaining gaps
- 195 Short Lookup
- 196 Fault code short descriptions
- 197 Niagara Baskstream Connector
- 198 Niagara baskStream connector (read-only)
  - 198.1 Read-only posture
  - 198.2 Niagara station setup
    - 198.2.1 Health check behavior
  - 198.3 Open-FDD configuration
    - 198.3.1 ORD encoding
    - 198.3.2 Allowed Origins
  - 198.4 Dependencies
  - 198.5 Polling
  - 198.6 Bench reference (benserver)
- 199 Data Model
- 200 Data Model — sync and FDD queries
  - 200.1 Explorer (BACnet poll sync)
  - 200.2 FDD query presets
- 201 Host Resources
- 202 Host Resources — Easy Pooge reset

- 203 Trend Plots
- 204 Trend plots
  - 204.1 HTTP / Tailscale
  - 204.2 Troubleshooting
- 205 Cloud Exporter
- 206 Cloud exporter sidecar
  - 206.1 Enable (edge compose)
  - 206.2 Security
- 207 Overnight Bench Smoke
- 208 Overnight bench smoke test
  - 208.1 Command
  - 208.2 Each checkpoint
  - 208.3 Final deliverable
  - 208.4 GH Actions loop
- 209 Ai Agent Workflow
- 210 AI agent workflow vs runtime use
  - 210.1 Three agent surfaces
  - 210.2 Runtime analyst (no git clone)
  - 210.3 Developer / coding agent (git working tree)
  - 210.4 Model / SPARQL validation (Acme)
  - 210.5 Secrets
  - 210.6 Model routing (test triage)
- 211 Docker Desktop Windows
- 212 RCx Central on Docker Desktop (Windows)
  - 212.1 Prerequisites
  - 212.2 Run
  - 212.3 Volumes
  - 212.4 Tailscale networking
  - 212.5 Smoke test
- 213 Edge Auth
- 214 Edge authentication (RCx Central)
  - 214.1 Browser flow
  - 214.2 API
  - 214.3 Auth types
  - 214.4 Rejected URLs
- 215 Index
- 216 OpenFDD RCx Central
  - 216.1 Quick start (host Python)
  - 216.2 Docker Desktop
  - 216.3 Pages
  - 216.4 Safety
- 217 Model Queries
- 218 Model queries (RCx Central → OpenFDD Edge + local TTL mirror)
  - 218.1 Primary endpoints
  - 218.2 Trends (matplotlib preview + fault overlays)
  - 218.3 Analytics
  - 218.4 RCx Central API wrappers
  - 218.5 FDD preset ids (Edge parity)
  - 218.6 BRICK typing (do not mis-count AHUs/VAVs)
- 219 Overnight Acme Validation



- 220 Overnight ACME validation via RCx Central
  - 220.1 Prerequisites
  - 220.2 Try-out (2 cycles, no sleep)
  - 220.3 Overnight (10 cycles, 2 h spacing)
  - 220.4 Read-only scope
- 221 Patch Cycle Release Process
- 222 Patch-cycle release process
  - 222.1 Authorization flags
  - 222.2 Image bump (when authorized)
- 223 Report Builder
- 224 RCx Report Builder (RCx Central)
  - 224.1 Flow
  - 224.2 Fault overlays
  - 224.3 DOCX
- 225 Rcx Report Builder
- 226 RCx Report Builder
  - 226.1 Workflow
  - 226.2 Implementation
  - 226.3 Safety
  - 226.4 Dependencies
  - 226.5 Central vs edge
- 227 3.1.2
- 228 Open-FDD 3.1.2
  - 228.1 Highlights
  - 228.2 Known warnings
  - 228.3 Validation
- 229 3.1.3
- 230 Open-FDD 3.1.3
  - 230.1 Patch fixes
  - 230.2 GHCR tags
  - 230.3 Validation

*Generated June 16, 2026*

---

# 1 Home

## 2 Open-FDD

Open-FDD is an open-source **building edge platform** for BACnet integration, feather historian storage, Arrow-native fault detection, and operator dashboards — designed for **trusted LAN / OT edge** deployment behind Caddy, not direct public internet exposure.

## 2.1 Who it is for

| Role                            | What you get                                                              |
|---------------------------------|---------------------------------------------------------------------------|
| <b>IT / controls engineer</b>   | Pull three GHCR images, log in, poll BACnet, view faults                  |
| <b>Commissioning integrator</b> | Discover points, bind BRICK model, pin FDD rules in Rule Lab              |
| <b>Developer / maintainer</b>   | Extend the Operator Bridge, publish images, run CI and LAN security scans |

## 2.2 Start here

| Path              | Go to                                                     |
|-------------------|-----------------------------------------------------------|
| <b>Try it</b>     | [Quick Start — Run with Docker]({{ "/quick-start/docker/" |
| <b>Operate it</b> | <u>Updating the stack</u> ({{ "/quick-start/updates/"     |
| <b>Develop it</b> | <u>Developer Guide</u> ({{ "/developer/"                  |

## 2.3 v3 edge stack

Three primary containers on a Linux edge host:

| Container                 | Role                                                    |
|---------------------------|---------------------------------------------------------|
| <b>openfdd-bridge</b>     | Operator Bridge — API, dashboard, feather <b>ingest</b> |
| <b>openfdd-commission</b> | BACnet discover/read/write + <b>poll loop</b>           |
| <b>openfdd-mcp-rag</b>    | Doc-search sidecar for agent tools                      |

Host **Caddy** on :80 or :443 is the normal LAN front door. BACnet UDP :47808 is bound by **commission** only — the bridge ingests samples.csv into the feather historian.

Details: Containers({{ "/architecture/containers/" | relative\_url }}) · Deployment modes({{ "/architecture/deployment-modes/" | relative\_url }})

**LAN / OT edge only.** Do not expose the Operator Bridge to the public internet without TLS, strong auth, and site review. BACnet writes are disabled by default — see [BACnet write guard]({{ "/security/bacnet-writes/" | relative\_url }}).

## 2.4 Distribution

| Channel                                                                                  | Use when                                           |
|------------------------------------------------------------------------------------------|----------------------------------------------------|
| <b>GHCR</b> <a href="https://ghcr.io/bbartling/openfdd-">ghcr.io/bbartling/openfdd-*</a> | Production or trial edge deploy                    |
| <b>This repository</b>                                                                   | Custom builds, commissioning, Rule Lab development |
| <b>PyPI</b> <a href="https://pypi.org/project/open-fdd/">open-fdd</a>                    | Arrow runtime lint/test without full UI            |

## 2.5 License

MIT — see repository LICENSE.

# 3 Open-FDD Central / Portfolio / RCx reports

# 4 Open-FDD Central / Portfolio / RCx reports

Planning doc for the next feature branch: [feature/openfdd-central-portfolio-rcx-reports](#).

## 4.1 Boundary

| Layer                   | Role                                                                           |
|-------------------------|--------------------------------------------------------------------------------|
| <b>Open-FDD Edge</b>    | Lives at the building. BACnet, local FDD, model/trends/faults, safe REST APIs. |
| <b>Open-FDD Central</b> | Multi-building desk over Tailscale/VPN. No direct BACnet. No equipment         |

| Layer | Role                                                      |
|-------|-----------------------------------------------------------|
|       | commands. Validation jobs, RCx reports, portfolio health. |

Existing code to extend (do not rewrite):

- portfolio/collector/ — edge login, rollup fetch, CSV history
- portfolio/dash/ — multi-site Dash dashboard
- scripts/portfolio\_collect.py — collector CLI
- Edge APIs: /api/building/portfolio-rollup, /api/faults/status, /api/fdd/results

## 4.2 Incremental build order

1. **Edge registry** — extend portfolio/sites.json schema (name, base URL, site\_id, last check-in).
2. **Remote edge client** — reuse portfolio/collector/edge\_client.py; add validation job endpoints.
3. **Portfolio page** — edge health, BACnet/model/fault status, last check-in.
4. **One-off validation** — trigger read-only harness against one edge (acme\_overnight\_fdd\_validate pattern).
5. **24-hour validation planner** — dropdown intervals (2h / 6h / 12h), store cycle results.
6. **Fault-hour analytics** — elapsed fault hours from historian + FDD runs.
7. **Minimal DOCX generator** — python-docx + matplotlib (BytesIO charts).
8. **Report download endpoint** — query edges via REST, render .docx.
9. **Tests/fixtures** — deterministic edge mocks; no live BACnet in CI.

## 4.3 RCx report content (normalized roles)

- Elapsed fault hours by equipment/severity
- SAT vs setpoint, duct static vs setpoint
- Fan/motor runtime analytics
- Missing data warnings
- Duplicate/model warnings from edge audit APIs
- **Never** severity-only cards without equipment context

## 4.4 Prerequisites

- ACME validation PR merged and **3.0.33** deployed (equipment context on live FDD alerts).
- Follow-ups: FDD run-history equipment grouping, RTU role mapping (see [ACME validation follow-ups]({{ "/ops/acme-validation-follow-ups/" | relative\_url }})).

## 4.5 Safety

Central is read-only toward edges. No BACnet writes, commands, or BAS changes from Central.

# 5 Run with Docker images

# 6 Run with Docker images

Deploy Open-FDD on a Linux edge host using **three published GHCR images**. No git clone required.

## 6.1 Images

| GHCR image                           | Service    | Role                                                          |
|--------------------------------------|------------|---------------------------------------------------------------|
| ghcr.io/bbartling/openfdd-bridge     | bridge     | Operator<br>API,<br>dashboard,<br>historian<br>ingest         |
| ghcr.io/bbartling/openfdd-commission | commission | BACnet<br>discover/<br>read/write<br><b>and poll<br/>loop</b> |
| ghcr.io/bbartling/openfdd-mcp-rag    | mcp-rag    | Doc-search<br>sidecar                                         |

Edge hosts pull **latest** from GHCR (three images: bridge, commission, mcp-rag). The retired **openfdd-bacnet-poll** image is no longer published.

## 6.2 1. Install and validate Docker

```
sudo systemctl enable --now docker
sudo usermod -aG docker "$USER"
newgrp docker    # or log out/in

docker --version
docker compose version
docker ps
docker run --rm hello-world
```

Expect: active / enabled for docker, docker in groups, hello-world succeeds.

## 6.3 2. Bootstrap the site (one script)

Downloads docker-compose.yml, creates workspace/, generates auth.env.local, and sets BACNET\_BIND from the host LAN NIC (e.g. ens192 → 10.200.200.185/24:47808):

```
curl -fsSL -o /tmp/openfdd_edge_bootstrap.sh \
    https://github.com/bbartling/open-fdd/raw/refs/heads/master/
scripts/openfdd_edge_bootstrap.sh
bash /tmp/openfdd_edge_bootstrap.sh
```

Bootstrap + pull + start in one go:

```
bash /tmp/openfdd_edge_bootstrap.sh --start
```

**Auth** — bootstrap writes ~/open-fdd/workspace/auth.env.local (gitignored). This file holds the API signing secret and one username/password per role. Use it for your first dashboard login:

```
cat ~/open-fdd/workspace/auth.env.local
```

| Variable          | Role                                                    |
|-------------------|---------------------------------------------------------|
| OFDD_AUTH_SECRET  | Signs session tokens (do not share)                     |
| OFDD_OPERATOR_*   | Read-only operator                                      |
| OFDD_INTEGRATOR_* | <b>Default dashboard login</b> — commissioning + BACnet |
| OFDD_AGENT_*      | Agent / automation                                      |

Example shape (passwords are **random on first bootstrap only** — existing file is kept on re-runs):

```

OFDD_AUTH_SECRET=<random>
OFDD_OPERATOR_USER=operator
OFDD_OPERATOR_PASSWORD=<random>
OFDD_INTEGRATOR_USER=integrator
OFDD_INTEGRATOR_PASSWORD=<random>
OFDD_AGENT_USER=agent
OFDD_AGENT_PASSWORD=<random>

```

Sign in as **integrator** with the password from that file → First login and health check({{ "/quick-start/health-check/" | relative\_url }}).

| Action                                 | Touches auth.env.local?                                    |
|----------------------------------------|------------------------------------------------------------|
| First bash ...<br>bootstrap.sh --start | Creates file with random passwords (if missing)            |
| bash ...<br>bootstrap.sh --start again | <b>Keeps</b> your file — does not overwrite                |
| bash ...<br>bootstrap.sh --restart     | <b>Keeps</b> your file — only restarts bridge to reload it |
| nano<br>auth.env.local + --restart     | Uses <b>your</b> edited passwords                          |
| --force-auth                           | <b>Regenerates</b> random passwords (opt-in only)          |

The **bridge** container reads auth.env.local at startup only. After you change passwords (manual edit or --force-auth), recreate bridge (env files are not reloaded by restart alone):

```
cd ~/open-fdd && docker compose up -d --force-recreate bridge
```

Regenerate passwords and reload (stack already running):

```
bash /tmp/openfdd_edge_bootstrap.sh --force-auth --restart --show-secrets
```

Manual edit then reload:

```

nano ~/open-fdd/workspace/auth.env.local
bash /tmp/openfdd_edge_bootstrap.sh --restart

```

Options:

| Flag            | Purpose                                                                       |
|-----------------|-------------------------------------------------------------------------------|
| --start         | docker compose pull && up -d after layout; curls http://127.0.0.1:8765/health |
| --image-tag TAG | default latest                                                                |

| Flag              | Purpose                                                       |
|-------------------|---------------------------------------------------------------|
| --repo-ref BRANCH | branch for compose.edge.yml if not on master yet              |
| --force-auth      | regenerate auth.env.local (restarts bridge if stack is up)    |
| --restart         | recreate bridge to reload auth.env.local; curls /health after |
| --show-secrets    | print passwords at end (lab only)                             |

From a repo checkout: `./scripts/openfdd_edge_bootstrap.sh --start`

The script prints **BACnet NIC**, **BACNET\_BIND**, and file paths — **validate** `commission.env` before polling OT devices:

```
nano ~/open-fdd/workspace/bacnet/commissioning/commission.env
```

If you used `--start`, the stack is already up. Next step → First login and health check({{ `"/quick-start/health-check/"` | `relative_url` }}).

### 6.3.1 Manual start (optional)

Use this only if you ran bootstrap **without** `--start` and want to bring the stack up yourself:

```
cd ~/open-fdd
docker compose pull
docker compose up -d
docker compose ps
curl -sf http://127.0.0.1:8765/health && echo
```

Expected: **bridge**, **commission**, **mcp-rag** — all up. Then → First login and health check({{ `"/quick-start/health-check/"` | `relative_url` }}).

## 6.4 Long-term operation

### 6.4.1 Survive power cycles

All services use restart: `unless-stopped`. After reboot:

```
cd ~/open-fdd && docker compose ps
curl -sf http://127.0.0.1:8765/health
```

### 6.4.2 Start / stop / restart

```
cd ~/open-fdd
docker compose up -d # idempotent
```



```
docker compose stop
docker compose restart commission
docker compose logs -f --tail 100 commission
```

Never run `docker compose down -v` or delete workspace/ on a live site.

6.4.3 LAN access

Put **Caddy** (or nginx) on port 80 → 127.0.0.1:8765, or expose 8765 through the firewall.

6.5 Deployment choices

| Mode                       | Compose services                                     | When to use                                                                                               |
|----------------------------|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Standard<br>OT /<br>BACnet | bridge + commission + mcp-rag                        | Real buildings — commission polls BACnet into <code>samples.csv</code> ; bridge ingests to feather        |
| Non-BACnet /<br>demo       | bridge + mcp-rag (omit commission)                   | JSON API, Modbus-only, or lab without field BACnet — see <a href="#">Driver framework</a> ({{ “/drivers/” |
| TLS on<br>OT LAN           | Same stack + <b>host Caddy</b><br>OFDD_CADDY_MODE=tl | Encrypt browser↔edge; trust self-signed CA on operator PCs — not for public internet                      |

Do **not** run the retired `openfdd-bacnet-poll` image or legacy `openfdd-bacnet-poll` systemd unit alongside Docker **commission** — that double-polls BACnet.

6.6 Next steps

- [First login and health check](#)({{ “/quick-start/health-check/” | relative\_url }})
- [Updating the stack](#)({{ “/quick-start/updating/” | relative\_url }})
- `./scripts/openfdd_site_update.sh`

# 7 First login and health check

# 8 First login and health check

Quick checks after `docker compose up -d` on the edge host. Services use `restart: unless-stopped` — after a reboot, run `docker compose ps` once Docker is up (see [Run with Docker]({{ "/quick-start/docker/" | relative\_url }}#survive-power-cycles)).

## 8.1 Public health

```
curl -s http://127.0.0.1:8765/health
```

Through Caddy on port 80:

```
curl -s http://127.0.0.1/health
```

Expect `"ok": true` and `"auth_required": true` when auth is configured.

## 8.2 Login

1. Open the dashboard (`http://<host-lan-ip>/` or `:8765`).
2. Sign in with credentials from `workspace/auth.env.local`.

```
curl -s -X POST http://127.0.0.1:8765/api/auth/login \
-H 'Content-Type: application/json' \
-d '{"username":"integrator","password":"<from-auth-env>"}
```

## 8.3 Smoke checklist

| Check                          | Pass                                                                                            |
|--------------------------------|-------------------------------------------------------------------------------------------------|
| <code>docker compose ps</code> | bridge, commission, mcp-rag<br><b>Up</b>                                                        |
| <code>GET /health</code>       | 200, <code>ok: true</code>                                                                      |
| Login                          | Token returned                                                                                  |
| BACnet poll                    | <code>samples.csv</code> growing<br>( <code>tail -1 workspace/bacnet/polls/samples.csv</code> ) |
| Historian                      | Files under <code>workspace/data/feather_store/</code>                                          |

# 8.4 Stack detail (authenticated)

Integrator token required:

```
TOKEN=$(curl -sf -X POST http://127.0.0.1:8765/api/auth/login \
-H 'Content-Type: application/json' \
-d '{"username":"integrator","password":"..."}' | jq -r .token)
curl -sf http://127.0.0.1:8765/health/stack -H "Authorization:
Bearer $TOKEN" | jq .
```

Look for bacnet\_poll green/yellow in the services list.

# 8.5 Troubleshooting

| Symptom               | Fix                                                                                                  |
|-----------------------|------------------------------------------------------------------------------------------------------|
| 401 everywhere        | Configure workspace/<br>auth.env.local                                                               |
| Empty BACnet discover | Fix BACNET_BIND in<br>commission.env —<br><u>network setup</u> ({ { “/<br>bacnet/network-<br>setup/” |
| No poll rows          | Commission<br>container running?<br>points.csv has<br>enabled rows?                                  |
| LAN browser timeout   | Open firewall port 80<br>or 8765                                                                     |

Advanced probes (model graph, Rule Lab, compose profiles):  
[Developer health check]({ { “/developer/health-check/” |  
relative\_url } } ).

# 9 Quick Start

# 10 Quick Start

Run Open-FDD on a Linux edge host using **three GHCR Docker images**. No git clone on the host.

| Step | Page                                                      |
|------|-----------------------------------------------------------|
| 1    | <u>Run with Docker images</u> ({ { “/quick-start/docker/” |

| Step | Page                                                                             |
|------|----------------------------------------------------------------------------------|
| 2    | <a href="#">First login and health check</a> ({{ "/quick-start/health-check/" }} |
| 3    | <a href="#">Updating the stack</a> ({{ "/quick-start/updating/" }}               |

**One-liner bootstrap** (edge host):

```
curl -fsSL -o /tmp/openfdd_edge_bootstrap.sh \
https://github.com/bbartling/open-fdd/raw/refs/heads/master/
scripts/openfdd_edge_bootstrap.sh
bash /tmp/openfdd_edge_bootstrap.sh --start
```

**Prerequisites:** Linux, Docker enabled on boot, BACnet LAN if polling OT equipment.

**BACnet writes are disabled by default.** See [Write safety](#)({{ "/bacnet/write-safety/" | relative\_url }}).

**Stack:** bridge + commission (BACnet + poll) + mcp-rag. Details: [Containers](#)({{ "/architecture/containers/" | relative\_url }}).

## 11 Updating the stack

## 12 Updating the stack

Upgrade GHCR images on the edge host: **backup workspace/, pull new tags, recreate containers**. No git pull on the host.

### 12.1 Before you start

1. SSH to the edge host (cd ~/open-fdd).
2. Pick a tag from [GitHub Packages](#) — default is **latest**.
3. Short maintenance window (containers restart briefly; restart: unless-stopped brings them back after reboot).

### 12.2 Fetch helper scripts (no git clone)

If you bootstrapped with openfdd\_edge\_bootstrap.sh, scripts are already under ~/open-fdd/scripts/. Otherwise:

```
mkdir -p ~/open-fdd/scripts
curl -fsSL -o ~/open-fdd/scripts/openfdd_site_backup.sh \
https://github.com/bbartling/open-fdd/raw/refs/heads/master/
```

```
scripts/openfdd_site_backup.sh
    curl -fsSL -o ~/open-fdd/scripts/openfdd_site_update.sh \
        https://github.com/bbartling/open-fdd/raw/refs/heads/master/
scripts/openfdd_site_update.sh
    chmod +x ~/open-fdd/scripts/openfdd_site_*.sh
```

## 12.3 One-command upgrade (recommended)

```
cd ~/open-fdd
./scripts/openfdd_site_backup.sh
./scripts/openfdd_site_update.sh
```

Pulls **:latest** by default, verifies all three images exist on GHCR, recreates containers, and hits /health.

## 12.4 Step by step

### 12.4.1 1. Backup site state

```
cd ~/open-fdd
./scripts/openfdd_site_backup.sh
```

Archives workspace/ (feather, BACnet CSVs, model, auth) under ~/openfdd-backups/<timestamp>/.

Custom backup location:

```
BACKUP_ROOT=~/openfdd-backups/manual ./scripts/
openfdd_site_backup.sh
```

### 12.4.2 2. Verify images (optional)

```
export NEW_TAG="${NEW_TAG:-latest}"

docker manifest inspect ghcr.io/bbartling/openfdd-bridge:${
{NEW_TAG}} >/dev/null &&
    docker manifest inspect ghcr.io/bbartling/openfdd-commission:${
{NEW_TAG}} >/dev/null &&
    docker manifest inspect ghcr.io/bbartling/openfdd-mcp-rag:${
{NEW_TAG}} >/dev/null &&
    echo "All images exist for ${NEW_TAG}"
```

### 12.4.3 3. Pull and recreate

```
cd ~/open-fdd
./scripts/openfdd_site_update.sh
```

**Manual equivalent** (same as the script):

```
cd ~/open-fdd
docker compose pull
docker compose up -d --force-recreate
docker compose ps
curl -sf http://127.0.0.1:8765/health && echo
```

### 12.4.4 4. Verify

```
docker compose ps
curl -sf http://127.0.0.1:8765/health | jq .
tail -1 workspace/bacnet/polls/samples.csv
ls -lah workspace/data/feather_store/
```

→ [Health check]({{ "/quick-start/health-check/" | relative\_url }})

## 12.5 Dashboard UI updates

If the release changed the React UI, copy a new workspace/api/static/app/ build onto the host before or after the image pull. Image pull alone may not change the browser bundle hash.

## 12.6 Rollback

Restore docker-compose.yml from backup and pull again, or re-run bootstrap from a known-good master commit. GHCR keeps only the **three newest** versions per image; early sites should rely on **latest** plus workspace/ backups.

workspace/ is untouched by image-only upgrades.

## 12.7 Safe cleanup

```
docker image prune -f
```

Never run docker compose down -v, docker volume prune, or delete workspace/ on a live site.

## 12.8 Next steps

→ Run with Docker images({{ "/quick-start/docker/" | relative\_url }}) — bootstrap a new host  
→ [Health check]({{ "/quick-start/health-check/" | relative\_url }})

## 13 Local development

## 14 Local development

### 14.1 Clone and auth

```
git clone https://github.com/bbartling/open-fdd.git && cd open-fdd
cp workspace/auth.env.example workspace/auth.env.local
# Edit integrator/operator passwords and OFDD_AUTH_SECRET (32+
chars)
```

### 14.2 Start stack

```
./scripts/build_and_test.sh          # dashboard build + pytest
./scripts/docker_build.sh
./scripts/openfdd_stack.sh up
./scripts/stack_health_check.sh
```

| URL                        | Service         |
|----------------------------|-----------------|
| http://127.0.0.1:8765      | Bridge (direct) |
| http://127.0.0.1:8765/docs | OpenAPI         |

### 14.3 BACnet bench overlay (optional)

```
docker compose -f docker/compose.dev.yml -f docker/
compose.bench.yml up -d
# commission.env — set BACNET_BIND to your OT interface
```

## 14.4 Production-like local stack (Caddy, LAN scans)

Same shape as an Acme edge deploy: **prod React UI, Caddy on :80**, bridge on loopback **8765**, auth enabled. Use for OWASP ZAP or nmap from another machine on the LAN.

```
./scripts/pentest_production_stack.sh start    # build prod UI +  
Caddy + auth.pentest.local  
./scripts/pentest_production_stack.sh status  # bridge,  
commission, caddy, mcp-rag  
./scripts/pentest_production_stack.sh verify  # LAN IP, /health,  
ZAP target URL  
./scripts/pentest_production_stack.sh stop    # when done
```

Credentials: workspace/auth.pentest.local (generated on first start). ZAP target is usually `http://<lan-ip>/` (benserver example: `http://192.168.204.18/`). If LAN clients cannot reach :80, run `sudo ./scripts/open_lan_port.sh 80`.

See also `Health checks (developer)({{ "/developer/health-check/" | relative_url }})` for probe details and [ZAP baseline expectations] (`{{ "/security/zap-baseline/" | relative_url }}`) for accepted Medium/Low findings.

## 14.5 Docs preview

```
cd docs && bundle install && bundle exec jekyll serve  
# http://127.0.0.1:4000/open-fdd/
```

# 15 Build Docker images

# 16 Build Docker images

## 16.1 Local build

```
./scripts/docker_build.sh  
# Optional tar for air-gap:  
./scripts/docker_build.sh --save
```

Images: openfdd-bridge, openfdd-commission, openfdd-mcp-rag.



# 16.2 Publish to GHCR (maintainers)

GitHub Actions workflow **Publish Docker images to GHCR** — manual dispatch or git tag vX.Y.Z. Publishes **pyproject.toml** **version** (e.g. 3.1.3), minor alias (3.1), and **latest**, then prunes old GHCR versions.

Maintainer checklist:

- 1. Green CI on master
- 2. Run **Publish Docker images to GHCR** workflow (or push tag vX.Y.Z)
- 3. Verify ghcr.io/bbartling/openfdd-{bridge,commission,mcp-rag}:3.1.3 and :latest
- 4. Edge hosts: OPENFDD\_IMAGE\_TAG=3.1.3 ./scripts/upgrade\_edge\_ghcr.sh --limit <host>

Details live in .github/workflows/ and scripts/docker\_build.sh — not duplicated here.

# 17 Developer Guide

# 18 Developer Guide

Clone, build, test, and contribute to Open-FDD.

| Topic            | Page                                                                             |
|------------------|----------------------------------------------------------------------------------|
| Local setup      | <a href="#">Local development</a> ({{ "/"<br>developer/local-setup/"             |
| Docker builds    | <a href="#">Build Docker images</a> ({{ "/"<br>developer/docker-build/"          |
| Health checks    | <a href="#">Health checks (developer)</a><br>({{ "/"developer/health-<br>check/" |
| Tests & CI       | <a href="#">Tests and CI</a> ({{ "/"developer/<br>tests-ci/"                     |
| Security testing | [Release & LAN scan cycle]<br>({{ "/"developer/security-<br>testing/"            |
| PRs              | <a href="#">Contributing</a> ({{ "/"developer/<br>contributing/"                 |

AI-assisted contributors: see repository AGENTS.md for workspace layout, skill routing, and agent policy (one paragraph in public docs; details stay internal).

# 19 Tests and CI

# 20 Tests and CI

## 20.1 Local tests

```
./scripts/build_and_test.sh           # UI + workspace bridge
pytest
python3 -m pytest open_fdd/tests -q   # PyPI package tests
cd workspace/dashboard && npm test    # dashboard unit tests (if
configured)
```

## 20.2 CI (GitHub Actions)

| Workflow             | Trigger              | Checks                                                      |
|----------------------|----------------------|-------------------------------------------------------------|
| ci.yml               | PR / push            | pytest, bridge security audit, dashboard build, Jekyll docs |
| docs-pages.yml       | push master          | GitHub Pages site                                           |
| publish-open-fdd.yml | tag v* / open-fdd-v* | PyPI wheel (Trusted Publishing)                             |
| docker-publish.yml   | tag v* / manual      | GHCR bridge, commission, mcp-rag                            |

## 20.3 Docs build (same as CI)

```
cd docs && bundle exec jekyll build -d /tmp/open-fdd-site
```

Fix any template or link errors before opening a PR that touches docs/.

## 20.4 Security-related CI

ci.yml includes bridge security tests (tests/workspace\_bridge/test\_security.py). That complements — but does not replace — LAN [ZAP + Nmap scans]({{ "/security/zap-baseline/" | relative\_url }}) on each patch revision. See [Security testing cycle](#)({{ "/developer/security-testing/" | relative\_url }}).

## 21 DataFusion SQL rule recipes

## 22 DataFusion SQL recipes

Use when you need simple threshold, CASE WHEN, or SQL-readable rules. Requires pip install open-fdd[datafusion]. SQL runs **server-side only**.

### 22.1 Temperature high

```
SELECT
  *,
  "duct-t" > 80.0 AS fault
FROM telemetry
```

Quote hyphenated historian columns.

### 22.2 Humidity high

```
SELECT
  *,
  "oa-h" > 70.0 AS fault
FROM telemetry
```

### 22.3 Occupied + temperature

```
SELECT
  *,
  CASE
    WHEN occupied = true AND "stat_zn-t" > 76.0 THEN true
    ELSE false
```

END AS fault  
FROM telemetry

## 22.4 With fault confirmation

Add to rule config:

```
{"min_true_rows": 5, "poll_interval_s": 60}
```

PyArrow remains better for rolling windows, custom helpers, and ML prep. See [PyArrow recipes](#).

# 23 Contributing

# 24 Contributing

## 24.1 Expectations

- Open an issue or discuss large changes before big refactors.
- Keep PRs focused; include test or docs updates when behavior changes.
- Do not commit secrets (auth.env.local, Ansible secrets/\*.env.local, commissioning CSVs with real site data).
- Follow existing code style in workspace/api/ and workspace/dashboard/.

## 24.2 Pull request checklist

☐

Tests pass locally (./scripts/build\_and\_test.sh or scoped pytest)

☐

Docs updated if user-facing behavior changed

☐

docs/ Jekyll build succeeds

☐

No credentials in diff

☐

Auth/Caddy/header changes: re-run [pentest verify + LAN ZAP]({{ "/developer/security-testing/" | relative\_url }}) and

```
update [ZAP baseline]({{ "/security/zap-baseline/" |
relative_url }}) if expectations changed
```

## 24.3 Repository layout (short)

| Path                 | Contents                                 |
|----------------------|------------------------------------------|
| workspace/api/       | Operator Bridge (FastAPI)                |
| workspace/dashboard/ | React SPA                                |
| open_fdd/            | PyPI package (arrow_runtime, playground) |
| docker/              | Compose files and image contexts         |
| infra/ansible/       | Edge deploy playbooks                    |
| docs/                | This site (Jekyll + Just the Docs)       |

## 24.4 Docs links (GitHub Pages)

The site is a **project page** at <https://bbartling.github.io/open-fdd/> (baseurl: /open-fdd in docs/\_config.yml).

- **Do not** use `{% raw %}/path/{% endraw %}` in Markdown body text — it omits baseurl and 404s on GitHub Pages.
- **Do** use `[label]({{ "/section/page/" | relative_url }})` for internal links.
- After `bundle exec jekyll build`, run `python3 scripts/check_docs_internal_links.py --site _site` from the repo root.

## 24.5 Security issues

Do not file public issues for vulnerabilities. Contact the maintainer or use GitHub private security advisories.

## 24.6 Agent-assisted development

Cursor/Claude contributors: read `AGENTS.md` and `skills/` for automation boundaries. Public docs intentionally avoid agent playbook detail.

## 25 Health checks (developer)

## 26 Health checks (developer)

Extended validation for engineers with a **full git checkout** — CI, compose overlays, and pre-release gates.

IT operators on edge hosts should use [Quick Start — health check]({{ "/quick-start/health-check/" | relative\_url }}) and Deployment validation({{ "/ops/deployment-validation/" | relative\_url }}) instead.

### 26.1 Clone and local stack

```
git clone https://github.com/bbartling/open-fdd.git && cd open-fdd
cp workspace/auth.env.example workspace/auth.env.local #
loopback dev only
./scripts/docker_build.sh # or: ./
scripts/openfdd_stack.sh prod
./scripts/openfdd_stack.sh up
```

GHCR production pull (no local build):

```
OPENFDD_IMAGE_TAG=2026.06.07-edge ./scripts/openfdd_stack.sh prod
```

### 26.2 Stack health script

```
./scripts/stack_health_check.sh
OPENFDD_BASE_URL=http://192.168.204.18:8765 ./scripts/
stack_health_check.sh
./scripts/stack_health_check.sh --require-ollama
```

Probes: containers up, /health, /api/model/health, sites, tree, SPARQL graph (with integrator auth when configured).

### 26.3 Compose overlays

| Overlay                  | Use                                   |
|--------------------------|---------------------------------------|
| docker/compose.dev.yml   | Local dev bind-mount                  |
| docker/compose.bench.yml | Host-network<br>BACnet lab            |
| docker/compose.prod.yml  | GHCR images<br>instead of local build |

| Overlay                 | Use                                                             |
|-------------------------|-----------------------------------------------------------------|
| docker/compose.edge.yml | IT edge template<br>(copy to ~/open-fdd/<br>docker-compose.yml) |

```
docker compose -f docker/compose.dev.yml -f docker/
compose.bench.yml ps
docker compose -f docker/compose.dev.yml config --images
```

## 26.4 BACnet poll pipeline

```
tail -1 workspace/bacnet/polls/samples.csv
cat workspace/data/bacnet_ingest_state.json
docker compose logs --since 10m commission | grep -i poll
docker compose logs --since 10m bridge | grep -i ingest
```

## 26.5 pytest (bridge security / BACnet)

```
PYTHONPATH=workspace/api pytest tests/workspace_bridge/
test_security.py -q
```

## 26.6 Pentest production stack (LAN ZAP)

```
./scripts/pentest_production_stack.sh start
./scripts/pentest_production_stack.sh verify
```

Then from a **LAN workstation**, run the packaged scan:

- Windows: scripts/security/Run-OpenFddSecurityScan.ps1
- macOS/Linux: scripts/security/run\_openfdd\_security\_scan.sh

See [scripts/security/README.md](#), [Security — ZAP baseline]({{ "/security/zap-baseline/" | relative\_url }}), and the full [security testing cycle](#)({{ "/developer/security-testing/" | relative\_url }}).  
Host-side notes: workspace/deploy/PENTEST.md.

## 26.7 Post-deploy insurance (inventory hosts)

For automated checks against named inventory hosts, see [infra/ansible/scripts/post\\_deploy\\_check.sh](#) (maintainer tooling — not required for IT docker-pull sites).

# 27 Security testing cycle

# 28 Security testing cycle

How Open-FDD validates each **patch revision** before it ships to the live Acme HVAC bench — local automation, AI-assisted PR review, LAN passive scans, and edge post-deploy checks.

Related security docs: [ZAP baseline]({{ "/security/zap-baseline/" | relative\_url }}), TLS and certificates({{ "/security/tls-and-certs/" | relative\_url }}), LAN hardening({{ "/security/lan-hardening/" | relative\_url }}), Linux host hardening({{ "/security/linux-host-hardening/" | relative\_url }}), [Tenable remediation]({{ "/security/tenable-remediation/" | relative\_url }}), Authenticated scanning (roadmap)({{ "/security/authenticated-scanning/" | relative\_url }}).

## 28.1 Who owns what

| Concern                                         | Owner                                    | Where                                                                                                                    |
|-------------------------------------------------|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| HTTP security headers (CSP, X-Frame, COOP, ...) | <b>Bridge</b><br>middleware              | workspace/api/<br>openfdd_bridge/<br>security_headers.py                                                                 |
| TLS termination + HSTS                          | <b>Caddy</b>                             | workspace/deploy/<br>caddy/, Ansible<br>Caddyfile.j2                                                                     |
| OT LAN TLS certificates                         | <b>Site integrator / maintainer</b>      | ./scripts/<br>setup_caddy_certs.sh<br>→ workspace/deploy/<br>caddy/certs/;<br>Ansible /etc/<br>openfdd/caddy/ on<br>edge |
| Pentest credentials                             | Generated per stack                      | workspace/<br>auth.pentest.local<br>(never commit)                                                                       |
| LAN ZAP + Nmap scans                            | <b>Developer workstation</b> on test LAN | scripts/security/                                                                                                        |

See [TLS and certificate ownership]({{ "/security/tls-and-certs/" | relative\_url }}) for cert lifecycle.



## 28.2 Per-revision workflow

Typical cycle on a **control machine** (local dev) through a **lab edge host** (live bench):

flowchart LR

```
A[Local dev + pytest] --> B[Pentest stack verify]
B --> C[LAN ZAP + Nmap]
C --> D[PR + GitHub CI]
D --> E[CodeRabbit AI review]
E --> F[Merge + tag open-fdd-vX.Y.Z]
F --> G[GHCR + PyPI publish]
G --> H[Acme upgrade + post_deploy_check --full]
H --> I[Bump next patch locally]
```

### 28.2.1 1. Local functional tests

```
./scripts/build_and_test.sh
PYTHONPATH=workspace/api pytest tests/workspace_bridge/
test_security.py -q
infra/ansible/scripts/bench_5007_arrow_battery.sh # when FDD/
Arrow paths change
```

### 28.2.2 2. Pentest production stack (on Open-FDD host)

```
./scripts/pentest_production_stack.sh start
./scripts/pentest_production_stack.sh verify
```

Confirms Caddy :80 fronts the bridge, auth is enabled, BACnet writes disabled, and security headers are singular (no Caddy duplicates). Details: [Health checks — pentest]({{ "/developer/health-check/" | relative\_url }}#pentest-production-stack-lan-zap).

### 28.2.3 3. Host + exposure checks (on edge VM)

After IT Nessus scans or before Acme deploy sign-off:

```
./scripts/security/check_host_security.sh
./scripts/security/check_openfdd_exposure.sh --edge-ip <lan-ip>
```

Host remediation (dry-run first): ./scripts/security/remediate\_ubuntu\_host.sh

### 28.2.4 4. LAN security scan (from laptop)

Use the packaged scripts — do not hand-roll one-off docker/nmap commands unless debugging:

| Platform      | Command                                         |
|---------------|-------------------------------------------------|
| Windows       | .\scripts\security\Run-OpenFddSecurityScan.ps1  |
| macOS / Linux | ./scripts/security/run_openfdd_security_scan.sh |

Setup (Docker Desktop, Nmap): [scripts/security/README.md](#).

Target URL is usually **http://<lan-ip>/** (Caddy), not :8765.

Review `openfdd-security-report/90-quick-findings-summary.txt` and compare against [ZAP baseline expectations]({{ "/security/zap-baseline/" | relative\_url }}).

### 28.2.5 5. Pull request + automated review

- Open PR against master
- **GitHub Actions** (`ci.yml`): pytest, bridge security audit, dashboard build, Jekyll docs
- **CodeRabbit** (or similar AI PR review): style, security footguns, missing tests/docs

Do not merge with failing CI or unresolved duplicate-header regressions.

### 28.2.6 6. Release

```
git tag open-fdd-v3.0.4 # example
git push origin open-fdd-v3.0.4
```

Tag triggers GHCR :latest and PyPI publish workflows.

### 28.2.7 7. Lab edge validation

After pulling new images on a live bench VM:

```
OPENFDD_IMAGE_TAG=latest ./scripts/upgrade_edge_full.sh --limit
<inventory_host>
    infra/ansible/scripts/post_deploy_check.sh --limit
<inventory_host> --full
```

Re-run LAN ZAP + Nmap against the edge LAN IP if Caddy or auth behavior changed. Site-specific example: [GL36 lab note]({{ "/examples/acme-gl36-lab/" | relative\_url }}).

### 28.2.8 8. Next cycle

Bump patch version locally (`pyproject.toml`, `open_fdd/__init__.py`) on master for the next development round — do not tag until the next release is ready.

## 28.3 What each scan layer covers

| Layer                       | Tool                               | Depth                                                                            |
|-----------------------------|------------------------------------|----------------------------------------------------------------------------------|
| Unit / integration          | pytest, test_security.py           | Header middleware, auth gates                                                    |
| Stack smoke                 | pentest_production_stack.sh verify | Live headers, health, auth file                                                  |
| Passive web                 | ZAP baseline                       | Public routes, CSP, cookies, CORS                                                |
| Port exposure               | Nmap scoped                        | One host, known service ports                                                    |
| Authenticated API/dashboard | ZAP full + login                   | <b>Roadmap</b> — [Authenticated scanning]({{ "/security/authenticated-scanning/" |

Baseline ZAP does **not** exercise integrator login or protected /api/\* routes deeply.

## 28.4 Open-source contributors

The scripts/security/ folder is part of the public repo so any fork can run the same LAN smoke against their bench. PRs that change Caddy, auth, or security\_headers.py should update [ZAP baseline]({{ "/security/zap-baseline/" | relative\_url }}) and re-run the scan scripts.

## 29 Python recipes (full Arrow library)

## 30 Python recipes — full Arrow library

Copy-paste `apply_faults_arrow` modules for Rule Lab. Replaces the legacy pandas/YAML Expression Rule Cookbook.

- **No pandas** on the edge — PyArrow + `pyarrow.compute` only
- Set `fault_code` in Rule Lab metadata (letter codes or Grade-A AHU-ECON-001)
- Map historian column names in the data model / rule bindings (`ma-t`, `oa-t`, `stat_zn-t`, ...)

Quick patterns: `[Arrow recipes]({{ "/rule-cookbook/arrow-recipes/" | relative_url }}) · Index: Expression cookbook \(Arrow-native\)({{ "/rule-cookbook/expression-cookbook/" | relative_url }})`

---

### 30.1 Shared helpers (paste once per rule file)

```
import pyarrow as pa
import pyarrow.compute as pc

from open_fdd.arrow_runtime.cookbook import mixing_envelope_mask
from open_fdd.arrow_runtime.windows import (
    arrow_rolling_min,
    arrow_rolling_max,
    arrow_abs_diff,
)

def _norm_cmd(col: pa.ChunkedArray) -> pa.ChunkedArray:
    c = pc.cast(col, pa.float64())
    return pc.if_else(pc.greater(c, 1.0), pc.divide(c, 100.0), c)

def _col(table, name: str) -> pa.ChunkedArray:
    return pc.cast(table[name], pa.float64())
```

```
def _hypot_tol(a: float, b: float) -> float:
    return (a * a + b * b) ** 0.5
```

---

## 30.2 GL36 AHU Rules A-M

### 30.2.1 Rule A — duct static low at full fan speed (AHU-A)

```
FAULT_CODE = "AHU-A"
SP, SP_SP, FAN = "duct-static-pressure", "duct-static-pressure-
sp", "supply-fan-speed-command"
SP_MARGIN, DRV_HI, DRV_NEAR = 0.12, 0.93, 0.06
```

```
def apply_faults_arrow(table, cfg, context=None):
    sp, sp_sp = _col(table, SP), _col(table, SP_SP)
    fan = _norm_cmd(table[FAN])
    return pc.and_(
        pc.less(sp, pc.subtract(sp_sp, SP_MARGIN)),
        pc.greater_equal(fan, DRV_HI - DRV_NEAR),
    )
```

### 30.2.2 Rules B & C — blended air outside OAT/RAT band (AHU-D)

Uses mixing\_envelope\_mask (covers below-band and above-band).

```
FAULT_CODE = "AHU-D"
```

```
def apply_faults_arrow(table, cfg, context=None):
    return mixing_envelope_mask(
        table,
        {**cfg, "mixing_tol": 1.15, "normalize_cmd_percent": 1},
        mat_col="ma-t",
        oat_col="oa-t",
        rat_col="ra-t",
        fan_col="supply-fan-speed-command",
    )
```

### 30.2.3 Rule D — discharge cold when heating commanded (AHU-B)

```
FAULT_CODE = "AHU-B"
BLEND_TOL, SAT_TOL, FAN_DT = 1.15, 1.15, 0.55
```

```

def apply_faults_arrow(table, cfg, context=None):
    mat, sat = _col(table, "ma-t"), _col(table, "sa-t")
    valve = _norm_cmd(table["heating-valve-command"])
    fan = _norm_cmd(table["supply-fan-speed-command"])
    rhs = pc.add(pc.subtract(mat, BLEND_TOL), FAN_DT)
    cold = pc.less_equal(pc.add(sat, SAT_TOL), rhs)
    return pc.and_(cold, pc.greater(valve, 0.01), pc.greater(fan,
0.01))

```

### 30.2.4 Rule E — SAT too low with full heating (AHU-C)

```

FAULT_CODE = "AHU-C"
SUPPLY_ERR = 1.0

```

```

def apply_faults_arrow(table, cfg, context=None):
    sat = _col(table, "sa-t")
    sp = _col(table, "sa-t-sp") if "sa-t-sp" in
table.column_names else _col(table, "supply-air-temperature-
setpoint")
    valve = _norm_cmd(table["heating-valve-command"])
    fan = _norm_cmd(table["supply-fan-speed-command"])
    return pc.and_(
        pc.less(sat, pc.subtract(sp, SUPPLY_ERR)),
        pc.greater(valve, 0.9),
        pc.greater(fan, 0.01),
    )

```

### 30.2.5 Rule F — SAT/MAT mismatch in economizer mode (AHU-E)

```

FAULT_CODE = "AHU-E"
FAN_DT, BLEND_TOL, SAT_TOL, ECON_MIN = 0.55, 1.15, 1.15, 0.12
TOL = _hypot_tol(SAT_TOL, BLEND_TOL)

```

```

def apply_faults_arrow(table, cfg, context=None):
    mat, sat = _col(table, "ma-t"), _col(table, "sa-t")
    damper = _norm_cmd(table["oa-damper-command"])
    cool = _norm_cmd(table["cooling-valve-command"])
    mismatch = pc.greater(pc.abs(pc.subtract(pc.subtract(sat,
FAN_DT), mat)), TOL)
    econ = pc.and_(pc.greater(damper, ECON_MIN), pc.less(cool,
0.1))
    return pc.and_(mismatch, econ)

```

### 30.2.6 Rule G — ambient too warm for free cooling (AHU-E)

```
FAULT_CODE = "AHU-E"  
OAT_TOL, FAN_DT, SAT_TOL, ECON_MIN = 1.15, 0.55, 1.15, 0.12
```

```
def apply_faults_arrow(table, cfg, context=None):  
    oat = _col(table, "oa-t")  
    sp = _col(table, "sa-t-sp")  
    damper = _norm_cmd(table["oa-damper-command"])  
    cool = _norm_cmd(table["cooling-valve-command"])  
    warm = pc.greater(oat, pc.add(pc.subtract(sp, FAN_DT),  
SAT_TOL - OAT_TOL))  
    econ = pc.and_(pc.greater(damper, ECON_MIN), pc.less(cool,  
0.1))  
    return pc.and_(warm, econ)
```

### 30.2.7 Rule H — OAT/MAT mismatch (econ + mech cooling) (AHU-E)

```
FAULT_CODE = "AHU-E"  
OAT_TOL, BLEND_TOL = 1.15, 1.15  
TOL = _hypot_tol(OAT_TOL, BLEND_TOL)
```

```
def apply_faults_arrow(table, cfg, context=None):  
    oat, mat = _col(table, "oa-t"), _col(table, "ma-t")  
    cool = _norm_cmd(table["cooling-valve-command"])  
    damper = _norm_cmd(table["oa-damper-command"])  
    return pc.and_(  
        pc.greater(pc.abs(pc.subtract(mat, oat)), TOL),  
        pc.greater(cool, 0.01),  
        pc.greater(damper, 0.9),  
    )
```

### 30.2.8 Rule I — OAT/MAT mismatch (economizer only) (AHU-E)

```
FAULT_CODE = "AHU-E"  
TOL = _hypot_tol(1.15, 1.15)
```

```
def apply_faults_arrow(table, cfg, context=None):  
    oat, mat = _col(table, "oa-t"), _col(table, "ma-t")  
    damper = _norm_cmd(table["oa-damper-command"])  
    return  
pc.and_(pc.greater(pc.abs(pc.subtract(mat, oat)), TOL),  
pc.greater(damper, 0.9))
```

### 30.2.9 Rule J — discharge above blended in cooling (AHU-B)

```
FAULT_CODE = "AHU-B"  
FAN_DT, BLEND_TOL, SAT_TOL, ECON_MIN = 0.55, 1.15, 1.15, 0.12  
TOL = _hypot_tol(SAT_TOL, BLEND_TOL)
```

```
def apply_faults_arrow(table, cfg, context=None):  
    sat, mat = _col(table, "sa-t"), _col(table, "ma-t")  
    cool = _norm_cmd(table["cooling-valve-command"])  
    damper = _norm_cmd(table["oa-damper-command"])  
    hot = pc.greater(sat, pc.add(mat, pc.add(TOL, FAN_DT)))  
    mode_a = pc.and_(pc.greater(damper, 0.9), pc.greater(cool,  
0.0))  
    mode_b = pc.and_(pc.less_equal(damper, ECON_MIN),  
pc.greater(cool, 0.9))  
    return pc.and_(hot, pc.or_(mode_a, mode_b))
```

### 30.2.10 Rule K — discharge above setpoint in full cooling (AHU-C)

```
FAULT_CODE = "AHU-C"  
SAT_TOL, ECON_MIN = 1.15, 0.12
```

```
def apply_faults_arrow(table, cfg, context=None):  
    sat = _col(table, "sa-t")  
    sp = _col(table, "sa-t-sp")  
    cool = _norm_cmd(table["cooling-valve-command"])  
    damper = _norm_cmd(table["oa-damper-command"])  
    high = pc.greater(sat, pc.add(sp, SAT_TOL))  
    full = pc.or_(  
        pc.and_(pc.greater(damper, 0.9), pc.greater(cool, 0.9)),  
        pc.and_(pc.less_equal(damper, ECON_MIN), pc.greater(cool,  
0.9)),  
    )  
    return pc.and_(high, full)
```

### 30.2.11 Rule L — cooling coil $\Delta T$ when inactive (CH-c)

```
FAULT_CODE = "CH-C"  
ENTER_TOL, LEAVE_TOL, ECON_MIN = 1.15, 1.15, 0.12  
TOL = _hypot_tol(ENTER_TOL, LEAVE_TOL)
```

```
def apply_faults_arrow(table, cfg, context=None):  
    ent = _col(table, "cooling-coil-entering-air-temperature")  
    leave = _col(table, "cooling-coil-leaving-air-temperature")
```



```

        heat = _norm_cmd(table["heating-valve-command"])
        cool = _norm_cmd(table["cooling-valve-command"])
        damper = _norm_cmd(table["oa-damper-command"])
        drop = pc.greater(pc.subtract(ent, leave), TOL)
        mode_a = pc.and_(pc.greater(heat, 0), pc.equal(cool, 0),
pc.less_equal(damper, ECON_MIN))
        mode_b = pc.and_(pc.equal(heat, 0), pc.equal(cool, 0),
pc.greater(damper, ECON_MIN))
        return pc.and_(drop, pc.or_(mode_a, mode_b))

```

### 30.2.12 Rule M — heating coil $\Delta T$ when inactive (AHU-B)

```

FAULT_CODE = "AHU-B"
ENTER_TOL, LEAVE_TOL, FAN_DT, ECON_MIN = 1.15, 1.15, 0.55, 0.12
TOL = _hypot_tol(ENTER_TOL, LEAVE_TOL) + FAN_DT

```

```

def apply_faults_arrow(table, cfg, context=None):
    ent = _col(table, "heating-coil-entering-air-temperature")
    leave = _col(table, "heating-coil-leaving-air-temperature")
    heat = _norm_cmd(table["heating-valve-command"])
    cool = _norm_cmd(table["cooling-valve-command"])
    damper = _norm_cmd(table["oa-damper-command"])
    rise = pc.greater(pc.subtract(leave, ent), TOL)
    m1 = pc.and_(pc.equal(heat, 0), pc.equal(cool, 0),
pc.greater(damper, ECON_MIN))
    m2 = pc.and_(pc.equal(heat, 0), pc.greater(cool, 0),
pc.greater(damper, 0.9))
    m3 = pc.and_(pc.equal(heat, 0), pc.greater(cool, 0),
pc.less_equal(damper, ECON_MIN))
    return pc.and_(rise, pc.or_(m1, m2, m3))

```

---

## 30.3 Starter pack (VAV / AHU baseline)

### 30.3.1 01\_vav\_zone\_temp\_bounds\_occupied (VAV-C)

```

from open_fdd.arrow_runtime.cookbook import sensor_bounds_mask,
_unoccupied_mask

```

```

FAULT_CODE = "VAV-C"

```

```

def apply_faults_arrow(table, cfg, context=None):
    occupied = pc.invert(_unoccupied_mask(table, cfg))
    oob = sensor_bounds_mask(table, "zone_temp", cfg)
    return pc.and_(oob, occupied)

```

### 30.3.2 02\_vav\_zone\_temp\_flatline\_occupied (VAV-C)

```
from open_fdd.arrow_runtime.cookbook import sensor_flatline_mask,  
_unoccupied_mask
```

```
FAULT_CODE = "VAV-C"
```

```
def apply_faults_arrow(table, cfg, context=None):  
    occupied = pc.invert(_unoccupied_mask(table, cfg))  
    flat = sensor_flatline_mask(table, "zone_temp", cfg)  
    return pc.and_(flat, occupied)
```

### 30.3.3 03\_vav\_damper\_command\_extreme\_flatline (VAV-D)

```
from open_fdd.arrow_runtime.cookbook import flatline_1h_mask
```

```
FAULT_CODE = "VAV-D"
```

```
DAMPER = "damper-position-command"
```

```
def apply_faults_arrow(table, cfg, context=None):  
    stuck = flatline_1h_mask(table, {**cfg, "column": DAMPER})  
    wide = pc.greater(_norm_cmd(table[DAMPER]), 0.975)  
    return pc.and_(stuck, wide)
```

### 30.3.4 04\_ahu\_runtime\_outside\_schedule (BLD-C)

Use run-hours analytics script + schedule compare; boolean mask:

```
from open_fdd.arrow_runtime.cookbook import _fan_on_mask,  
_unoccupied_mask
```

```
FAULT_CODE = "BLD-C"
```

```
def apply_faults_arrow(table, cfg, context=None):  
    return pc.and_( _fan_on_mask(table, cfg),  
_unoccupied_mask(table, cfg))
```

### 30.3.5 05 / 06 / 07 — see Rules A, sensor\_bounds\_mask on sa-t / sensor\_flatline\_mask on sa-t

---

## 30.4 Economizer starters

### 30.4.1 ahu\_econ\_100oa\_temp\_tracking\_fault (AHU-E)

```
FAULT_CODE = "AHU-E"
DAMPER_FULL, FAN_ON, TRACK_TOL, FAN_HEAT = 0.95, 0.5, 2.0, 0.5

def apply_faults_arrow(table, cfg, context=None):
    oat, mat, sat = _col(table, "oa-t"), _col(table, "ma-t"),
    _col(table, "sa-t")
    damper = _norm_cmd(table["oa-damper-command"])
    fan = _col(table, "supply-fan-status")
    econ = pc.and_(pc.greater(fan, FAN_ON),
pc.greater_equal(damper, DAMPER_FULL))
    mat_bad = pc.greater(pc.abs(pc.subtract(mat, oat)), TRACK_TOL)
    sat_bad = pc.greater(pc.abs(pc.subtract(pc.subtract(sat,
FAN_HEAT), mat))), TRACK_TOL)
    return pc.and_(econ, pc.or_(mat_bad, sat_bad))
```

### 30.4.2 ahu\_mech\_cooling\_when\_free\_cooling\_available (AHU-E)

```
FAULT_CODE = "AHU-E"
MARGIN, DAMPER_LOW, COOL_ON = 2.0, 0.50, 0.10

def apply_faults_arrow(table, cfg, context=None):
    oat = _col(table, "oa-t")
    sp = _col(table, "sa-t-sp")
    damper = _norm_cmd(table["oa-damper-command"])
    cool = _norm_cmd(table["cooling-valve-command"])
    free = pc.less_equal(oat, pc.subtract(sp, MARGIN))
    return pc.and_(free, pc.less(damper, DAMPER_LOW),
pc.greater_equal(cool, COOL_ON))
```

### 30.4.3 ahu\_oa\_damper\_excess\_open\_extreme\_ambient (AHU-E)

```
FAULT_CODE = "AHU-E"
DAMPER_MIN, HI_OAT, LO_OAT = 0.50, 70.0, 40.0

def apply_faults_arrow(table, cfg, context=None):
    oat = _col(table, "oa-t")
    damper = _norm_cmd(table["oa-damper-command"])
    extreme = pc.or_(pc.greater_equal(oat, HI_OAT),
pc.less_equal(oat, LO_OAT))
    return pc.and_(pc.greater_equal(damper, DAMPER_MIN), extreme)
```

---

## 30.5 VAV zones

### 30.5.1 zone\_reheat\_warm\_ambient (VAV-A)

```
FAULT_CODE = "VAV-A"
T_CUTOFF, REHEAT_MIN = 78.0, 0.52

def apply_faults_arrow(table, cfg, context=None):
    oat = _col(table, "oa-t")
    reheat = _norm_cmd(table["reheat-valve-command"])
    return pc.and_(pc.greater(oat, T_CUTOFF), pc.greater(reheat,
REHEAT_MIN))
```

### 30.5.2 zone\_damper\_valve\_full\_open (VAV-D)

```
FAULT_CODE = "VAV-D"
FULL_OPEN, ROLL = 97.5, 105

def apply_faults_arrow(table, cfg, context=None):
    d = _norm_cmd(table["damper-position-command"])
    hi = pc.greater(d, FULL_OPEN)
    rmin = arrow_rolling_min(d, ROLL)
    return pc.and_(hi, pc.greater(rmin, FULL_OPEN))
```

### 30.5.3 Zone / IAQ bounds

Use `sensor_bounds_mask(table, "zone_temp", cfg)` or `sensor_bounds_mask` with CO<sub>2</sub> profile — see [sensor validation]({{ "/rule-cookbook/expression-cookbook/" | relative\_url }}#sensor-validation-bounds-flatline-rate-of-change).

---

## 30.6 Central plant

### 30.6.1 dp\_below\_sp\_pump\_max (CH-E)

```
FAULT_CODE = "CH-E"
DP_MARGIN, PMP_HI, PMP_NEAR = 2.2, 0.93, 0.06

def apply_faults_arrow(table, cfg, context=None):
    dp = _col(table, "differential-pressure")
    sp = _col(table, "differential-pressure-sp")
    pump = _norm_cmd(table["pump-speed-command"])
    return pc.and_(
        pc.less(dp, pc.subtract(sp, DP_MARGIN)),
```

```

        pc.greater_equal(pump, PMP_HI - PMP_NEAR),
    )

```

### 30.6.2 flow\_high\_pump\_max (CH-B)

```

FAULT_CODE = "CH-B"
FLOW_HI, PMP_HI, PMP_NEAR = 1100.0, 0.93, 0.06

```

```

def apply_faults_arrow(table, cfg, context=None):
    flow = _col(table, "water-flow")
    pump = _norm_cmd(table["pump-speed-command"])
    return pc.and_(pc.greater(flow, FLOW_HI),
pc.greater_equal(pump, PMP_HI - PMP_NEAR))

```

### 30.6.3 plant\_supply\_temp\_deadband (CH-D)

```

FAULT_CODE = "CH-D"
SP_BAND = 2.2

```

```

def apply_faults_arrow(table, cfg, context=None):
    temp = _col(table, "chilled-water-supply-temperature")
    sp = _col(table, "chilled-water-supply-temperature-sp")
    pump = _norm_cmd(table["pump-speed-command"])
    low = pc.less(temp, pc.subtract(sp, SP_BAND))
    high = pc.greater(temp, pc.add(sp, SP_BAND))
    return pc.and_(pc.greater(pump, 0.01), pc.or_(low, high))

```

### 30.6.4 chiller\_excessive\_runtime (CH-F)

```

from open_fdd.arrow_runtime.cookbook import arrow_rolling_mean

```

```

FAULT_CODE = "CH-F"
WINDOW, MAX_ON = 276, 264 # 5-min data ≈ 23 h window, 22 h max on

```

```

def apply_faults_arrow(table, cfg, context=None):
    on = pc.greater(_col(table, "chiller-status"),
0.5).cast(pa.float64())
    roll = arrow_rolling_mean(on, WINDOW)
    count = pc.multiply(roll, float(WINDOW))
    return pc.greater(count, float(MAX_ON))

```

---

## 30.7 Heat pumps

### 30.7.1 hp\_discharge\_cold\_when\_heating (HP-D)

```
FAULT_CODE = "HP-D"  
MIN_SAT, ZONE_COLD, FAN_ON = 85.0, 69.0, 0.01
```

```
def apply_faults_arrow(table, cfg, context=None):  
    sat = _col(table, "sa-t")  
    zone = _col(table, "stat_zn-t")  
    fan = _col(table, "supply-fan-status")  
    return pc.and_(  
        pc.greater(fan, FAN_ON),  
        pc.less(zone, ZONE_COLD),  
        pc.less(sat, MIN_SAT),  
    )
```

---

## 30.8 Opportunistic / ventilation

### 30.8.1 econ\_active\_warm\_ambient (AHU-E)

```
FAULT_CODE = "AHU-E"  
T_CUTOFF, DPR_MIN = 63.0, 0.42
```

```
def apply_faults_arrow(table, cfg, context=None):  
    oat = _col(table, "oa-t")  
    damper = _norm_cmd(table["oa-damper-command"])  
    return pc.and_(pc.greater(oat, T_CUTOFF), pc.greater(damper,  
DPR_MIN))
```

### 30.8.2 mech\_cool\_when\_econ\_available (AHU-E)

```
FAULT_CODE = "AHU-E"  
T_CUTOFF, DPR_MAX = 63.0, 0.32
```

```
def apply_faults_arrow(table, cfg, context=None):  
    oat = _col(table, "oa-t")  
    damper = _norm_cmd(table["oa-damper-command"])  
    cool = _norm_cmd(table["cooling-valve-command"])  
    return pc.and_(pc.less(oat, T_CUTOFF), pc.less(damper,  
DPR_MAX), pc.greater(cool, 0.01))
```

### 30.8.3 low\_oa\_fraction\_estimated (VAV-B)

```
FAULT_CODE = "VAV-B"  
OA_MIN, GAP = 21.0, 2.2
```

```
def apply_faults_arrow(table, cfg, context=None):  
    mat, rat, oat = _col(table, "ma-t"), _col(table, "ra-t"),  
    _col(table, "oa-t")  
    fan = _norm_cmd(table["supply-fan-speed-command"])  
    denom = pc.subtract(oat, rat)  
    safe = pc.greater(pc.abs(denom), GAP)  
    num = pc.subtract(mat, rat)  
    oa_pct = pc.multiply(pc.divide(num, denom), 100.0)  
    low = pc.less(oa_pct, OA_MIN)  
    return pc.and_(pc.greater(fan, 0.01), safe, low)
```

### 30.8.4 preheat\_excess\_temp (AHU-B)

```
FAULT_CODE = "AHU-B"  
EXCESS = 2.2
```

```
def apply_faults_arrow(table, cfg, context=None):  
    pre = _col(table, "preheat-coil-leaving-air-temperature")  
    sp = _col(table, "sa-t-sp")  
    oat = _col(table, "oa-t")  
    valve = _norm_cmd(table["preheat-valve-command"])  
    hot_oa = pc.and_(  
        pc.greater(oat, sp),  
        pc.greater(pc.subtract(pre, oat), EXCESS),  
    )  
    cold_oa = pc.and_(  
        pc.less(oat, sp),  
        pc.greater(pc.subtract(pre, sp), EXCESS),  
    )  
    return pc.and_(pc.greater(valve, 0.01), pc.or_(hot_oa,  
cold_oa))
```

---

## 30.9 Weather station

### 30.9.1 weather\_temp\_spike (BLD-B)

```
from open_fdd.arrow_runtime.cookbook import rate_of_change_mask  
  
FAULT_CODE = "BLD-B"
```

```

def apply_faults_arrow(table, cfg, context=None):
    return rate_of_change_mask(
        table,
        {**cfg, "max_per_sample": 16.0, "column": "oa-t"},
        col="oa-t",
    )

```

### 30.9.2 weather\_gust\_lt\_wind (BLD-B)

```

FAULT_CODE = "BLD-B"

```

```

def apply_faults_arrow(table, cfg, context=None):
    gust = _col(table, "wind-gust-speed")
    wind = _col(table, "wind-speed")
    return pc.and_(
        pc.is_valid(gust),
        pc.is_valid(wind),
        pc.less(gust, wind),
    )

```

### 30.9.3 RH bounds

```

from open_fdd.arrow_runtime.cookbook import sensor_bounds_mask

```

```

FAULT_CODE = "DC-C"

```

```

def apply_faults_arrow(table, cfg, context=None):
    merged = {**cfg, "value_kind": "rh"}
    return sensor_bounds_mask(table, "relative_humidity", merged)

```

---

## 30.10 Schedule + weather gating (occupied hours)

```

from open_fdd.arrow_runtime.cookbook import _unoccupied_mask,
_fan_on_mask

```

```

FAULT_CODE = "BLD-C"

```

```

def apply_faults_arrow(table, cfg, context=None):
    """Fan running when schedule says unoccupied."""
    return pc.and_( _fan_on_mask(table, cfg),
        _unoccupied_mask(table, cfg))

```



Pass `occupied_start_hour`, `occupied_end_hour`, `tz_offset_hours` in Rule Lab `cfg`.

## 30.11 FC4 — PID hunting (legacy GL36)

Ports `pandas FaultConditionFour` / `check_hunting` to Arrow. Two modes:

| Mode                       | <code>cfg["hunting_mode"]</code> | Use on                                                            |
|----------------------------|----------------------------------|-------------------------------------------------------------------|
| Per-loop command           | <code>command</code> (default)   | VAV damper, reheat, pump VFD, RTU cooling %                       |
| AHU operating-state bitmap | <code>ahu_os</code>              | RTU/AHU with economizer + fan + heat + cool columns in wide frame |

**Fault codes:** **VAV-F** (VAV outputs), **CH-G** (plant pump), **RTU-E** (RTU cooling capacity), **AHU-G** (multi-signal OS / legacy FC4 alias).

```
"""pid_hunting_fc4.py – deploy via setup_gl36_fdd.py or Rule
Lab."""

import pyarrow as pa

from open_fdd.arrow_runtime.cookbook import
pid_hunting_ahu_os_mask, pid_hunting_command_mask

FAULT_CODE = "VAV-F" # or AHU-G, CH-G, RTU-E per binding

def apply_faults_arrow(table, cfg, context=None):
    mode = str(cfg.get("hunting_mode") or
"command").strip().lower()
    if mode in ("ahu", "ahu_os", "os"):
        return pid_hunting_ahu_os_mask(table, cfg)
    col = str(cfg.get("value_column") or cfg.get("column") or
"").strip()
    if col and col not in table.column_names:
        return pa.array([False] * table.num_rows, type=pa.bool_)
    return pid_hunting_command_mask(table, cfg, col=col or None)
```

**Tuning (Acme 60 s poll, 1 h window):**

```
cfg = {
    "poll_interval_s": 60,
    "hunting_window_hours": 1.0,
    "delta_os_max": 10,          # max command steps or OS bitmap
changes per window
    "min_command_delta": 0.03,   # 3 % minimum step to count as a
reversal
    "min_active_command": 0.05,
# loop must be modulating above 5 %
}
```

Legacy interlink: LEGACY\_CODE\_MAP["FC4"] → **AHU-G**; per-loop VAV/  
plant rules use family-specific codes above.

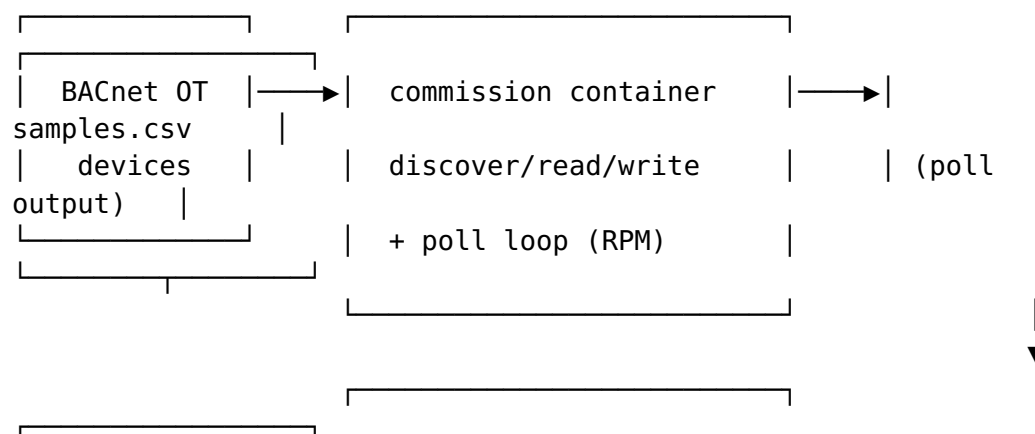
## 30.12 Next steps

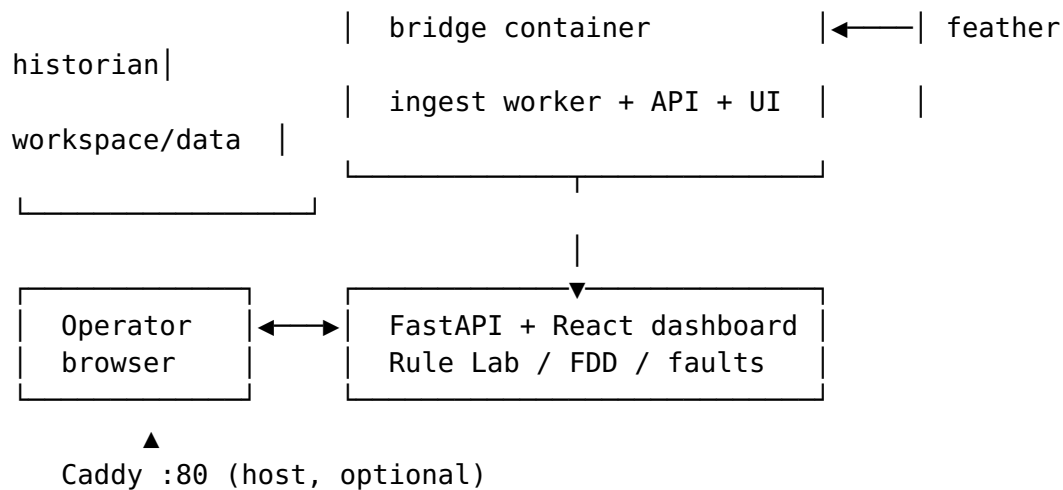
1. Copy a module into workspace/data/rules\_py/<site>\_<rule>.py
2. Bind feather columns on the Rule Lab row
3. Set fault\_code metadata
4. Quick-test on 3-24 h feather window
5. Ship via setup\_gl36\_fdd.py or Rule Lab save

**See also:** [\[Expression cookbook\]\({{ "/rule-cookbook/expression-cookbook/" | relative\\_url }}\)](#) · [Fault codes\({{ "/fault-codes/" | relative\\_url }}\)](#) · [Rule Lab\({{ "/operator-bridge/rule-lab/" | relative\\_url }}\)](#)

## 31 System overview

## 32 System overview





## 32.1 Docker edge stack (production)

Three GHCR images — see [Containers](#)(`{{ "/architecture/containers/" | relative_url }}`):

| Container         | BACnet OT                  | Operator LAN                         |
|-------------------|----------------------------|--------------------------------------|
| <b>bridge</b>     | No (HTTP only)             | Yes, via Caddy :80 or loopback :8765 |
| <b>commission</b> | Yes (host network, :47808) | No (internal :8767)                  |
| <b>mcp-rag</b>    | No                         | No (internal :8090)                  |

BACnet **polling** is not a separate container — it runs inside **commission**. The bridge only **ingests** poll CSV into feather.

## 32.2 Components

| Layer                    | Responsibility                                                                                        |
|--------------------------|-------------------------------------------------------------------------------------------------------|
| <b>Operator Bridge</b>   | Auth, REST/WebSocket API, compiled dashboard, Rule Lab, historian ingest                              |
| <b>BACnet commission</b> | Who-Is, read/write (gated), <b>scheduled poll loop</b> → <code>samples.csv</code>                     |
| <b>Historian</b>         | Feather files per point; local-first retention                                                        |
| <b>FDD runner</b>        | <code>workspace/data/rules_py/*.py</code> on timer or <code>POST /api/rules/batch</code>              |
| <b>Model</b>             | Brick TTL + <code>model.json</code> bindings ( <code>fdd_input</code> , <code>equipment tree</code> ) |
| <b>MCP RAG</b>           | Optional doc retrieval for agent tools                                                                |
| <b>Caddy</b>             | Host reverse proxy :80 → bridge (typical edge)                                                        |

| Layer  | Responsibility                                                |
|--------|---------------------------------------------------------------|
| Ollama | Optional host LLM for building insight (not required for FDD) |

32.3 Deployment

IT operators: [Quick Start — Docker]({{ "/quick-start/docker/" | relative\_url }}). Modes and Caddy: `Deployment modes`({{ "/architecture/deployment-modes/" | relative\_url }}).

32.4 Optional PyPI-only path

`pip install open-fdd ships Arrow runtime + playground lint helpers` **without** the Operator Bridge UI. Use for offline rule tests or CI. See [Appendix — Python package]({{ "/appendix/python-package/" | relative\_url }}).

33 Deployment modes

34 Deployment modes

How Open-FDD is typically run: local development, lab LAN trials, and production edge on a trusted OT/IT network. Open-FDD is designed for **building LAN or OT edge** deployment — not direct exposure to the public internet.

34.1 Mode summary

| Mode            | Stack                      | BACnet                       | Front door                                 | Auth                                        |
|-----------------|----------------------------|------------------------------|--------------------------------------------|---------------------------------------------|
| Local dev       | Compose on control machine | Optional bench simulator     | 127.0.0.1:8765                             | OFDD_AUTH_DISABLED=1 optional on loopback   |
| Lab LAN         | Three GHCR containers      | Real or simulator VLAN       | Caddy :80 or direct :8765                  | Required unless explicit insecure lab flags |
| Edge production | Three GHCR containers      | OT NIC via <b>commission</b> | <b>Caddy</b> :80 or :443 → loopback bridge | <b>Required</b> — workspace/auth.env.local  |

| Mode | Stack           | BACnet | Front door | Auth |
|------|-----------------|--------|------------|------|
|      | + host<br>Caddy |        |            |      |

## 34.2 v3 container stack (all edge modes)

| Image              | Role                                                            |
|--------------------|-----------------------------------------------------------------|
| openfdd-bridge     | Operator Bridge API, dashboard, <b>feather historian ingest</b> |
| openfdd-commission | BACnet discover/read/write + <b>poll loop</b>                   |
| openfdd-mcp-rag    | Doc-search sidecar for agent tools                              |

**openfdd-bacnet-poll is retired.** BACnet polling runs inside **commission** only. Do not run the legacy poll container or openfdd-bacnet-poll systemd unit alongside Docker commission — that double-polls the OT network.

See [Containers](#)(`{{ "/architecture/containers/" | relative_url }}`) for ports, networks, and persistence.

## 34.3 Caddy HTTP (typical edge)

- Caddy listens on building LAN :80
- Reverse-proxies to bridge at 127.0.0.1:8765
- Bridge stays loopback-only; operators bookmark `http://<edge-ip>/`

Bootstrap and compose: [\[Quick Start — Docker\]](#)(`{{ "/quick-start/docker/" | relative_url }}`).

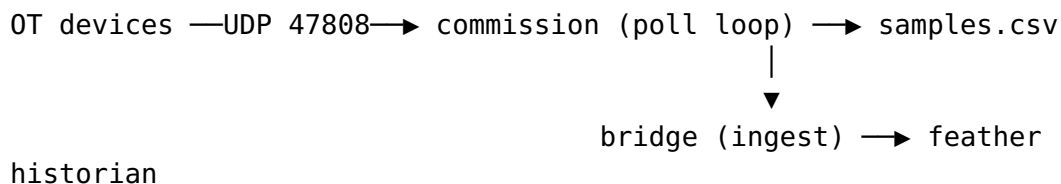
## 34.4 Caddy TLS (OT LAN)

For HTTPS on the edge LAN (self-signed or site CA):

1. Generate certs: `./scripts/setup_caddy_certs.sh` (control machine) or site PKI
2. Start with `OFDD_CADDY_MODE=tls`
3. Trust the CA on operator PCs

Details: [TLS and certificates](#)({{ "/security/tls-and-certs/" | relative\_url }}). Security headers are set by the **Operator Bridge**; Caddy adds **HSTS** only in TLS mode.

## 34.5 BACnet data path



The bridge thread named `bacnet_poll_worker` is **ingest only** — it does not bind BACnet.

## 34.6 What not to do on production edges

- Port-forward :8765 to the public internet without TLS and strong auth
- Run `docker compose down -v` or delete workspace/ on a live site
- Deploy the retired `openfdd-bacnet-poll` image alongside `commission`

## 34.7 Related

| Topic            | Page                                                                      |
|------------------|---------------------------------------------------------------------------|
| First deploy     | <a href="#">[Quick Start — Docker]</a> ({{ "/quick-start/docker/"         |
| Upgrade          | <a href="#">Updating the stack</a> ({{ "/quick-start/updating/"           |
| LAN security     | <a href="#">LAN hardening</a> ({{ "/security/lan-hardening/"              |
| Lab example site | <a href="#">[Examples — GL36 lab note]</a> ({{ "/examples/acme-gl36-lab/" |

# 35 Containers

# 36 Containers

Open-FDD v3 edge sites run **three** GHCR containers plus optional **host** services (Caddy, Ollama). There is no separate BACnet poll container — **field BACnet polling lives in commission**.

**Quick answer:** Who binds BACnet UDP 47808? **openfdd-commission**. Bridge only ingests samples.csv into the feather historian.

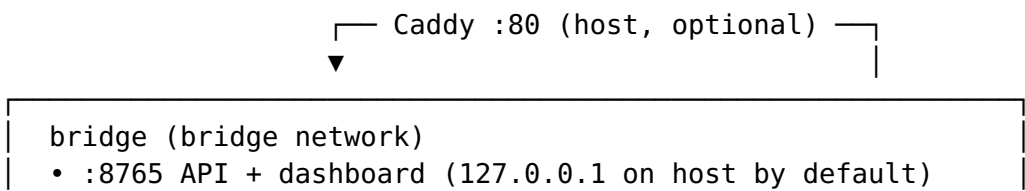
## 36.1 GHCR images

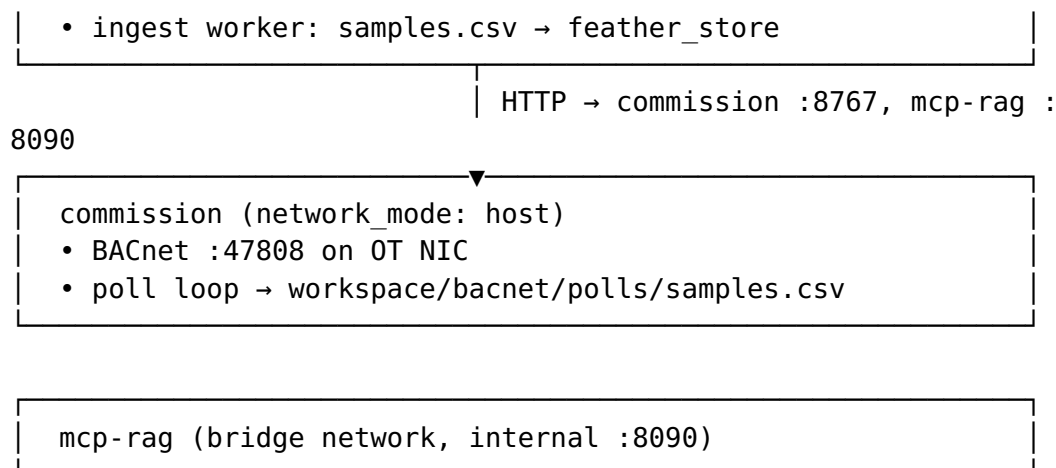
| Image                                | Compose service | Role                                                 |
|--------------------------------------|-----------------|------------------------------------------------------|
| ghcr.io/bbartling/openfdd-bridge     | bridge          | Operator API, React dashboard, feather <b>ingest</b> |
| ghcr.io/bbartling/openfdd-commission | commission      | BACnet discover / read / write + <b>poll loop</b>    |
| ghcr.io/bbartling/openfdd-mcp-rag    | mcp-rag         | Doc-search sidecar for agent tools                   |

Tag: **latest** on [GitHub Packages](#). Retired: openfdd-bacnet-poll (poll runs inside **commission**). GHCR retains only the **three newest** versions per image.

Template: docker/compose.edge.yml → copy to ~/open-fdd/docker-compose.yml.

## 36.2 Stack diagram





## 36.3 Responsibilities

| Concern                             | Container                                                                            |
|-------------------------------------|--------------------------------------------------------------------------------------|
| BACnet Who-Is / read / write        | <b>commission</b>                                                                    |
| BACnet scheduled RPM reads          | <b>commission</b> (bacnet_poll_loop)                                                 |
| CSV → feather historian             | <b>bridge</b> (feather ingest worker — bacnet_poll_worker module name is historical) |
| Operator login, Rule Lab, faults UI | <b>bridge</b>                                                                        |
| Agent doc search                    | <b>mcp-rag</b>                                                                       |

The bridge thread named `bacnet_poll_worker` is **ingest only** — it does not bind BACnet.

## 36.4 Network and ports (typical edge)

| Endpoint           | Reachable from     | Notes                     |
|--------------------|--------------------|---------------------------|
| Caddy :80          | Building LAN       | Reverse proxy to bridge   |
| Bridge :8765       | Loopback (default) | Bind 127.0.0.1 in compose |
| Commission : 8767  | Bridge / loopback  | HTTP API for BACnet ops   |
| Commission : 47808 | OT BACnet LAN      | UDP; requires             |



| Endpoint  | Reachable from           | Notes                                               |
|-----------|--------------------------|-----------------------------------------------------|
| MCP :8090 | Bridge container network | network_mode: host<br>Not exposed to LAN by default |

## 36.5 Long-lived deployment

All compose services set `restart: unless-stopped`. After host reboot:

```
sudo systemctl enable docker
cd ~/open-fdd && docker compose ps
```

If containers are down: `docker compose up -d` (idempotent).

See [Run with Docker images](#)({{ "/quick-start/docker/" | relative\_url }}).

## 36.6 Persistent data

State lives on the **host** under `workspace/` (bind mount). Containers are replaceable; **backup workspace/** before image upgrades.

| Path                                            | Contents                                              |
|-------------------------------------------------|-------------------------------------------------------|
| <code>workspace/data/feather_store/</code>      | Historian                                             |
| <code>workspace/data/*.json</code>              | Model, rules, FDD results                             |
| <code>workspace/bacnet/commissioning/</code>    | <code>commission.env</code> , <code>points.csv</code> |
| <code>workspace/bacnet/polls/samples.csv</code> | Poll output                                           |
| <code>workspace/auth.env.local</code>           | Login secrets                                         |
| <code>workspace/api/static/app/</code>          | Dashboard bundle (if updated separately)              |

## 36.7 Optional host services

| Service      | Role                                   |
|--------------|----------------------------------------|
| <b>Caddy</b> | TLS or plain HTTP :80 → 127.0.0.1:8765 |

| Service        | Role                                                        |
|----------------|-------------------------------------------------------------|
| Ollama         | Optional LLM on :11434 (not in the three-image stack)       |
| FDD loop timer | Optional openfdd-fdd-loop.timer on host (docker exec batch) |

## 37 Data flow

## 38 Data flow

### 38.1 Telemetry ingest

Open-FDD supports three commissioning drivers. Each appends long-format samples to workspace CSVs and writes **feather shards** tagged by source (bacnet, modbus, json\_api).

#### 38.1.1 BACnet

1. **Commission poll loop** (inside the commission container) reads enabled BACnet points from points.csv on a schedule (host network, BACnet :47808).
2. RPM results append to workspace/bacnet/polls/samples.csv.
3. **Bridge ingest worker** loads new rows into feather\_store/bacnet/.

BACnet devices → commission (poll loop) → samples.csv → bridge (ingest) → feather\_store/bacnet

#### 38.1.2 Modbus TCP

1. Operator configures registers on the **Modbus** tab (or read\_and\_store API).
2. Poll worker reads holding/input registers via Modbus TCP.
3. Samples append to workspace/modbus/polls/samples.csv → feather\_store/modbus/.

#### 38.1.3 JSON API (HTTP/HTTPS)

1. Operator configures REST endpoints on the **JSON API** tab — GET/POST, Bearer or Basic auth, \${ENV:VAR} placeholders from

json\_api.env.local, optional TLS verify off for self-signed OT gateways.

2. Poll worker issues HTTP requests (one GET can fan out to multiple JSON paths, e.g. OpenWeather web-oat-t / web-rh / web-weather-desc).
3. Samples append to workspace/json\_api/polls/samples.csv → feather\_store/json\_api/.
4. FDD batch **merges** json\_api columns with BACnet/Modbus on the same site (nearest timestamp, 30 min tolerance) for cross-source rules.

OT REST / weather API → bridge JSON API worker → samples.csv → feather\_store/json\_api

↘ merged

into FDD frame with bacnet/modbus

Showcase: [OpenWeatherMap bundle]({{ "/drivers/json-api/" | relative\_url }})#openweathermap-showcase-recommended-demo).  
Details: Driver framework({{ "/drivers/" | relative\_url }}), [BACnet polling]({{ "/bacnet/polling/" | relative\_url }}), JSON API driver({{ "/drivers/json-api/" | relative\_url }}), Containers({{ "/architecture/containers/" | relative\_url }}).

## 38.2 Rule evaluation

1. Rules are Python files in workspace/data/rules\_py/ with metadata in rules\_store.json.
2. **Bindings** map rules to points via the Brick model (fdd\_input tags).
3. Batch runner (POST /api/rules/batch or host timer) produces fdd\_results.json.
4. Dashboard **faults** view and check-engine status read aggregated results.

## 38.3 Model sync

- Commissioning imports update\_model.json / TTL graph.
- POST /api/model/sync-ttl refreshes SPARQL-backed tree used by the UI.

## 38.4 Retention

Feather files grow on disk; retention is configurable (workspace/data.env.local). Image upgrades must **preserve** workspace/data/ — backup before docker compose up -d --force-recreate. See Updating the stack({{ "/quick-start/updating/" | relative\_url }}).

# 39 Architecture

# 40 Architecture

Open-FDD v3 edge: **three Docker containers** (bridge, commission, mcp-rag), BACnet polling in **commission**, feather ingest in **bridge**, optional host Caddy and Ollama.

| Page                                                                     | Topic           |
|--------------------------------------------------------------------------|-----------------|
| <a href="#">System overview</a> ({{ "/" architecture/overview/"          | relative_url }} |
| <a href="#">Deployment modes</a> ({{ "/" architecture/deployment-modes/" | relative_url }} |
| <a href="#">Data flow</a> ({{ "/"architecture/data-flow/"                | relative_url }} |
| <a href="#">Containers</a> ({{ "/" architecture/containers/"             | relative_url }} |
| <a href="#">Driver framework</a> ({{ "/" drivers/"                       | relative_url }} |

**Operators:** [Quick Start — Docker]({{ "/"quick-start/docker/" | relative\_url }) — no git clone on the edge host.

**BACnet polling:** [Polling](#)({{ "/"bacnet/polling/" | relative\_url }) — commission container only.

# 41 Fault confirmation / minimum duration

# 42 Fault confirmation

A raw rule condition can flicker true for one sample. **Fault confirmation** delays the final fault mask until the condition stays true for N consecutive rows or N minutes.

## 42.1 Config fields

| Field               | Type  | Meaning                                         |
|---------------------|-------|-------------------------------------------------|
| min_true_rows       | int   | Consecutive true samples required               |
| min_elapsed_minutes | float | Elapsed time from streak start (uses timestamp) |
| poll_interval_s     | float | Fallback when timestamps missing                |
| timestamp_column    | str   | Default timestamp                               |

## 42.2 Example

At **60-second polling**, `min_true_rows: 5` means the condition must persist about **5 minutes** before fault becomes true.

PyArrow:

```
def apply_faults_arrow(table, cfg, context=None):  
    return pc.greater(table["duct-t"], 80.0)
```

Rule config:

```
{"min_true_rows": 5, "poll_interval_s": 60}
```

DataFusion SQL (same config — confirmation runs after SQL):

```
SELECT  
    *,  
    "duct-t" > 80.0 AS fault  
FROM telemetry
```

The runtime applies confirmation in `open_fdd.arrow_runtime.confirmation` for both backends.

## 42.3 Paired smoke default (5 minutes)

The hardcoded paired FDD harness (`open_fdd/validation/paired_fdd_contract.py`) sets **5-minute confirmation** on every smoke rule:

```
        {"min_elapsed_minutes": 5, "min_true_rows": 5, "poll_interval_s":  
60}
```

This suppresses false positives during bench 5007 / Acme OAT spread toggles. See [Paired FDD smoke](#).

# 43 Rule Cookbook

## 44 Rule Cookbook

Practical patterns for **Arrow-native Rule Lab** on feather historian PyArrow tables — **no pandas on the IoT edge**.

| Page                                                                                 | Use when        |
|--------------------------------------------------------------------------------------|-----------------|
| [Rule Lab quick start]<br>({{ "/operator-bridge/<br>rule-lab/"                       | relative_url }} |
| [PyArrow rule recipes]<br>({{ "/rule-cookbook/<br>python-recipes-<br>arrow/"         | relative_url }} |
| <u>DataFusion SQL</u><br>recipes({{ "/rule-<br>cookbook/datafusion-<br>sql-recipes/" | relative_url }} |
| <u>Fault</u><br>confirmation({{ "/rule-<br>cookbook/fault-<br>confirmation/"         | relative_url }} |
| <u>Windowing &amp;</u><br>debugging({{ "/rule-<br>cookbook/windowing-<br>debugging/" | relative_url }} |
| [Expression cookbook]<br>({{ "/rule-cookbook/<br>expression-cookbook/"               | relative_url }} |
| <u>GL36 algorithm</u><br>stubs({{ "/rule-<br>cookbook/gl36-<br>algorithm-stubs/"     | relative_url }} |

**PyArrow** is the primary path for complex HVAC FDD and ML prep. **DataFusion SQL** is optional, Rust-ready, and best for simple expression-style rules.

Programmatic sensor defaults:

`open_fdd.arrow_runtime.sensor_catalog.SENSOR_PROFILES.`

```
import pyarrow.compute as pc
from open_fdd.arrow_runtime.cookbook import sensor_bounds_mask

def apply_faults_arrow(table, cfg, context=None):
```

```
        return sensor_bounds_mask(table, "zone_temp", cfg,
min_true_rows=5)
```

## 45 Windowing & debugging

## 46 Windowing & debugging

### 46.1 Lookback vs rolling window

|               | Historian lookback                                                                                                            | Rolling window                        |
|---------------|-------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| Set by        | Batch lookback_hours, FDD loop, Quick test UI                                                                                 | WINDOW_SAMPLES constant in rule.py    |
| Typical       | 1 h scheduled; 24 h Update all                                                                                                | 12 samples $\approx$ 1 h @ 5 min poll |
| Validate with | <code>_kit_lookback_stats(table) — see [Lookback window helper]</code><br>( <code>{{ "/rule-cookbook/lookback-window/"</code> | <code>relative_url }}</code> )        |

### 46.2 Rolling windows (Arrow)

Use `open_fdd.arrow_runtime.windows` for sample-based rolling min/max and consecutive-true streaks:

```
from open_fdd.arrow_runtime.windows import arrow_rolling_min,
arrow_rolling_max, arrow_consecutive_true
```

Cookbook masks (`flatline_1h_mask`, `spread_1h_mask`, `oob_mask`) default to  $\sim 12$  samples ( $\sim 1$  h at 5 min poll).

Bench rules use constants: `WINDOW_SAMPLES`, `FLATLINE_TOLERANCE`, `OAT_LOW` / `OAT_HIGH`, etc. Legacy `cfg` keys still work for uploaded rules that read `cfg.get(...)`.

### 46.3 Rule Lab console

Quick-test and batch responses include backend: `arrow`, `ms`, and flagged row counts. Arrow summary events appear in the event console on fault hits.

# 47 Expression cookbook (Arrow-native)

# 48 Expression cookbook (Arrow-native)

Reference for **Open-FDD 3.x Rule Lab**: every rule is a Python module with `apply_faults_arrow(table, cfg, context)` using `pyarrow.compute` — **no pandas, no YAML expression files, no NumPy DataFrames on the IoT edge.**

This is the **only** expression cookbook for Open-FDD 3.x. The pandas/YAML RuleRunner path is retired — use Arrow `apply_faults_arrow` and the operator bridge Rule Lab. Legacy GL36-style recipes map to **fixed fault codes**(`{{ "/fault-codes/" | relative_url }}`) and Arrow patterns below.

| Topic                                                 | Page                                                                                                              |
|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Full copy-paste library (GL36 A-M, VAV, plant)</b> | <a href="#">**Python recipes (full Arrow library)</a><br>( <code>{{ "/rule-cookbook/python-recipes-arrow/"</code> |
| Quick templates                                       | <a href="#">[Arrow recipes]</a> ( <code>{{ "/rule-cookbook/arrow-recipes/"</code>                                 |
| Shared imports                                        | <a href="#">[Python recipes]</a> ( <code>{{ "/rule-cookbook/python-recipes/"</code>                               |
| Console window stats                                  | <a href="#">[Lookback window]</a> ( <code>{{ "/rule-cookbook/lookback-window/"</code>                             |
| Fault code catalog                                    | <a href="#">Fault codes</a> ( <code>{{ "/fault-codes/"</code>                                                     |
| Programmatic defaults                                 | <code>open_fdd.arrow_runtime.sensor_catalog</code>                                                                |

## 48.1 Legacy → modern translation

| Legacy (pandas / YAML)                             | Arrow-native (3.x)                                                    |
|----------------------------------------------------|-----------------------------------------------------------------------|
| <code>type: expression + expression: string</code> | <code>apply_faults_arrow()</code> in <code>rules_py/*.py</code>       |
| <code>type: hunting (GL36 <b>FC4</b>)</code>       | <code>pid_hunting_fc4.py</code> → <b>AHU-G / VAV-F / CH-G / RTU-E</b> |
| <code>RuleRunner.run(df, column_map=...)</code>    | Rule Lab batch on <b>feather PyArrow table</b>                        |
| <code>np.maximum(a, b)</code>                      | <code>pc.max(a, b)</code>                                             |



| Legacy (pandas / YAML)               | Arrow-native (3.x)                                                                                                 |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <code>series.rolling(n).min()</code> | <code>arrow_rolling_min(vals, n)</code><br>from<br><code>open_fdd.arrow_runtime.windows</code>                     |
| <code>series.diff().abs()</code>     | <code>arrow_abs_diff(vals, 1)</code>                                                                               |
| <code>normalize_cmd(x)</code>        | <code>norm_cmd_array() /</code><br><code>pc.if_else(pc.greater(x, 1),</code><br><code>pc.divide(x, 100), x)</code> |
| <code>params.max_temp</code> in YAML | Module constant or<br><code>cfg["bounds_high"]</code>                                                              |
| <code>flag: rule_a_flag</code>       | Rule metadata <b>fault_code</b> →<br>AHU-A ... VAV-C                                                               |
| Numeric codes VAV-03                 | Letter codes <b>VAV-C</b> (see<br>LEGACY_CODE_MAP in bridge)                                                       |
| Pandas FC4 / <code>fc4_flag</code>   | LEGACY_CODE_MAP: FC4 → <b>AHU-G</b>                                                                                |

**Edge constraint:** Docker / BACnet poll path never imports pandas. Central portfolio Dash may use pandas for CSV analytics only.

## 48.2 How Arrow rules are structured

1. **Historian columns** — Feather column names from BACnet poll (oa-t, sa-t, stat\_zn-t, ...) or Brick labels via model export.
2. **Module constants** — Thresholds at top of file (bench style) or read from `cfg` for site tuning.
3. **apply\_faults\_arrow** — Returns a **boolean PyArrow array** (True = fault sample).
4. **fault\_code** — Set in Rule Lab metadata; must exist in GET `/api/faults/catalog`.

```
import pyarrow.compute as pc
from open_fdd.arrow_runtime.cookbook import sensor_bounds_mask

FAULT_CODE = "VAV-C" # metadata in Rule Lab UI

def apply_faults_arrow(table, cfg, context=None):
    return sensor_bounds_mask(table, "zone_temp", cfg)
```

### 48.2.1 Command scaling (0-1 vs 0-100 %)

BACnet often exposes **0-100** for damper/VFD commands. Scale explicitly:

```
def _norm_cmd(col):
    return pc.if_else(pc.greater(col, 1.0), pc.divide(col,
100.0), col)
```

### 48.2.2 Occupied hours (schedule gating)

Use `open_fdd.arrow_runtime.cookbook._unoccupied_mask` or compare local hour from timestamp column. Example: **fan on when unoccupied** → **BLD-C** / `schedule_compare`.

---

## 48.3 Sensor validation (bounds, flatline, rate of change)

Use `open_fdd.arrow_runtime.sensor_catalog.SENSOR_PROFILES` for defaults, or copy constants into `rules_py`. Tune per site in Rule Lab `cfg` or building-agent tuning brief.

### 48.3.1 Bounds (out of range) — `oob_rolling`

| Sensor kind                      | Min  | Max | Flatline tol | Max $\Delta$ / hour | Max $\Delta$ / 15 min | Typical fault code            |
|----------------------------------|------|-----|--------------|---------------------|-----------------------|-------------------------------|
| Zone temp (°F)                   | 55   | 90  | 0.10         | 4.0                 | 2.0                   | <b>VAV-C</b>                  |
| Supply air temp                  | 50   | 110 | 0.15         | 8.0                 | 3.0                   | <b>AHU-C, RTU-C</b>           |
| Return air temp                  | 55   | 95  | 0.10         | 3.0                 | 1.5                   | <b>AHU-D</b><br>(also mixing) |
| Mixed air temp                   | 40   | 110 | 0.15         | 6.0                 | 2.5                   | <b>AHU-D</b>                  |
| Outdoor air temp                 | −40  | 130 | 0.10         | 12.0                | 6.0                   | <b>BLD-B</b>                  |
| Duct static (inH <sub>2</sub> O) | −0.5 | 3.0 | 0.02         | 0.5                 | 0.25                  | <b>AHU-A</b>                  |
| Relative humidity (%)            | 0    | 100 | 1.0          | 15.0                | 8.0                   | <b>DC-C</b>                   |
| Chilled water (°F)               | 40   | 90  | 0.10         | 4.0                 | 2.0                   | <b>CH-D</b>                   |
| Hot water (°F)                   | 70   | 200 | 0.15         | 6.0                 | 3.0                   | <b>CH-D</b>                   |

| Sensor kind                     | Min | Max  | Flatline tol | Max $\Delta$ / hour | Max $\Delta$ / 15 min | Typical fault code            |
|---------------------------------|-----|------|--------------|---------------------|-----------------------|-------------------------------|
| Condenser water (°F)            | 50  | 110  | 0.15         | 5.0                 | 2.5                   | <b>CH-A</b>                   |
| CO <sub>2</sub> (ppm, occupied) | 400 | 1000 | 5.0          | 200                 | 80                    | <b>VAV-B</b><br>(ventilation) |
| Discharge air temp              | 45  | 120  | 0.15         | 10.0                | 4.0                   | <b>VAV-E, HP-D</b>            |

**Return air** uses a **narrow band** vs zone/OAT because RAT should track building return conditions, not outdoor extremes. When OAT/RAT/MAT are all present, prefer **mixing envelope** (below) over RAT bounds alone.

**CO<sub>2</sub> upper bound** is an occupied ventilation target; unoccupied rules may use 400–2000 ppm per policy.

### 48.3.2 Flatline (stuck sensor) — flatline\_1h

Default **12 samples  $\approx$  1 hour** at 5-minute poll. True when rolling max – min  $\leq$  tolerance.

```

from open_fdd.arrow_runtime.cookbook import sensor_flatline_mask

def apply_faults_arrow(table, cfg, context=None):
    return sensor_flatline_mask(table, "outdoor_air_temp", cfg)
# BLD-B

```

Bench references: workspace/data/rules\_py/bench\_oa-t\_flatline\_1h.py, bench\_stat\_zn-t\_flatline\_1h.py.

### 48.3.3 Rate of change (spike) — rate\_of\_change

Flags physically impossible step changes (comms glitch, substituted value).

```

from open_fdd.arrow_runtime.cookbook import rate_of_change_mask
from open_fdd.arrow_runtime.sensor_catalog import cfg_from_profile

def apply_faults_arrow(table, cfg, context=None):
    merged = cfg_from_profile("outdoor_air_temp", cfg)
    merged["samples_per_hour"] = 12 # 5-min poll
    return rate_of_change_mask(table, merged, col="oa-t")

```

Legacy **weather\_temp\_spike** (spike\_limit: 16 °F per step) → **BLD-B** with max\_per\_15min: 6 or stricter per-step limit.

### 48.3.4 Mixing envelope (MAT vs OAT/RAT) — `mixing_envelope`

Legacy GL36 **Rule B / C** and **AHU-D**. MAT should sit between OAT and RAT ( $\pm$  tolerance) while fan runs.

```
from open_fdd.arrow_runtime.cookbook import mixing_envelope_mask

def apply_faults_arrow(table, cfg, context=None):
    return mixing_envelope_mask(
        table,
        {**cfg, "mixing_tol": 1.15},
        mat_col="ma-t",
        oat_col="oa-t",
        rat_col="ra-t",
        fan_col="supply-fan-speed-command",
    )
```

---

## 48.4 GL36-inspired AHU rules (legacy expression → fault code)

ASHRAE Guideline 36-style rules from the old YAML cookbook, translated to Arrow. Assign `fault_code` per row in Rule Lab.

| Legacy rule                          | Summary                          | Code         | Full Arrow module                                       |
|--------------------------------------|----------------------------------|--------------|---------------------------------------------------------|
| Rule A — duct static low @ full fan  | SP below SP setpoint at high VFD | <b>AHU-A</b> | [Rule A]({{ "/rule-cookbook/python-recipes-arrow/"      |
| Rule B — blend below band            | MAT below OAT/RAT envelope       | <b>AHU-D</b> | [Rules B & C]({{ "/rule-cookbook/python-recipes-arrow/" |
| Rule C — blend above band            | MAT above OAT/RAT envelope       | <b>AHU-D</b> | [Rules B & C]({{ "/rule-cookbook/python-recipes-arrow/" |
| Rule D — discharge cold when heating | SAT low vs MAT, heat valve open  | <b>AHU-B</b> | [Rule D]({{ "/rule-cookbook/python-recipes-arrow/"      |
| Rule E — SAT low, full heating       | SAT below SP, valve > 90%        | <b>AHU-C</b> | [Rule E]({{ "/rule-cookbook/python-recipes-arrow/"      |
| Rule F — SAT/MAT mismatch econ       | Econ mode, SAT $\neq$ MAT        | <b>AHU-E</b> | [Rule F]({{ "/rule-cookbook/python-recipes-arrow/"      |

| Legacy rule                            | Summary                                       | Code                                | Full Arrow module                                           |
|----------------------------------------|-----------------------------------------------|-------------------------------------|-------------------------------------------------------------|
| Rule G — ambient warm free cool        | OAT > SAT SP, econ open, cool off             | <b>AHU-E</b>                        | [Rule G]({{ "/rule-cookbook/python-recipes-arrow/"          |
| Rule H — OAT/MAT mismatch econ+mech    | Mech + econ, MAT ≠ OAT                        | <b>AHU-E</b>                        | [Rule H]({{ "/rule-cookbook/python-recipes-arrow/"          |
| Rule I — OAT/MAT mismatch econ-only    | Econ only, MAT ≠ OAT                          | <b>AHU-E</b>                        | [Rule I]({{ "/rule-cookbook/python-recipes-arrow/"          |
| Rule J — discharge above blend cooling | SAT > MAT in cooling                          | <b>AHU-B</b>                        | [Rule J]({{ "/rule-cookbook/python-recipes-arrow/"          |
| Rule K — discharge above SP full cool  | SAT > SP, full cooling                        | <b>AHU-C</b>                        | [Rule K]({{ "/rule-cookbook/python-recipes-arrow/"          |
| Rule L — cooling coil ΔT when off      | CHW drop when valves closed                   | <b>CH-C</b>                         | [Rule L]({{ "/rule-cookbook/python-recipes-arrow/"          |
| Rule M — heating coil ΔT when off      | HW rise when valves closed                    | <b>AHU-B</b>                        | [Rule M]({{ "/rule-cookbook/python-recipes-arrow/"          |
| <b>FC4 — PID hunting</b>               | Excessive command reversals or AHU OS changes | <b>AHU-G / VAV-F / CH-G / RTU-E</b> | [FC4 PID hunting]({{ "/rule-cookbook/python-recipes-arrow/" |

All rules A-M have **full apply\_faults\_arrow modules** in [Python recipes \(full Arrow library\)](#)({{ "/rule-cookbook/python-recipes-arrow/" | relative\_url }}).

## 48.5 Starter pack (VAV / AHU baseline)

Maps to legacy YAML starter filenames; use as first deploy bundle.

| Legacy YAML starter              | Arrow approach                                       | Fault |
|----------------------------------|------------------------------------------------------|-------|
| 01_vav_zone_temp_bounds_occupied | sensor_bounds_mask(..., "zone_temp") + occupied mask | VAV   |

| Legacy YAML starter                        | Arrow approach                                     | Fault                               |
|--------------------------------------------|----------------------------------------------------|-------------------------------------|
| 02_vav_zone_temp_flatline_occupied         | sensor_flatline_mask(..., "zone_temp") + occupied  | VAV                                 |
| 03_vav_damper_command_extreme_flatline     | flatline_1h_mask on damper cmd column              | VAV                                 |
| 04_ahu_runtime_outside_schedule            | after_hours_fan_satisfied_mask / run-hours script  | BL                                  |
| 05_ahu_duct_static_pressure_not_maintained | [Rule A]({{ "/rule-cookbook/python-recipes-arrow/" | rela<br>#ru<br>stat<br>full-<br>ahu |
| 06_ahu_internal_temp_sensor_bounds         | sensor_bounds_mask on SAT/MAT                      | AH                                  |
| 07_ahu_internal_temp_sensor_flatline       | sensor_flatline_mask on SAT                        | AH                                  |

### 48.5.1 Economizer starters

Full modules: [economizer section]({{ "/rule-cookbook/python-recipes-arrow/" | relative\_url }}#economizer-starters)  
(ahu\_econ\_100oa\_temp\_tracking\_fault,  
ahu\_mech\_cooling\_when\_free\_cooling\_available,  
ahu\_oa\_damper\_excess\_open\_extreme\_ambient).

## 48.6 VAV, central plant, heat pump

Full modules: [VAV]({{ "/rule-cookbook/python-recipes-arrow/" | relative\_url }}#vav-zones) · Central plant({{ "/rule-cookbook/python-recipes-arrow/" | relative\_url }}#central-plant) · Heat pumps({{ "/rule-cookbook/python-recipes-arrow/" | relative\_url }}#heat-pumps).

Rolling persistence: arrow\_rolling\_min + pc.and\_ (replaces pandas .rolling(n).min()).

## 48.7 Opportunistic / ventilation

Full modules: [Opportunistic section]({{ "/rule-cookbook/python-recipes-arrow/" | relative\_url }}#opportunistic-ventilation) · [Weather]({{ "/rule-cookbook/python-recipes-arrow/" | relative\_url }}#weather-station).

## 48.8 Data quality vs equipment faults

| Symptom                 | Pattern         | Code                      | Do not confuse with           |
|-------------------------|-----------------|---------------------------|-------------------------------|
| Flatline 1h             | flatline_1h     | VAV-C,<br>AHU-C,<br>BLD-B | Equipment off / damper closed |
| OOB sample              | oob_rolling     | sensor-specific           | True process excursion        |
| Spike between polls     | rate_of_change  | BLD-B                     | Fast legitimate weather front |
| No new samples          | stale_points    | BLD-D                     | Equipment fault               |
| MAT $\notin$ [OAT, RAT] | mixing_envelope | AHU-D                     | Stratification during startup |

Always gate sensor faults with **fan on**, **valve open**, or **occupied** where appropriate — see [Sensor & data quality]({{ "/fault-codes/sensor-quality/" | relative\_url }}).

## 48.9 Metric (°C) equivalents

Set `cfg["temp_unit"] = "metric"` in Rule Lab or scale constants:

| Imperial                | Metric (approx) |
|-------------------------|-----------------|
| 55 °F                   | 12.8 °C         |
| 90 °F                   | 32.2 °C         |
| 50 °F                   | 10.0 °C         |
| 110 °F                  | 43.3 °C         |
| 2.0 °F tol              | 1.1 °C          |
| 0.15 inH <sub>2</sub> O | ~37 Pa          |

`open_fdd.playground.temp_units` documents UI field keys for °F/°C toggle.

## 48.10 Binding rules to the fault catalog

1. Pick a **letter code** from `Fault codes`({{ "/fault-codes/" | relative\_url }}) — never invent suffixes.
2. In Rule Lab, set **fault\_code** on the rule row.

3. Batch FDD aggregates into GET /api/faults/status and portfolio rollup.
4. Legacy numeric codes (AHU-03) migrate via LEGACY\_CODE\_MAP in the bridge.

```
curl -s http://127.0.0.1:8765/api/faults/catalog | jq  
' .families[].codes[] | select(.code=="VAV-C") '
```

---

## 48.11 Pandas YAML → Arrow checklist (commissioning)

- ☐ Replace each type: expression YAML with a rules\_py module
- ☐ Map inputs Brick labels → feather column names in model/poll CSV
- ☐ Set fault\_code per rule (letter suffix)
- ☐ Run Rule Lab kit on 7-day feather window; confirm flag rate sane
- ☐ Apply sensor\_catalog defaults; tune bounds for site climate
- ☐ Enable building-agent check-in; verify faults in portfolio rollup

**Next:** Python recipes (full Arrow library)({{ "/rule-cookbook/python-recipes-arrow/" | relative\_url }}) · [Arrow recipes]({{ "/rule-cookbook/arrow-recipes/" | relative\_url }}) · Fault codes({{ "/fault-codes/" | relative\_url }}) · Rule Lab({{ "/operator-bridge/rule-lab/" | relative\_url }})

## 49 GL36 algorithm stubs

## 50 GL36 algorithm stubs (doc-only)

Draft **supervisory** PyArrow patterns for the future Algorithms tab. These mirror Niagara GL36 blocks documented in:

**README\_TRIM\_RESPOND.md**



FDD rules return **fault masks**. Algorithms return **setpoints or request integers** — the stubs below use `apply_algorithm_arrow` as the planned entryptpoint. For early testing in Rule Lab, you can adapt the logic into advisory FDD rules (boolean “sequence unhealthy” flags).

## 50.1 Shared lookback helper

```
import pyarrow.compute as pc

LOOKBACK_HOURS = 6

def _kit_lookback_stats(table, *, hours=None):
    h = hours if hours is not None else LOOKBACK_HOURS
    ts = pc.cast(table["timestamp"], "timestamp[us, UTC]")
    tmin = pc.min(ts).as_py()
    tmax = pc.max(ts).as_py()
    span_h = (tmax - tmin).total_seconds() / 3600.0 if tmin and
tmax else 0.0
    print(f"lookback={h}h rows={table.num_rows} start={tmin}
stop={tmax} span={span_h:.2f}h")
```

## 50.2 VAV zone cooling requests (0-3)

Simplified from GL36 VAV box request generator — zone temp vs cooling setpoint bands.

```
"""GL36 VAV cooling requests – doc stub."""

import pyarrow.compute as pc

ZONE_TEMP = "zn-t"
ZONE_SP = "zn-t-sp"
ZONE_DEMAND = "zn-cool-loop"
HIGH_DIFF_F = 5.0
MED_DIFF_F = 3.0
LOOP_ON = 95.0
LOOP_OFF = 85.0

def apply_algorithm_arrow(table, cfg, context=None):
    _kit_lookback_stats(table)
    tz = pc.cast(table[ZONE_TEMP], "float64")
    sp = pc.cast(table[ZONE_SP], "float64")
    diff = pc.subtract(tz, sp)
    demand = pc.cast(table[ZONE_DEMAND], "float64")
    # Last row request level for kit demo (full port needs timer
state)
```

```

last_diff = diff[-1].as_py() if table.num_rows else 0.0
last_demand = demand[-1].as_py() if table.num_rows else 0.0
if last_diff >= HIGH_DIFF_F:
    req = 3
elif last_diff >= MED_DIFF_F:
    req = 2
elif last_demand > LOOP_ON:
    req = 1
else:
    req = 0
print(f"cooling_requests={req} dT={last_diff:.1f}F
loop={last_demand:.0f}%")
return {"cooling_requests": req}

```

## 50.3 AHU duct static trim & respond (advisory fault)

Detect **duct static stuck high** while VAV pressure requests are low — trim & respond may not be trimming.

```

"""AHU duct static trim advisory – doc stub."""

import pyarrow.compute as pc

DUCT_SP = "duct-static"
DUCT_SP_MAX = 1.50
DUCT_SP_TRIM_TARGET = 0.80
VAV_PRESS_REQ_SUM = "vav-press-req-sum"
LOOKBACK_HOURS = 12

def apply_faults_arrow(table, cfg, context=None):
    _kit_lookback_stats(table, hours=LOOKBACK_HOURS)
    sp = pc.cast(table[DUCT_SP], "float64")
    reqs = pc.cast(table[VAV_PRESS_REQ_SUM], "float64")
    stuck_high = pc.and_(pc.greater(sp, DUCT_SP_TRIM_TARGET),
pc.less(reqs, 1.0))
    return pc.and_(stuck_high, pc.greater(sp, DUCT_SP_MAX * 0.9))

```

## 50.4 Chiller plant enable (valve request counting)

From GL36 §5.18.15.2 — count AHUs with CHW valve above threshold.

```

"""Chiller plant enable advisory – doc stub."""

import pyarrow.compute as pc

```

```

AHU_VALVE_COLS = ["ahu1-clg-vlv", "ahu2-clg-vlv", "ahu3-clg-vlv"]
REQ_THRESH = 95.0
DIS_THRESH = 10.0
MIN_AHUS_REQ = 2

def apply_algorithm_arrow(table, cfg, context=None):
    _kit_lookback_stats(table)
    requesting = 0
    for col in AHU_VALVE_COLS:
        if col not in table.column_names:
            continue
        v = pc.cast(table[col], "float64")[-1].as_py()
        if v >= REQ_THRESH:
            requesting += 1
    enable = requesting >= MIN_AHUS_REQ
    print(f"chiller_enable={enable} requesting_ahus={requesting}/
{MIN_AHUS_REQ}")
    return {"chiller_enable": enable, "requesting_ahus":
requesting}

```

## 50.5 Hot water plant HWST trim & respond (advisory)

Flags **HWST trimmed too low** while heating requests remain high.

```

"""HWST trim advisory – doc stub."""

import pyarrow.compute as pc

HWST = "hw-supply-t"
HWST_SP = "hw-supply-t-sp"
HEAT_REQ_SUM = "hw-reset-req-sum"
LOW_HWST_F = 120.0
LOOKBACK_HOURS = 6

def apply_faults_arrow(table, cfg, context=None):
    _kit_lookback_stats(table, hours=LOOKBACK_HOURS)
    temp = pc.cast(table[HWST], "float64")
    reqs = pc.cast(table[HEAT_REQ_SUM], "float64")
    return pc.and_(pc.less(temp, LOW_HWST_F), pc.greater(reqs,
2.0))

```

## 50.6 Chilled water plant (DP + CHWST reset advisory)

Flags **CHWST at design cold** with low cooling requests — possible stuck 100% plant loop.

```
"""CHW plant reset advisory – doc stub."""

import pyarrow.compute as pc

CHWST = "chw-supply-t"
CHWST_DESIGN_COLD_F = 44.0
COOL_REQ_SUM = "chw-reset-req-sum"
LOOKBACK_HOURS = 6

def apply_faults_arrow(table, cfg, context=None):
    _kit_lookback_stats(table, hours=LOOKBACK_HOURS)
    temp = pc.cast(table[CHWST], "float64")
    reqs = pc.cast(table[COOL_REQ_SUM], "float64")
    return pc.and_(pc.less(temp, CHWST_DESIGN_COLD_F + 1.0),
pc.less(reqs, 1.0))
```

## 50.7 Testing later

1. Bind historian columns in **Model & assignments** (fdd\_input / brick types).
2. Copy a stub into Rule Lab as an FDD advisory rule, or wait for the **Algorithms** tab kit export.
3. Run python run\_test.py from a downloaded kit and confirm \_kit\_lookback\_stats prints expected start / stop / span.

Reference implementation (Niagara Java):  
[README\\_TRIM\\_RESPOND.md](#).

# 51 Central plants (CHW / CTW / BLR)

## 52 Central plant Arrow recipes

Chiller plant, condenser water, cooling tower, boiler, and hot-water distribution faults use the same Arrow-native primitives as air-side rules.

### 52.1 Starter fault codes

| Code       | Primitive        | Recipe                                     |
|------------|------------------|--------------------------------------------|
| CHW-DT-001 | low_delta_t_mask | Low chilled-water $\Delta T$ syndrome      |
| CHW-DP-001 | reset_stuck_mask | CHW differential pressure reset stuck high |

### 52.2 Required point roles (CHW)

- chw\_supply\_temp, chw\_return\_temp —  $\Delta T$  rules
- chw\_dp, chw\_dp\_setpoint — reset stuck rules
- pump\_command, pump\_feedback — command/feedback mismatch

### 52.3 Synthetic test pattern

```
import pyarrow as pa
from open_fdd.arrow_runtime.primitives import low_delta_t_mask

table = pa.table({
    "chw_supply_temp": [44.0] * 10,
    "chw_return_temp": [44.5] * 10,
})
mask = low_delta_t_mask(
    table,
    {"min_delta_t": 4.0},
    supply_col="chw_supply_temp",
    return_col="chw_return_temp",
```

```
)
assert mask.to_pylist()[-1] is True
```

See [Fault codes — chiller plant]({{ "/fault-codes/" | relative\_url }}) (expanded in 3.0.19).

## 53 Dashboard overview

## 54 Dashboard overview

### 54.1 Primary areas

| Area                           | Purpose                                                                                |
|--------------------------------|----------------------------------------------------------------------------------------|
| <b>Home / Building insight</b> | Check-engine status; fault cards show equipment, rule, point, historian column, BACnet |
| <b>Trends</b>                  | Point history from feather store                                                       |
| <b>Faults</b>                  | Rule hits grouped by equipment family                                                  |
| <b>Rule Lab</b>                | Edit, lint, and test Python FDD rules (PyArrow kit zip)                                |
| <b>Algorithms</b>              | Supervisory GL36 trim & respond (coming soon)                                          |
| <b>Model &amp; assignments</b> | Equipment tree, FDD rule pins, commissioning JSON                                      |
| <b>BACnet</b>                  | Discovery, reads, commission workflows                                                 |
| <b>Settings / health</b>       | Stack status, auth mode                                                                |

### 54.2 Live updates

Dashboard tiles use REST polling and WS /ws/dashboard for selected live views (ticket auth via POST /api/auth/ws-ticket).

### 54.3 Roles

Read-only operators see trends and faults; integrators access Rule Lab and model import. See [Auth and roles](#)({{ "/operator-bridge/auth-roles/" | relative\_url }}).

# 55 Auth and roles

# 56 Auth and roles

## 56.1 Production (LAN / edge)

- Set `OFDD_AUTH_SECRET` and role passwords in `workspace/auth.env.local`.
- Bridge on non-loopback addresses **requires** auth unless explicit insecure dev flags are set (lab only).

| Role       | Typical access                   |
|------------|----------------------------------|
| viewer     | Read trends, faults, model tree  |
| operator   | Run batch FDD, acknowledge views |
| integrator | Rule Lab, bindings, model import |
| commission | BACnet writes (plus write guard) |
| agent      | Agent tools / app-edit gates     |
| admin      | Full configuration               |

Login: `POST /api/auth/login` → Bearer JWT on API calls.

## 56.2 Localhost dev

`OFDD_BRIDGE_HOST=127.0.0.1` with `OFDD_AUTH_DISABLED=1` is acceptable on a single machine. **Never** use auth-disabled mode on a building LAN.

Full hardening: `Security({ { "/security/" | relative_url } })`.

# 57 Rule Lab

# 58 Rule Lab

Rule Lab authors **Arrow-native** Python FDD rules against the feather historian.

## 58.1 Equipment-scoped test and export

Use **Equipment-scoped test & export** on Rule Lab:

1. Select site, equipment, and sensor (point).
2. **Test selected rule** or **Test all for equipment** against recent feather data.
3. **Download equipment kit** — zip with manifest.json, equipment.json, points.json, per-rule samples, expanded helper source, and commissioning export subset.

API: GET /api/rules/export-equipment-kit?equipment\_id=... · expanded helpers: GET /api/playground/rules/{id}/source-expanded.

## 58.2 Export all rules

Integrators can download **Export all rules** — one zip with per-rule kits, manifest.json, catalog snapshots, and model TTL. API: GET /api/rules/export-all-kit. Bench rules use **module constants** at the top of rule.py — no browser config panel, no config.json in the dev kit zip.

```
"""Bench OA-T out of bounds (Arrow)."""

import pyarrow.compute as pc

VALUE_COLUMN = "oa-t"
OAT_LOW = 68.0
OAT_HIGH = 88.0
LOOKBACK_HOURS = 1

def _kit_lookback_stats(table, *, hours=None):
    h = hours if hours is not None else LOOKBACK_HOURS
    ts = pc.cast(table["timestamp"], "timestamp[us, UTC]")
    tmin = pc.min(ts).as_py()
    tmax = pc.max(ts).as_py()
    span_h = (tmax - tmin).total_seconds() / 3600.0 if tmin and
tmax else 0.0
    print(f"lookback={h}h rows={table.num_rows} start={tmin}
stop={tmax} span={span_h:.2f}h")

def _kit_value_stats(table):
    vals = pc.cast(table[VALUE_COLUMN], "float64")
    print(
        f"column={VALUE_COLUMN} min={pc.min(vals).as_py():.2f} "
        f"max={pc.max(vals).as_py():.2f}
mean={pc.mean(vals).as_py():.2f}"
    )
```



```

def apply_faults_arrow(table, cfg, context=None):
    _kit_lookback_stats(table)
    _kit_value_stats(table)
    vals = pc.cast(table[VALUE_COLUMN], "float64")
    return pc.or_(pc.less(vals, OAT_LOW), pc.greater(vals,
OAT_HIGH))

```

## 58.3 Workflow

1. **Download kit** — PyArrow zip (see below)
2. **Edit constants** locally (VALUE\_COLUMN, limits, LOOKBACK\_HOURS, WINDOW\_SAMPLES)
3. **Run** `pip install -r requirements.txt` then `python run_test.py`
4. **Upload** rule.py on Rule Lab (integrator)

Upload validation (Phase B): AST parse, forbidden imports (**no pandas/numpy**), `apply_faults_arrow(table, cfg, context=None)` signature for fault rules. Script-mode rules must use **table** (PyArrow), not legacy **df** DataFrames — lint errors explain this for AI agents.

The bridge runs rules on **PyArrow tables** via `open_fdd.arrow_runtime`, and persists .py sources under `workspace/data/rules_py/`.

- **Quick test** — `POST /api/playground/test-rule` (default 3 h lookback)
- **Batch** — `POST /api/rules/batch / openfdd-fdd-loop timer (1 h default lookback)`
- **Update all records** — 24 h lookback, 6 h chunks (`use_chunks: true`)
- **Templates** — `GET /api/playground/arrow-templates`

Pin points to rules via **Model & assignments** (`/model`) commissioning JSON or BACnet tree right-click. Export shows Rule Lab **names** on each point (`fdd_rules_linked`); import uses rule **ids** (`fdd_rule_ids`).

Bench seed: `python3 scripts/setup_bench_afdd.py` — imports `bench_import_model.json` and bench cookbook rules with matching id/name pairs.

## 58.4 Dev kit zip (download)

**Download kit** on Rule Lab exports `openfdd-rule-kit-<rule_id>.zip` with:

| File             | Purpose                                                                                                            |
|------------------|--------------------------------------------------------------------------------------------------------------------|
| rule.py          | Arrow rule + <b>constants</b> at top; optional <code>_kit_lookback_stats / _kit_value_stats</code> helpers         |
| data.py          | <code>SITE_ID</code> , <code>LOOKBACK_HOURS</code> , <code>load_table()</code> reading <code>sample.feather</code> |
| sample.feather   | Full historian window for the kit lookback (all rows, not a CSV preview)                                           |
| run_test.py      | Runs <code>rule.apply_faults_arrow(table, {}, context=...)</code> and prints <code>flagged=N</code>                |
| requirements.txt | <code>open-fdd&gt;=3.0.1</code> and <code>pyarrow</code>                                                           |
| README.md        | Install and run instructions                                                                                       |
| column_map.json  | BRICK logical keys → feather column names                                                                          |
| kit_meta.json    | Export metadata (site, columns, row count, lookback hours)                                                         |

**Not included:** `config.json` (retired — tune thresholds as Python constants).

Local test:

```
unzip openfdd-rule-kit-*.zip
cd openfdd-rule-kit-*
pip install -r requirements.txt
python run_test.py
```

Expected console output:

```
site=demo lookback_h=3 rows=...
lookback=3h rows=... start=... stop=... span=2.98h
column=oa-t min=... max=... mean=...
flagged=...
```

## 58.5 Lookback windows

Rules that need multi-hour history should call `_kit_lookback_stats(table)` (see [\[Lookback window helper\]](#)(`{{ "/rule-cookbook/lookback-window/" | relative_url }}`)). Pass `hours=1, 3, 6, 12, or 24` to validate timestamp span in the console.

Rolling-window rules (`flatline`, `spread`) use `WINDOW_SAMPLES` (~12 samples ≈ 1 h at 5 min poll) — see [Windowing & debugging](#)(`{{ "/rule-cookbook/windowing-debugging/" | relative_url }}`).

## 58.6 Algorithms (supervisory — coming soon)

GL36 trim & respond and plant reset sequences will live on the **Algorithms**(`{{ "/operator-bridge/algorithms/" | relative_url }}`) tab with the same zip kit shape but `apply_algorithm_arrow` outputs. Reference: README\_TRIM\_RESPOND.

## 58.7 Related

- Model workflow(`{{ "/operator-bridge/model-workflow/" | relative_url }}`)
- Arrow recipes(`{{ "/rule-cookbook/arrow-recipes/" | relative_url }}`)
- GL36 algorithm stubs(`{{ "/rule-cookbook/gl36-algorithm-stubs/" | relative_url }}`) (doc-only)

# 59 JSON API driver

# 60 JSON API driver

Poll **HTTP or HTTPS** REST endpoints on the OT LAN (or public APIs such as OpenWeatherMap) and store extracted JSON values in the feather historian (`source=json_api`).

Use this driver for gateways, micro-PLCs, IoT hubs, vendor APIs, and **web weather** when you want to cross-check shaky local outdoor-air sensors against a reference.

## 60.1 Supported features

| Feature                | Support                                                                                 |
|------------------------|-----------------------------------------------------------------------------------------|
| Methods                | GET, POST                                                                               |
| Transport              | http:// and https://                                                                    |
| TLS certificate verify | On by default; disable for self-signed OT gateways                                      |
| Authentication         | None, <b>Bearer token</b> , <b>HTTP Basic</b> , or <b>query-string API keys via env</b> |
| Env placeholders       |                                                                                         |

| Feature            | Support                                                                              |
|--------------------|--------------------------------------------------------------------------------------|
|                    | <code>\${ENV:VAR}</code> or <code>\${VAR}</code> in URL, headers, body, bearer token |
| Custom headers     | Optional headers map (merged with auth headers)                                      |
| Value extraction   | Dot-path into JSON body (title, main.temp, weather.0.description)                    |
| Multi-extract poll | One HTTP request per URL group; fan-out to multiple json_path rows                   |
| Polling            | <b>1 / 5 / 15 / 30 min</b> or <b>1 hour</b> (shared with BACnet/Modbus)              |
| Historian          | workspace/json_api/polls/samples.csv → feather_store/json_api/                       |
| FDD merge          | Scheduled FDD merges bacnet + modbus + json_api columns by nearest timestamp         |

## 60.2 Secrets: json\_api.env.local

Copy workspace/json\_api.env.example → workspace/json\_api.env.local (gitignored). The bridge loads this file at startup and when run\_local.sh starts the stack.

Use placeholders in commissioning URLs so API keys never land in git:

```
https://api.openweathermap.org/data/2.5/weather?q=${ENV:OPENWEATHER_CITY}&appid=${ENV:OPENWEATHER_API_KEY}&units=${ENV:OPENWEATHER_UNITS}
```

Check which vars are configured: GET /api/json-api/env/status.

## 60.3 OpenWeatherMap showcase (recommended demo)

This is the flagship example of what the generic JSON API driver can do — **one URL, three historian columns**, env-based API key, browser tree like BACnet/Modbus.

### 60.3.1 1. Configure env

```
cp workspace/json_api.env.example workspace/json_api.env.local
# Edit: OPENWEATHER_API_KEY, OPENWEATHER_CITY, OPENWEATHER_UNITS
(imperial = °F)
```

Restart the bridge (or `./scripts/run_local.sh restart`) so env vars load.

### 60.3.2 2. Register from the UI

1. Sign in as **integrator** → **JSON API** tab.
2. Open the **OpenWeatherMap showcase** panel.
3. Use the **OpenWeatherMap** preset → **Register** (default poll 30 min; choose interval in the form).

Three endpoints appear under `api.openweathermap.org` in the commissioning tree:

| Label            | JSON path             | Historian column | Units       |
|------------------|-----------------------|------------------|-------------|
| web-oat-t        | main.temp             | web-oat-t        | degF / degC |
| web-rh           | main.humidity         | web-rh           | %           |
| web-weather-desc | weather.0.description | web-weather-desc | text        |

Poll worker deduplicates: **one GET** serves all three extractions per cycle.

### 60.3.3 3. Headless / script

```
# Standalone fetch (like the original playground tester)
python3 scripts/openweather_json_api_example.py

# Register via REST (integrator login)
python3 scripts/openweather_json_api_example.py --register --base
http://127.0.0.1:8000
```

### 60.3.4 4. FDD: local OA-T vs web weather

With BACnet oa-t and JSON API web-oat-t in the merged site frame, scheduled FDD compares local outdoor air to the weather service.

| Item        | Detail                                  |
|-------------|-----------------------------------------|
| Rule module | oat_vs_web_spread_1h.py                 |
| Fault code  | <b>BLD-B</b> (outdoor air sensor fault) |

| Item              | Detail                                                                                            |
|-------------------|---------------------------------------------------------------------------------------------------|
| Default threshold | <code> oa-t - web-oat-t  &gt; 8 °F</code><br>( <code>max_spread_f</code> in rule cfg)             |
| Acme rule id      | <code>acme-oat-vs-web-spread</code> (via<br><code>setup_gl36_fdd.py</code> )                      |
| Local OAT source  | RTU 1100-unknown-2 (Outdoor Air<br>Temperature Local) — same value<br>as boiler/AHU networked OAT |
| Web column        | <code>web-oat-t</code> from OpenWeather bundle                                                    |

**Model alias:** local BACnet historian column must be `oa-t`:

```
python3 scripts/acme_patch_oat_column.py --point-id 1100-unknown-2 --host "$ACME_HOST" --token "$TOKEN"
```

Or set `external_id` / `fdd_input` to `oa-t` on the chosen `Outside_Air_Temperature_Sensor` in the model.

**Units:** keep `OPENWEATHER_UNITS=imperial` when BACnet OAT is °F.

Enable the rule in Rule Lab (or `setup_gl36_fdd.py` push) and run an FDD batch. Missing either column → rule returns all-false (no crash).

## 60.4 Commissioning (UI)

The **JSON API** tab follows [Home Assistant sensor.rest](#):

1. Sign in → **RESTful sensor (JSON API)** tab.
2. Pick a **preset** (JSONPlaceholder, DummyJSON, httpbin, Open-Meteo, OpenAQ, OpenWeather) or enter a **resource URL**.
3. Add one or more **sensors** — each row is a `value_json_path` + historian **label** (multi-value from one GET, like HA `json_attributes`).
4. Click **Test resource** — probes without saving; shows extracted values and raw JSON.
5. Click **Register & poll** — writes `endpoints.csv`, enables polling at the selected interval.
6. Poll choices match BACnet/Modbus: **1 / 5 / 15 / 30 min** or **1 hour**.

### 60.4.1 API for agents

| Endpoint                               | Purpose                                                    |
|----------------------------------------|------------------------------------------------------------|
| GET <code>/api/json-api/presets</code> | Full preset catalog + poll intervals (human + AI readable) |

| Endpoint                                 | Purpose                                  |
|------------------------------------------|------------------------------------------|
| POST /api/json-api/test                  | Probe resource + multi-sensor extraction |
| POST /api/json-api/register-bundle       | Register one resource with many sensors  |
| POST /api/json-api/presets/{id}/register | One-click preset registration            |

## 60.4.2 Bench preset

Use **JSONPlaceholder** — **todo title** → **Test resource** to verify outbound HTTPS without field hardware.

## 60.5 Authentication examples

### 60.5.1 Query-string API key via env (OpenWeather)

```
{
  "url": "https://api.openweathermap.org/data/2.5/weather?q=${ENV:OPENWEATHER_CITY}&appid=${ENV:OPENWEATHER_API_KEY}&units=${ENV:OPENWEATHER_UNITS}",
  "method": "GET",
  "json_path": "main.temp",
  "label": "web-oat-t",
  "auth_type": "none"
}
```

### 60.5.2 Bearer token (API key)

```
{
  "url": "https://192.168.10.50/api/v1/status",
  "method": "GET",
  "json_path": "sensors.zone_temp",
  "label": "zone-temp",
  "auth_type": "bearer",
  "bearer_token": "${ENV:GATEWAY_API_KEY}",
  "verify_tls": false
}
```

### 60.5.3 HTTP Basic

```
{
  "url": "http://192.168.10.50/data.json",
  "method": "GET",
  "json_path": "value",
  "label": "sensor-value",
}
```

```

    "auth_type": "basic",
    "basic_user": "operator",
    "basic_password": "changeme"
  }

```

#### 60.5.4 POST with JSON body

```

{
  "url": "https://gateway.local/api/read",
  "method": "POST",
  "json_path": "result.pv",
  "label": "pv",
  "body": { "point": "AHU-1_SAT", "command": "read" },
  "auth_type": "bearer",
  "bearer_token": "..."
}

```

Credentials in endpoints.csv are redacted in API responses (\*\*\*).  
Prefer \${ENV:...} for production keys.

## 60.6 REST API

| Method | Path                              | Role        | Description                                         |
|--------|-----------------------------------|-------------|-----------------------------------------------------|
| GET    | /api/json-api/env/status          | operator+   | Which env vars are configured (no secret values)    |
| POST   | /api/json-api/presets/openweather | integrator+ | Register 3-point OpenWeather bundle + optional poll |
| GET    | /api/json-api/driver/tree         | operator+   | Commissioning tree grouped by host                  |
| GET    | /api/json-api/poll/status         | operator+   | Poll worker status                                  |
| POST   | /api/json-api/poll/once           | integrator+ | Run one poll cycle                                  |
| POST   | /api/json-api/request             | operator+   | One-shot HTTP request (no store)                    |
| POST   | /api/json-api/read_and_store      | integrator+ | Request + CSV + feather ingest                      |
| POST   | /api/json-api/refresh             | operator+   | Re-fetch one endpoint                               |
| PATCH  | /api/json-api/endpoint/poll       | integrator+ | Enable/disable poll interval                        |
| DELETE |                                   | integrator+ | Remove endpoint                                     |



| Method | Path                                      | Role | Description |
|--------|-------------------------------------------|------|-------------|
|        | /api/json-api/<br>endpoint/<br>{point_id} |      |             |

Full route list: `API routes({ { "/appendix/bridge_api/" | relative_url } })`.

## 60.7 Typical OT URLs

| Device type    | Example URL                                                      | Notes                                   |
|----------------|------------------------------------------------------------------|-----------------------------------------|
| Web weather    | <code>https://api.openweathermap.org/data/2.5/weather?...</code> | Env-based appid; 3 historian columns    |
| Gateway status | <code>http://192.168.1.50/api/status</code>                      | Often plain HTTP on LAN                 |
| HTTPS BMS API  | <code>https://10.0.0.12/rest/points/zone1</code>                 | May need <code>verify_tls: false</code> |
| Local mock     | <code>https://jsonplaceholder.typicode.com/todos/1</code>        | Bench / CI only                         |

## 60.8 Limitations

- Response body must be **JSON** (not XML or plain text).
- Numeric paths (e.g. `main.temp`) are stored as strings in poll CSV; FDD rules cast with `pc.cast(..., "float64")`.
- Outbound requests originate from the **bridge** container/host — ensure routing and firewall rules allow the edge box to reach the target (OT subnet or public internet for weather).

## 60.9 Disable polling (save API quota)

Stop scheduled calls without deleting endpoints: JSON API tree → select OpenWeather points → **Stop polling**, or integrator API:

```
curl -X PATCH http://127.0.0.1:8765/api/json-api/endpoint/poll \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"point_id":"ja-api-openweathermap-org-get-web-oat-t","enabled":false,"poll_interval_s":0}'
```

API keys live in `json_api.env.local` (gitignored) — never committed; audit logs do not record expanded URLs with `appid=`. See [Logging and audit](#)(`{{ "/ops/logging/" | relative_url }}`).

## 60.10 Smoke test

```
python3 scripts/smoke_multi_driver_stack.py
```

Validates JSON API ingest alongside BACnet and Modbus, feather isolation, BRICK scope, and FDD batch.

# 61 Algorithms

## 62 Algorithms (coming soon)

The **Algorithms** dashboard tab (`/algorithms`) is a placeholder for **supervisory Python sequences** — the mirror image of Rule Lab FDD:

|            | FDD (Rule Lab)                                       | Algorithms (planned)                                              |
|------------|------------------------------------------------------|-------------------------------------------------------------------|
| Purpose    | Detect faults                                        | Compute setpoints, request levels, plant enables                  |
| Entrypoint | <code>apply_faults_arrow(table, cfg, context)</code> | <code>apply_algorithm_arrow(table, cfg, context)</code> (planned) |
| Output     | Boolean fault mask                                   | Numeric outputs / structured dict                                 |
| Historian  | Same feather_store<br>PyArrow tables                 | Same                                                              |

Until the tab ships, draft patterns live in this doc and in [GL36 algorithm stubs](#)(`{{ "/rule-cookbook/gl36-algorithm-stubs/" | relative_url }}`).

### 62.1 GL36 Trim & Respond reference

Open-FDD algorithms will follow the same ASHRAE Guideline 36 trim & respond methodology already implemented for Niagara in:

#### **README\_TRIM\_RESPOND.md**

That guide covers:

- VAV zone request generators (cooling + static pressure)

- AHU duct static pressure reset
- AHU supply air temperature reset
- Central plant chiller / boiler enable and HWST / CHWST trim & respond

Porting those blocks to PyArrow is in progress; the dashboard tab shows **COMING SOON** until upload, batch, and BACnet write paths are wired.

## 62.2 Planned workflow (same kit shape as Rule Lab)

1. Download **algorithm kit** zip from the Algorithms tab (future)
2. Edit **constants** at the top of algorithm.py (no config.json)
3. Run `python run_test.py` locally against `sample.feather`
4. Upload algorithm.py when satisfied

Kit files will match the Rule Lab zip layout (see [Rule Lab — dev kit zip]({{ "/operator-bridge/rule-lab/" | relative\_url }}#dev-kit-zip-download)).

## 62.3 Lookback windows

Rules and algorithms that need multi-hour history should use the shared **lookback stats helper** documented in [Lookback window helper]({{ "/rule-cookbook/lookback-window/" | relative\_url }}). Pass `hours=1, 3, 6, 12, or 24` to print row count, timestamp start/stop, and span for console validation.

Scheduled batch defaults: **1 h** lookback on the FDD loop; **Update all records** in Rule Lab uses **24 h** with 6 h chunks.

## 62.4 AHU supervisory checks (doc-only stubs)

These are **not** uploaded rules yet — copy-paste references for future algorithm or FDD work.

### 62.4.1 Duct static pressure too high

Flags sustained high duct static (possible stuck setpoint, blocked filter, or trim/respond not trimming).

```
"""AHU duct static high – doc stub (Arrow constants)."""
```

```
import pyarrow.compute as pc
```

```

VALUE_COLUMN = "duct-static"
HIGH_INWC = 1.20
LOOKBACK_HOURS = 3
MIN_SAMPLES_ABOVE = 6

def _kit_lookback_stats(table, *, hours=None):
    h = hours if hours is not None else LOOKBACK_HOURS
    ts = pc.cast(table["timestamp"], "timestamp[us, UTC]")
    tmin = pc.min(ts).as_py()
    tmax = pc.max(ts).as_py()
    span_h = (tmax - tmin).total_seconds() / 3600.0 if tmin and
tmax else 0.0
    print(f"lookback={h}h rows={table.num_rows} start={tmin}
stop={tmax} span={span_h:.2f}h")

def apply_faults_arrow(table, cfg, context=None):
    _kit_lookback_stats(table)
    vals = pc.cast(table[VALUE_COLUMN], "float64")
    high = pc.greater(vals, HIGH_INWC)
    # Require several consecutive highs (~30 min at 5 min poll)
    from open_fdd.arrow_runtime.windows import
arrow_consecutive_true

    streak = arrow_consecutive_true(high, MIN_SAMPLES_ABOVE)
    return streak

```

## 62.4.2 Supply air temperature too cold (cooling coil over-delivering)

```

"""AHU SAT too cold vs setpoint – doc stub."""

import pyarrow.compute as pc

SAT_COLUMN = "sa-t"
SP_COLUMN = "sa-t-sp"
COLD_DELTA_F = 5.0
LOOKBACK_HOURS = 1

def _kit_lookback_stats(table, *, hours=None):
    h = hours if hours is not None else LOOKBACK_HOURS
    ts = pc.cast(table["timestamp"], "timestamp[us, UTC]")
    tmin = pc.min(ts).as_py()
    tmax = pc.max(ts).as_py()
    span_h = (tmax - tmin).total_seconds() / 3600.0 if tmin and
tmax else 0.0
    print(f"lookback={h}h rows={table.num_rows} start={tmin}
stop={tmax} span={span_h:.2f}h")

```

```
def apply_faults_arrow(table, cfg, context=None):
    _kit_lookback_stats(table)
    sat = pc.cast(table[SAT_COLUMN], "float64")
    sp = pc.cast(table[SP_COLUMN], "float64")
    return pc.less(pc.subtract(sat, sp), -COLD_DELTA_F)
```

## 62.5 Plant supervisory checks (doc-only)

See [GL36 algorithm stubs](#)({{ "/rule-cookbook/gl36-algorithm-stubs/" | relative\_url }}) for chiller plant enable, HWST trim & respond, and CHW DP/CHWST reset patterns aligned with the Niagara README.

## 62.6 Related

- [Rule Lab](#)({{ "/operator-bridge/rule-lab/" | relative\_url }}) — live FDD authoring
- [Model workflow](#)({{ "/operator-bridge/model-workflow/" | relative\_url }}) — point bindings
- [Windowing & debugging](#)({{ "/rule-cookbook/windowing-debugging/" | relative\_url }}) — rolling windows

# 63 Model workflow

# 64 Model workflow

## 64.1 Brick model

- **Equipment tree** — SPARQL over synced TTL (workspace/data/data\_model.ttl)
- **Point registry** — BACnet inventory → series\_id for historian bindings
- **Health score** — orphan points, missing fdd\_input, stale telemetry

## 64.2 Typical integrator flow

1. Import or sync BACnet point registry.

- 2. Map equipment classes (AHU, VAV, ...) in the model UI — set brick\_type on points and feeds on equipment (AHU→VAV).
- 3. Tag points with fdd\_input roles (zone temp, SAT, damper cmd, ...).
- 4. Bind Rule Lab rules to points, equipment, or BRICK classes (bindings in rules\_store.json).
- 5. Run POST /api/model/sync-ttl after bulk imports.
- 6. Validate on **Trend plot** with FDD overlays (?fdd=1) — faults render on a right-hand 0/1 axis per device scope.

AI Agent and commissioning JSON import can bulk-apply steps 2-4; integrator should review model health before production FDD. See FDD and assignments({{ "/operator-bridge/fdd-assignments/" | relative\_url }}) and Ollama and analytics({{ "/operator-bridge/ollama-analytics/" | relative\_url }}).

Export: GET /api/model/export (TTL/JSON). Commissioning bundle: GET /api/model/commissioning-export — includes per-point fdd\_rules\_linked (Rule Lab names) plus fdd\_rules[] catalog. Use **Import / export** tab preview table or **Point → FDD rule pins** before editing raw JSON. API detail in Appendix({{ "/appendix/bridge\_api/" | relative\_url }}).

## 65 Driver framework

## 66 Driver framework

Open-FDD edge commissioning uses a **shared driver pattern** for OT data sources. Each driver polls field devices on a schedule, stores long-format samples in workspace CSVs, and ingests into the **feather historian** under a distinct source tag for FDD and plots.

### 66.1 Drivers

| Driver                                                 | Protocol        | Feather source                            | Commissioning tab                         |
|--------------------------------------------------------|-----------------|-------------------------------------------|-------------------------------------------|
| <u>BACnet</u> ({{ "/bacnet/"   relative_url }})        | relative_url }} | MS/TP / IP (: 47808)                      | bacnet                                    |
| Modbus                                                 | Modbus TCP      | modbus                                    | Modbus                                    |
| [JSON API] ({{ "/drivers/json-api/"   relative_url }}) | relative_url }} | HTTP/HTTPS REST (+ [OpenWeather showcase] | relative_url }} #openweathermap-showcase- |

| Driver | Protocol | Feather source           | Commissioning tab  |
|--------|----------|--------------------------|--------------------|
|        |          | ({{ "/drivers/json-api/" | recommended-demo)) |

All three drivers share the same operator workflow:

1. **Discover or configure** — add devices/endpoints to the driver registry.
2. **Request once** — on-demand read to verify connectivity and JSON path / register map.
3. **Request & store** — append to poll CSV and write a feather shard.
4. **Enable polling** — 1 / 5 / 10 / 15 minute intervals via tree right-click or bulk toolbar.
5. **Poll all now** — force one worker cycle without waiting for the interval.

## 66.2 Data layout

```
workspace/
  bacnet/commissioning/  points.csv, discovered inventory
  bacnet/polls/          samples.csv
  modbus/commissioning/  registers.csv
  modbus/polls/          samples.csv
  json_api/commissioning/ endpoints.csv
  json_api/polls/        samples.csv
  data/feather_store/
    bacnet/{site_id}/    shard-*.feather
    modbus/{site_id}/
    json_api/{site_id}/
```

## 66.3 Historian and FDD

- **Plot tab** — numeric columns from bacnet and modbus sources (GET /api/timeseries/plot?source=...).
- **String JSON fields** — stored in feather under json\_api; use the JSON API tree or feather directly (plot is numeric-only).
- **FDD batch** — merges bacnet + modbus + json\_api historian columns by nearest timestamp; rules use BRICK-bound external\_id names (e.g. compare BACnet oa-t vs JSON API web-oat-t — see [JSON API]({{ "/drivers/json-api/" | relative\_url }})).
- **BRICK model** — bind points in **Model & assignments** commissioning JSON; sync TTL for SPARQL scope.

## 66.4 Rule Lab (Arrow upload/download)

1. **Download kit** — zip with rule.py (constants at top), data.py, sample.feather (~3h), run\_test.py, requirements.txt.
2. Local test: pip install -r requirements.txt then edit constants in rule.py and python run\_test.py.
3. **Upload rule.py** — Arrow-only; must define apply\_faults\_arrow(table, cfg, context=None).
4. Browser shows **read-only** source; use Quick test + Update all records.

API: GET /api/rules/export-kit, POST /api/rules/upload.

## 66.5 Smoke validation

From repo root on a running stack (http://127.0.0.1:8765):

```
python3 scripts/smoke_multi_driver_stack.py
python3 scripts/validate_modbus_temp_e2e.py --no-server
```

## 66.6 Related docs

- [Data flow](#)({{ "/architecture/data-flow/" | relative\_url }}) — poll → CSV → feather → FDD
- [API routes](#)({{ "/appendix/bridge\_api/" | relative\_url }}) — REST reference
- [\[BACnet polling\]](#)({{ "/bacnet/polling/" | relative\_url }}) — commission container loop

# 67 Operator Bridge

# 68 Operator Bridge

The Operator Bridge is the FastAPI service and React dashboard operators use daily.

| Page                                                                                    | Topic           |
|-----------------------------------------------------------------------------------------|-----------------|
| <a href="#">Dashboard overview</a> ({{ "/operator-bridge/dashboard/"   relative_url }}) | relative_url }} |
| <a href="#">Auth and roles</a> ({{ "/operator-bridge/auth-roles/"   relative_url }}     | relative_url }} |
|                                                                                         | relative_url }} |



| Page                                                                          | Topic           |
|-------------------------------------------------------------------------------|-----------------|
| <a href="#">Rule Lab</a> ({{ "/operator-bridge/rule-lab/"                     |                 |
| <a href="#">Algorithms</a> ({{ "/operator-bridge/algorithms/"                 | relative_url }} |
| <a href="#">Model workflow</a> ({{ "/operator-bridge/model-workflow/"         | relative_url }} |
| <a href="#">FDD and assignments</a> ({{ "/operator-bridge/fdd-assignments/"   | relative_url }} |
| <a href="#">Ollama and analytics</a> ({{ "/operator-bridge/ollama-analytics/" | relative_url }} |

REST details: [Appendix — API routes]({{ "/appendix/bridge\_api/" | relative\_url }}). Env files: [Workspace env files](#)({{ "/appendix/workspace-env-files/" | relative\_url }}).

# 69 Ollama and analytics

# 70 Ollama and analytics

## 70.1 What runs without login

The **Building status** home page (/) and GET /openfdd-agent/building-insight are **public** (same policy as fault status). Anyone on the LAN can read the briefing sentence and underlying numeric levers; only the **AI Agent** chat and stack revision endpoints require sign-in.

## 70.2 Ollama setup

1. Copy workspace/ollama.env.example → workspace/ollama.env.local.
2. Pick a model for available RAM (comments in the example file):

| RAM tier | Suggested model |
|----------|-----------------|
| 4 GB Pi  | qwen3:0.6b      |
| 8 GB     | qwen3:1.7b      |

| RAM tier | Suggested model |
|----------|-----------------|
| 16 GB    | qwen3:4b        |
| 32 GB    | qwen3:8b        |
| 64 GB    | qwen3:14b       |

1. Set OFDD\_OLLAMA\_BASE\_URL (default `http://127.0.0.1:11434`).
2. Optional: OFDD\_OLLAMA\_GPU\_MODE (`cpu` / `auto` / `gpu`),  
OFDD\_OLLAMA\_TIMEOUT\_S for slow CPUs.
3. Bootstrap helper: `./scripts/bootstrap_ollama.sh`.

Bridge reads env via `run_local.sh` / `compose`. Model resolution:  
`workspace/api/openfdd_bridge/ollama_profiles.py`.

## 70.3 Automatic analytics (deterministic, always on)

These pandas calculations run on a refresh interval and feed the home dashboard **whether or not Ollama is up**:

| Lever                     | Module                                                                | Refresh env                               | What it computes                                                                                                                                                        |
|---------------------------|-----------------------------------------------------------------------|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Zone temperature snapshot | <code>zone_temp_analytics.py</code>                                   | OFDD_ZONE_TEMP_INTERVAL_S (default 3600s) | Site-wide and per AHU zone temperature averages, spread out-of-band counts using BRICK feeds (AHU→VAV) which are modeled; falls back to all Zone_Air_Temperature points |
| Device poll health        | <code>device_poll_health.py</code>                                    | stack health cycle                        | BACnet device health seen / stale poll detection                                                                                                                        |
| Operational brief         | <code>building_insight.py</code> → <code>get_operational_brief</code> | same as insight                           | Compact numerical status for integrators                                                                                                                                |
| Root-cause hints          | <code>root_cause_hints.py</code>                                      | on insight refresh                        | Multi-zone fault chains using brick:feeds from                                                                                                                          |

Shared lookback window: OFDD\_ANALYTICS\_LOOKBACK\_DAYS (default 14) in `operational_analytics.py`.

API routes (auth optional unless noted):

- GET `/openfdd-agent/building-insight` — sentence + zone snapshot + fault sentences

- GET /openfdd-agent/zone-temp-analytics
- GET /openfdd-agent/device-poll-health
- GET /openfdd-agent/ollama/health — integrator diagnostics

## 70.4 When Ollama is used

`building_insight.get_building_insight():`

1. Refreshes zone + device snapshots (pandas).
2. Calls `ollama_client.health()`.
3. If Ollama is reachable and `should_use_ollama_for_insight()` passes, sends a **compact context** (status, zone levers, feeds chains) to the configured model for a one-sentence briefing (source: "ollama").
4. On failure, uses `_fallback_sentence()` (source: "deterministic").

Context assembly: `building_insight._compact_context()`. Prompt template references BRICK feeds and `root_cause_hints`.

## 70.5 AI Agent (authenticated)

/agent page uses POST /openfdd-agent/chat with tool access (`agent_tools.py`): BRICK graph, scope bundles, rule binding patches, model health, etc. This is separate from the public home briefing.

## 70.6 Code map

```
workspace/api/openfdd_bridge/
  building_insight.py      # cached home briefing + Ollama
sentence
  zone_temp_analytics.py   # zone temp levers (BRICK feeds)
  device_poll_health.py    # BACnet poll staleness
  operational_analytics.py # lookback window + methodology dict
  root_cause_hints.py      # multi-fault BRICK chain hints
  ollama_client.py         # HTTP client to Ollama
  ollama_profiles.py       # RAM tiers / model names
```

Dashboard consumers: `BuildingStatusPage.tsx`,  
`StackStatusStrip.tsx` (WebSocket /ws/dashboard).

# 71 FDD and assignments

# 72 FDD and assignments

## 72.1 What the product actually does

Open-FDD separates **data modeling** (BRICK equipment, points, feeds relationships) from **rule assignment** (which FDD rules apply to which points, equipment, or BRICK classes). Both are editable in **Model & assignments** and via commissioning JSON import.

### 72.1.1 BRICK modeling (integrator + AI-assisted)

| Concern              | Where it lives                       | Notes                                                                   |
|----------------------|--------------------------------------|-------------------------------------------------------------------------|
| Equipment tree       | ModelService + TTL sync              | AHU, VAV, sensors; equipment_type and brick_type on points              |
| brick:feeds          | model_feeds.py, ttl_service.py       | AHU→VAV→zone relationships; used by zone analytics and root-cause hints |
| Point registry       | BACnet inventory → model points      | fdd_input roles (oa-t, stat_zn-t, ...), series_id / historian column    |
| Commissioning import | POST /api/model/commissioning-import | Bulk JSON: sites, equipment, points, optional fdd_rules                 |

**AI Agent** (agent\_tools.py) can read slim\_brick\_graph, scope\_bundle, patch rule bindings, and upsert points — but it does **not** auto-generate a full site model without operator review. Expected workflow:

1. BACnet discover/poll populates point registry.
2. Integrator or agent tags brick\_type, fdd\_input, and feeds on equipment.
3. POST /api/model/sync-ttl writes workspace/data/data\_model.ttl.

4. Model health (model\_health.py) flags orphan points and missing fdd\_input.

### 72.1.2 Rule assignment (three binding kinds)

Saved rules in rules\_store.json carry bindings:

```
{
  "point_ids": ["5007-analog-input-1173"],
  "equipment_ids": ["bench-1"],
  "brick_types": ["Outside_Air_Temperature_Sensor"]
}
```

| Binding kind  | Effect                                                |
|---------------|-------------------------------------------------------|
| point_ids     | Rule runs for that historian point / series           |
| equipment_ids | Rule runs for all points on equipment (mass bind)     |
| brick_types   | Rule runs for any point with that BRICK class on site |

Assign via:

- **Model & assignments** UI — equipment/point chips, rule pin menu on trend plot
- **Rule Lab** — bindings panel
- **Commissioning import** — fdd\_rules[].bindings and per-point fdd\_rule\_ids
- **AI Agent** — patch\_rule\_binding tool

Runtime evaluation: fdd\_runner.py (batch FDD) and plot\_readings.evaluate\_fault\_plots() (trend overlays). Plot scope filters rules to those bound to **selected telemetry** (point, equipment, or BRICK class); unbound rules still evaluate site-wide.

**Brick bindings (3.0.20+):** when a rule binds brick\_types (or equipment\_ids / point\_ids), the batch runner resolves **every** matching historian column via historian\_columns\_for\_rule, runs the Arrow rule per column with value\_column set, and OR-combines fault masks. One rule config therefore applies consistently to all zone temps, discharge temps, etc.

### 72.1.3 CLASS vs device

- **BRICK class** (brick\_type on points, or brick\_types on rule bindings) — template-style assignment (“all Zone\_Air\_Temperature\_Sensor”).
- **Device / equipment** (equipment\_id, BACnet device instance) — instance assignment (“this VAV-12”).
- **Point** (point\_ids) — single-sensor assignment.

applies\_to on legacy rules is deprecated; use bindings in 3.0+.

## 72.2 Commissioning JSON shape

Export: GET /api/model/commissioning-export. Import accepts import\_ready\_json wrappers (see commissioningImport.ts).

### 72.2.1 Rule names vs ids (Rule Lab alignment)

Rule Lab shows **human names** (Bench OA-T flatline 1h). Saved rules use stable **ids** (bench-oa-t-flatline-1h from setup scripts, or a UUID for uploaded custom rules). Commissioning export includes both so you never have to cross-reference sections by hand:

| Field                     | Direction          | Purpose                                                                |
|---------------------------|--------------------|------------------------------------------------------------------------|
| points[].fdd_rule_ids     | import + export    | Assignment surface — array of rule ids                                 |
| points[].fdd_rules_linked | <b>export only</b> | [{id, name}] — Rule Lab names beside each point                        |
| fdd_rules[].id            | import + export    | Stable rule key (matches Rule Lab / rules_store.json)                  |
| fdd_rules[].name          | import + export    | Same label as Rule Lab dropdown                                        |
| fdd_rules[].source_file   | export             | Baseline of deployed .py when present (e.g. bench-oa-t-flatline-1h.py) |

On import, fdd\_rules\_linked is stripped (like fdd\_rule\_ids — not stored in model.json). Use the **Point → FDD rule pins** table on Model & assignments to read names without parsing JSON.

Minimum fields integrators/AI should produce:

- sites, equipment[] with id, equipment\_type, optional feeds[]
- points[] with site\_id, equipment\_id, brick\_type, fdd\_input, BACnet IDs, optional fdd\_rule\_ids
- fdd\_rules[] with id, name, bindings, enabled (optional source\_file on export)

After import, verify in **Trend plot** with ?fdd=1 — fault lanes appear on the right-hand 0/1 axis for bound rules.

## 72.3 Related

- [Model workflow](#)({{ "/operator-bridge/model-workflow/" | relative\_url }})
- [Rule Lab](#)({{ "/operator-bridge/rule-lab/" | relative\_url }})
- [\[Trend plot / fault overlays\]](#)({{ "/operator-bridge/dashboard/" | relative\_url }}#trend-plot)

# 73 Network setup

# 74 Network setup

## 74.1 Bind address

Commission and poll containers need the **OT interface** IP and UDP **47808**:

```
# workspace/bacnet/commissioning/commission.env
BACNET_BIND=192.168.1.50/24:47808
BACNET_INSTANCE=599999
```

- Use the NIC that can reach controllers (not only the management VLAN unless routed).
- Poll driver uses network\_mode: host on Linux edges.

## 74.2 Discovery range

```
DISCOVER_LOW=1
DISCOVER_HIGH=4194302
DISCOVER_TIMEOUT=30
```

Narrow the range in large sites to avoid broadcast storms.

## 74.3 Verify

From commission container logs or API: Who-Is → I-Am responses. If zero devices, check firewall, bind IP, and VLAN routing before tuning rules.

## 75 Discover and read

## 76 Discover and read

The **commission** container exposes HTTP APIs for discovery and read-property workflows used by the dashboard BACnet tools.

| Capability     | Notes                                                                                     |
|----------------|-------------------------------------------------------------------------------------------|
| Who-Is / I-Am  | Device discovery on OT subnet                                                             |
| Read property  | Present value, object name, units                                                         |
| RPM            | Bulk reads for inventory export                                                           |
| Write property | <b>Gated</b> — see <a href="#">Write safety</a> ( <code>{{ "/bacnet/write-safety/"</code> |

Exported inventory feeds `points_discovered.csv` and the Brick point registry.

### 76.1 Operator workflow

1. Confirm `network_setup({{ "/bacnet/network-setup/" | relative_url }}`).
2. Run discover from dashboard or API.
3. Review discovered objects; enable points for poll profiles.
4. Sync model bindings for FDD inputs.



# 77 Polling

# 78 Polling

**BACnet field reads run in the openfdd-commission container.**  
The Operator Bridge does not bind UDP 47808.

**Retired:** openfdd-bacnet-poll image and openfdd-bacnet-poll systemd unit. Poll loop lives in **commission** only — do not run both.

BACnet polling has two stages:

- 1. **Field reads** — commission poll loop → workspace/bacnet/polls/samples.csv
- 2. **Historian ingest** — bridge background worker → workspace/data/feather\_store/

## 78.1 Commission poll loop (default)

The **commission** container runs the poll loop at startup. It reads enabled rows from points.csv and appends BACnet RPM results to samples.csv.

| Setting        | Location                                      |
|----------------|-----------------------------------------------|
| BACnet bind    | workspace/bacnet/commissioning/commission.env |
| Enabled points | workspace/bacnet/commissioning/points.csv     |
| Poll output    | workspace/bacnet/polls/samples.csv            |
| Historian      | workspace/data/feather_store/                 |

Check activity:

```
tail -1 workspace/bacnet/polls/samples.csv
docker compose logs --since 5m commission | grep -i poll
```

## 78.2 Bridge ingest worker

Inside **bridge**, a background thread watches samples.csv and ingests new rows into feather. Disable only for debugging:  
OFDD\_DISABLE\_POLL\_WORKER=1.

# 78.3 Health

Dashboard stack health (GET /health/stack) reports bacnet\_poll status from the commission agent — not a separate container.

Stale points trigger data-quality fault patterns — see [Sensor quality faults]({{ "/fault-codes/sensor-quality/" | relative\_url }}).

# 78.4 Poll strategy (edge)

Open-FDD intentionally keeps BACnet field traffic **minimal**:

- **One enabled row per object** in points.csv — poll only what the model/FDD needs.
- **Uniform cycle** — every enabled point is RPM-polled each cycle; the loop sleeps max(15, min(poll\_interval\_s)) (typically **60 s** on Acme).
- Per-point poll\_interval\_s in the UI is the **commissioned target**; throughput analytics compare configured vs observed samples.
- **Unit conversion** (e.g. Trane °C → °F) uses device\_poll\_profiles.csv at ingest — not a poll-rate file.

Acme ships **FDD-minimal polling** (~76 points on a typical GL36 lab when 8 rules are enabled), not the full discovery tree (~340+ GL36 objects or 566 discovered objects). Disable unneeded objects in BACnet commissioning rather than raising global poll rates.

**AI agents / MCP:** Never enable BACnet polling for every discovered object. Poll **only** historian columns bound to **enabled** FDD rules (rules\_store.json brick\_types, point\_ids, and rule config column hints). Use bacnet\_toolshed.fdd\_minimal\_poll or acme\_commission\_fdd\_minimal.sh — not bulk --all enable scripts.

# 78.5 Async driver and single-stack I/O

| Layer                      | Behavior                                                                     |
|----------------------------|------------------------------------------------------------------------------|
| Dedicated bacnet-io thread | One asyncio loop + serial lock for all field I/O                             |
| RPM batching               | Present-value reads chunked (25 objects) to cut APDUs                        |
| Poll loop                  | Sequential per-device RPM when interruptible=True so operator UI can preempt |

| Layer          | Behavior                                                                                                       |
|----------------|----------------------------------------------------------------------------------------------------------------|
| Operator UI    | <b>INTERACTIVE</b> priority — cancels in-flight background work                                                |
| Override scans | <b>BACKGROUND</b> , 15 min timeout — see <a href="#">Operator override scans</a> ({{ “/bacnet/override-scans/” |
| Bridge ingest  | Async notify after each poll cycle → feather historian                                                         |

Do **not** run a second BACnet bind on 47808 (no parallel openfdd-bacnet-poll container). See [Containers](#)({{ “/architecture/containers/” | relative\_url }}).

## 78.6 Operator override scans

Hourly P8 supervisory scans rotate one device at a time. Documented in [Operator override scans](#)({{ “/bacnet/override-scans/” | relative\_url }}).

# 79 Operator override scans

## 80 Operator override scans (P8)

The commission agent rotates **one BACnet device per hour** (default) through a supervisory priority-array scan. Results are persisted for operator review and portfolio rollup — not for automatic writes.

### 80.1 Schedule

| Setting                         | Default | Meaning                             |
|---------------------------------|---------|-------------------------------------|
| OFDD_OVERRIDE_SCAN_INTERVAL_S   | 3600    | Seconds between single-device scans |
| OFDD_OPERATOR_OVERRIDE_PRIORITY | 8       | BACnet priority counted as          |

| Setting     | Default               | Meaning                                                      |
|-------------|-----------------------|--------------------------------------------------------------|
|             |                       | “operator override”                                          |
| Device list | points_discovered.csv | Unique device_instance rows                                  |
| Rotation    | registry.json cursor  | Full plant rotation $\approx$ device_count $\times$ interval |

Example: **33 devices  $\times$  1 h  $\approx$  33 h** for a full pass. Dashboard shows **next device** and **last scan** time.

## 80.2 What each scan does

1. point\_discovery on the target device (commandable objects).
2. read\_property on each object’s **priority-array** (not RPM — RPM loses priority slots).
3. Merge into workspace/bacnet/overrides/registry.json and append overrides\_export.csv.
4. Advance cursor to the next device.

Fault code **operator\_override** (P8) surfaces active overrides on the BACnet tree and building status.

## 80.3 BACnet I/O priority

Override scans run at **BACKGROUND** priority on the single shared BACnet stack (UDP 47808). Operator reads/writes use **INTERACTIVE** and preempt background work. Scheduled RPM polling also uses BACKGROUND and shares the same asyncio lock — polls and override scans take turns; neither starves the other indefinitely.

Override scans allow up to **15 minutes** per device (`_run_bacnet_override_scan` timeout). Bridge `POST /api/bacnet/overrides/scan-once` proxies with a matching 900 s timeout.

## 80.4 Manual trigger

```
curl -X POST -H "Authorization: Bearer $TOKEN" \
  http://EDGE:8765/api/bacnet/overrides/scan-once
```

Dashboard: BACnet page → **Scan next device now**.

## 80.5 Health checks

```
curl -H "Authorization: Bearer $TOKEN" http://EDGE:8765/api/bacnet/overrides/status
```

Expect `device_count > 0`, `last_scan_at` within the last few hours on a healthy edge, and `export_row_count` growing over time.

## 80.6 Data paths

| Path                                            | Content                                |
|-------------------------------------------------|----------------------------------------|
| workspace/bacnet/overrides/registry.json        | Per-device last scan + override points |
| workspace/bacnet/overrides/overrides_export.csv | Flat export for spreadsheets           |

Preserved across Docker upgrades when `workspace/` is on the edge volume — see [Backup & restore]({{ "/ops/backup-restore/" | relative\_url }}).

# 81 Write safety

# 82 Write safety

**BACnet writes can change live plant behavior.** Open-FDD disables writes by default. Enable only with operator sign-off, a tested allowlist, and audit logging.

## 82.1 Defaults

| Control            | Default                              |
|--------------------|--------------------------------------|
| Supervisory writes | <b>Off</b>                           |
| Write allowlist    | Empty — must enumerate device/object |
| Audit              | Commands logged when writes enabled  |

## 82.2 Enable writes (commission role)

1. Set env flag documented in [Security](#)({{ "/security/" | relative\_url }}) (OFDD\_BACNET\_WRITE\_ENABLED or site equivalent).
2. Populate write allowlist CSV / config with explicit object IDs.
3. Use **commission** role JWT; every write should include reason/ note where API supports it.
4. Test on bench hardware before production OT.

## 82.3 Operator rules

- Never write setpoints or overrides on life-safety or fire/smoke interlocks through experimental rules.
- Prefer read-only polling for FDD until rules are validated in Rule Lab.
- Revert overrides through BACnet priority relinquish where supported.

# 83 BACnet

# 84 BACnet

Open-FDD integrates with field controllers via BACnet/IP (and bench MSTP overlays in dev compose).

| Page                                                                                     | Topic           |
|------------------------------------------------------------------------------------------|-----------------|
| <a href="#">Network setup</a> ({{ "/bacnet/network-setup/"   relative_url }})            | relative_url }} |
| <a href="#">Discover and read</a> ({{ "/bacnet/discover-read/"   relative_url }})        | relative_url }} |
| <a href="#">Polling</a> ({{ "/bacnet/polling/"   relative_url }})                        | relative_url }} |
| <a href="#">Operator override scans</a> ({{ "/bacnet/override-scans/"   relative_url }}) | relative_url }} |
| <a href="#">Write safety</a> ({{ "/bacnet/write-safety/"   relative_url }})              | relative_url }} |

# 85 Convention

## 86 Fault code convention

### 86.1 Format (Grade-A, 3.0.18+)

**Short stable code:** FAMILY-SUBSYSTEM-NNN — e.g. AHU-ECON-001, CHW-DT-001, DATA-STAL-001.

**Semantic ID:** dotted canonical\_id —  
e.g. ahu.economizer.not\_using\_free\_cooling.

**Legacy letter codes** (AHU-E, VAV-C) remain **aliases** in open\_fdd/faults/catalog/\*.yaml until bridge sync completes.

| Field         | Example                               |
|---------------|---------------------------------------|
| code          | AHU-ECON-001                          |
| canonical_id  | ahu.economizer.not_using_free_cooling |
| family        | AHU, VAV, CHW, DATA, CRS, ...         |
| rule_doc_path | Link to rule cookbook recipe          |

Full schema: open\_fdd/faults/schema.py. Catalog loader: open\_fdd/faults/catalog.py.

### 86.2 Categories (Grade-A)

| Category           | Meaning                                        |
|--------------------|------------------------------------------------|
| energy             | Waste, reset stuck, plant inefficiency         |
| comfort            | Setpoint not met, poor zone performance        |
| reliability        | Short cycling, staging, equipment stress       |
| maintenance        | Leaking valves, filters, fouling proxies       |
| indoor_air_quality | Ventilation shortfall, humidity                |
| healthcare_risk    | Pressure, isolation, critical-space excursions |
| data_quality       | Stale, flatline, OOB, missing historian        |
| controls_integrity | Overrides, chatter, schedule violations        |

## 86.3 Severity

| Level    | Operator response                                       |
|----------|---------------------------------------------------------|
| info     | Log; trend for maintenance window                       |
| low      | Review in routine rounds                                |
| medium   | Investigate within days                                 |
| high     | Prompt attention; comfort/energy impact likely          |
| critical | Immediate operational risk (especially healthcare_risk) |

## 86.4 Linking rules

In Rule Lab, set `fault_code` on the rule metadata. Batch FDD aggregates hits into `GET /api/faults/status` by family.

## 86.5 Cookbook mapping

Each catalog entry lists `cookbook_patterns` (e.g. `flatline_1h`, `mixing_envelope`, `rate_of_change`) — see [Expression cookbook \(Arrow-native\)](#)(`{{ "/rule-cookbook/expression-cookbook/" | relative_url }}`).

Legacy pandas type: expression YAML rules are **not** used on the edge in 3.x; translate to `rules_py` Arrow modules and bind `fault_code` in Rule Lab.

# 87 Live site update

# 88 Live site update

Upgrade GHCR images on a **live edge host** that already has `~/open-fdd/` — no `git pull` on the host.

**IT operators:** prefer [\[Quick Start — Updating the stack\]](#) (`{{ "/quick-start/updating/" | relative_url }}`) and `openfdd_site_backup.sh` / `openfdd_site_update.sh` on the edge host. This page adds control-machine UI deploy and Ansible paths.



## 88.1 Layout on the edge host

```
~/open-fdd/  
  docker-compose.yml  
  workspace/          # site state – backup first
```

workspace/ holds BACnet commissioning, poll CSV, feather historian, BRICK model, FDD rules, and auth.env.local. Image upgrades must **preserve** it.

See [Backup and restore](#)(({ { "/ops/backup-restore/" | relative\_url } }) before any upgrade.

## 88.2 Concepts

| Term       | Meaning                                                               |
|------------|-----------------------------------------------------------------------|
| Container  | Running instance (bridge, commission, mcp-rag) — recreated on upgrade |
| Image      | GHCR package (tag latest or dated tag)                                |
| workspace/ | Open-FDD site state on disk — <b>do not delete</b>                    |

## 88.3 1. Verify new images on GHCR

Three images must exist for every tag:

```
export NEW_TAG="${NEW_TAG:-latest}"  
  
docker manifest inspect ghcr.io/bbartling/openfdd-bridge:${  
{NEW_TAG} >/dev/null &&  
docker manifest inspect ghcr.io/bbartling/openfdd-commission:${  
{NEW_TAG} >/dev/null &&  
docker manifest inspect ghcr.io/bbartling/openfdd-mcp-rag:${  
{NEW_TAG} >/dev/null &&  
echo "All three images exist for ${NEW_TAG}"
```

## 88.4 2. Update dashboard UI (when release changed React)

**Image pull alone does not update the browser UI.** The Operator Bridge serves workspace/api/static/app/ from the bind mount **before** image-baked assets.

**From control machine (full repo):**

```
cd /path/to/open-fdd
./scripts/build_operator_dashboard.sh prod
cd infra/ansible
RUN_POST_CHECK=0 ./deploy.sh ui --limit <inventory_host>
```

Confirm hash on edge:

```
curl -sf http://<edge-ip>/ | grep -o 'index-[^"]*\.' | head -1
```

**One command (UI + images + insurance check):**

```
OPENFDD_IMAGE_TAG=latest ./scripts/upgrade_edge_full.sh --limit
<inventory_host>
```

## 88.5 3. Pull and recreate (edge host)

```
cd ~/open-fdd
./scripts/openfdd_site_backup.sh
./scripts/openfdd_site_update.sh
```

Manual equivalent:

```
cd ~/open-fdd
docker compose pull
docker compose up -d --force-recreate
docker compose ps
curl -sf http://127.0.0.1:8765/health
```

## 88.6 4. Ansible image-only (control machine)

```
OPENFDD_IMAGE_TAG=latest RUN_POST_CHECK=1 ./scripts/
upgrade_edge_ghcr.sh --limit <inventory_host>
```

## 88.7 5. Validate

→ Deployment validation({ { “/ops/deployment-validation/” | relative\_url } })

## 88.8 Rollback

Restore docker-compose.yml from backup, pull previous tag, docker compose up -d --force-recreate. workspace/ is unchanged. Details: Backup and restore({ { “/ops/backup-restore/” | relative\_url } }).

## 88.9 Related

- [Updating the stack](#)({{ "/quick-start/updating/" | relative\_url }}) — edge-host helper scripts
- [Containers](#)({{ "/architecture/containers/" | relative\_url }}) — three-image architecture

## 89 Central collection

## 90 OpenFDD RCx Central — collection

**OpenFDD Edge** sites emit **interval summaries** (not raw historian by default). **OpenFDD RCx Central** ingests rollups over Tailscale/VPN, stores them locally, and feeds the Dash overview and validation jobs.

RCx Central is read-only toward Edge/BACnet. It runs on an analyst workstation or Docker Desktop — not on the OT edge by default.

### 90.1 Interval summary schema

Defined in `portfolio/store/interval_schema.py` as `FaultIntervalSummary`:

- Identity: `site_id`, `building_id`, `equipment_id`, `equipment_family`
- Window: `timestamp_start`, `timestamp_end`
- Fault: `fault_code`, `canonical_id`, `severity`, `confidence`
- Activity: `active_minutes`, `percent_active`, `occurrence_count`
- Evidence: `evidence_json` (summarized, not full trend)
- Tuning: `tuning_version`, `tuning_params_hash`
- Operator: `operator_status` (new | acknowledged | resolved | false\_positive | ...)

### 90.2 Edge export formats

- JSONL (append-only events)
- Parquet / Feather (batch intervals)

## 90.3 Collect today (rollup API)

```
source infra/ansible/secrets/acme.env.local
python3 scripts/portfolio_collect.py
```

## 90.4 Tuning proposals

Human-reviewable proposals use `TuningProposal` in the same module. The AI agent **never** auto-writes BACnet points on edge — use `GET /api/building-agent/tuning-brief` and `POST /api/building-agent/apply-tuning` with `apply: false` first. Maintainer workflow: `skills/openfdd-edge-deploy-tune/SKILL.md`.

# 91 AHU faults

# 92 AHU faults

Air handling unit codes (sample from catalog).

| Code  | Title                              | Severity | Detection summary                   | Likely causes                       | Operator checks                          |
|-------|------------------------------------|----------|-------------------------------------|-------------------------------------|------------------------------------------|
| AHU-A | Supply fan performance degradation | warning  | Spread/runtime vs baseline          | Belt slip, dirty filters, VFD limit | Trend speed vs airflow; filter dP        |
| AHU-B | Simultaneous heating and cooling   | critical | Heat + cool outputs active together | Sequencing bug, leaking valve       | Trend coil commands; valve feedback      |
| AHU-C | SAT sensor fault                   | warning  | Flatline / OOB SAT                  | Failed sensor, bad calibration      | SAT trend; compare to coil discharge     |
| AHU-D | MAT inconsistent with OAT/RAT      | warning  | MAT outside OAT-RAT envelope        | MAT/OAT/RAT sensor errors           | Verify damper positions; cross-check OAT |
| AHU-E | Economizer not economizing         | warning  | Free cool available, OA minimum     | Stuck damper, lockout               | OA damper vs OAT trend                   |

| Code         | Title                               | Severity | Detection summary   | Likely causes           | Operator checks             |
|--------------|-------------------------------------|----------|---------------------|-------------------------|-----------------------------|
| <b>AHU-F</b> | Damper command vs feedback mismatch | warning  | Cmd $\neq$ feedback | Stuck actuator, linkage | Compare cmd/feedback trends |

**Example rule:** flatline\_1h on SAT  $\rightarrow$  tag **AHU-C**. Recipe: [Python recipes]({{ "/rule-cookbook/python-recipes/" | relative\_url }}).

## 93 Standards crosswalk

## 94 Standards crosswalk

Grade-A fault definitions carry a standards\_crosswalk block in open\_fdd/faults/catalog/\*.yaml.

| Field                     | Source                                             | Use in Open-FDD                                               |
|---------------------------|----------------------------------------------------|---------------------------------------------------------------|
| ornl_lbnl_taxonomy        | ORNL/<br>LBNL<br>unified<br>HVAC fault<br>taxonomy | Canonical ontology<br>backbone                                |
| ashrae_g36_reference_type | ASHRAE<br>Guideline<br>36                          | Trim-and-respond,<br>plant requests,<br>supervisory sequences |
| ashrae_207_reference_type | ASHRAE<br>Standard<br>207                          | Economizer FDD test<br>categories                             |
| brick_classes             | <u>Brick</u><br><u>Schema</u>                      | Equipment/point<br>semantic roles                             |
| haystack_tags             | Project<br>Haystack<br>(optional)                  | Interoperability tags                                         |

### 94.1 Example

```
standards_crosswalk:
  ornl_lbnl_taxonomy: ahu/economizer/free_cooling
  ashrae_g36_reference_type: supply_air_temp_reset
  ashrae_207_reference_type: economizer_fault_detection
```

```
brick_classes: [brick:Air_Handler_Unit, brick:Economizer]
haystack_tags: [equip, ahu, econ]
```

```
Export full catalog: python3 -c "from open_fdd.faults.catalog
import catalog_export; import json;
print(json.dumps(catalog_export(), indent=2))"
```

## 95 Backup and restore

## 96 Backup and restore

workspace/ on the edge host is the **live site state** — feather historian, BACnet commissioning files, BRICK model, FDD rules, auth, and dashboard static bundle. Container images are replaceable; **backup workspace/ before every upgrade.**

### 96.1 Quick backup (edge host)

```
cd ~/open-fdd
./scripts/openfdd_site_backup.sh
```

Archives default to **~/openfdd-backups/latest** on the edge host (overwrites each run — one rolling copy for rigorous testing).

```
cd ~/open-fdd
./scripts/openfdd_site_backup.sh
```

Custom location (also overwritten each run unless you use a new path):

```
BACKUP_ROOT=~/openfdd-backups/manual ./scripts/
openfdd_site_backup.sh
```

Fast pre-upgrade backup (skips large workspace/bacnet/polls/ CSV history; feather/model/rules kept):

```
BACKUP_INCLUDE_POLL_SAMPLES=0 ./scripts/openfdd_site_backup.sh
```

On **bensserver**, site packs under `edge_backup/local/<site>/<building>/` are likewise overwritten by `edge_site_backup.sh` — no timestamp pile-up during validation.

From **bensserver** for any edge host (Ansible — backup runs **on the edge**, no workspace transfer over Tailscale):

```
OPENFDD_IMAGE_TAG=3.1.3 ./scripts/upgrade_edge_site.sh --limit
acme_vm_bbartling
```

```
OPENFDD_IMAGE_TAG=3.1.3 ./scripts/upgrade_edge_site.sh --limit
acme_vm_bbartling --fast-backup
```

Acme wrapper (same flow):

```
OPENFDD_IMAGE_TAG=3.1.3 ./scripts/upgrade_acme_site.sh --fast-
backup
```

## 96.2 Manual backup (SSH)

Container processes may write files as **root:root** with mode **0600**.  
Use **sudo** for a full archive:

```
cd ~/open-fdd

export BACKUP_ROOT="$HOME/openfdd-backups/$(date +%Y%m%d-%H%M%S)"
mkdir -p "$BACKUP_ROOT"

cp docker-compose.yml "$BACKUP_ROOT/docker-compose.yml.before"
docker compose ps > "$BACKUP_ROOT/docker-compose-ps-before.txt"
docker compose config --images > "$BACKUP_ROOT/docker-images-
before.txt"

sudo tar --xattrs --acls -czf "$BACKUP_ROOT/workspace-full.tgz"
workspace
sudo chown "$USER:$USER" "$BACKUP_ROOT/workspace-full.tgz"

du -h "$BACKUP_ROOT/workspace-full.tgz"
```

## 96.3 What to protect

| Path                                   | Contents                                     |
|----------------------------------------|----------------------------------------------|
| workspace/data/<br>feather_store/      | Feather historian                            |
| workspace/bacnet/<br>commissioning/    | points.csv, commission.env                   |
| workspace/bacnet/<br>polls/samples.csv | Poll output                                  |
| workspace/data/*.json                  | Model, rules store, FDD results              |
| workspace/<br>auth.env.local           | Login secrets                                |
| workspace/api/static/<br>app/          | Dashboard bundle (if deployed<br>separately) |

## 96.4 Restore after a bad upgrade

1. Stop containers: `docker compose stop` (do **not** use `down -v`)
2. Restore `docker-compose.yml` from backup
3. If data corruption: extract selective files from `workspace-full.tgz` — model, rules, feather shards
4. `docker compose pull && docker compose up -d --force-recreate`
5. Verify: `Deployment validation({{ "/ops/deployment-validation/" | relative_url }})`

## 96.5 Safe cleanup

```
docker image prune -f
```

**Never on live sites:** `docker compose down -v`, `docker volume prune`, `docker system prune -a --volumes`, or deleting `workspace/`.

## 96.6 Related

- `Updating the stack({{ "/quick-start/updating/" | relative_url }})`
- `Live site update({{ "/ops/live_site_update/" | relative_url }})`

# 97 OpenFDD RCx Central API

# 98 OpenFDD RCx Central API

Local analyst API for multi-edge RCx workflows over Tailscale/VPN. **Read-only toward OpenFDD Edge** — no BACnet, no commands.

Package path: `portfolio/central/api.py` — service name **openfdd-rcx-central**.

## 98.1 Start

```
pip install -r portfolio/requirements.txt
./scripts/run_central_api.sh # http://127.0.0.1:8060/health
./scripts/run_portfolio_dash.sh # http://127.0.0.1:8050
```



Bind defaults: 0.0.0.0 (OPENFDD\_CENTRAL\_API\_HOST, OPENFDD\_RCX\_DASH\_HOST). LAN: sudo ./scripts/open\_rcx\_central\_lan\_ports.sh.

Agent guide: <docs/agent-skills/rcx-central-dash-agent.md>

## 98.2 Edge registry (browser auth)

| Method | Path                         | Purpose                               |
|--------|------------------------------|---------------------------------------|
| GET    | /api/central/edges           | Saved Edge instances (secrets masked) |
| POST   | /api/central/edges           | Add/update Edge                       |
| POST   | /api/central/edges/test      | Test connection (health + model)      |
| PUT    | /api/central/edges/{site_id} | Update Edge                           |
| DELETE | /api/central/edges/{site_id} | Remove Edge                           |

## 98.3 Analytics & RCx

| Method | Path                                          | Purpose                                             |
|--------|-----------------------------------------------|-----------------------------------------------------|
| GET    | /api/central/overview/{site_id}               | Dash payload: KPIs, fault pie, building summary, P8 |
| GET    | /api/central/mechanical-summary/{site_id}     | Equipment counts, BACnet, narrative                 |
| GET    | /api/central/fdd-analytics/{site_id}          | FDD rules + fault descriptions                      |
| GET    | /api/central/fdd-preset/{site_id}/{preset_id} | Proxy Edge FDD/BRICK preset (Data Model tab parity) |
| POST   | /api/central/rcx/preview                      | Data readiness + chart catalog                      |
| POST   | /api/central/rcx/charts/preview               | Matplotlib PNG previews (base64)                    |
| POST   | /api/central/rcx/report                       | Download DOCX report                                |

## 98.4 Validation (legacy collect workflow)

| Method | Path                                          | Purpose                       |
|--------|-----------------------------------------------|-------------------------------|
| GET    | /api/central/sites                            | Edge registry + last check-in |
| POST   | /api/central/sites/{site_id}/collect-validate | Collect rollup + validation   |
| POST   | /api/central/validation/run                   | One-off or scheduled plan     |
| GET    | /api/central/validation/jobs                  | Stored jobs                   |
| GET    | /api/central/validation/jobs/{id}             | Job detail                    |

## 98.5 RCx report body

```
{
  "site_id": "acme",
  "hours": 24,
  "charts": ["fault_hours_by_severity", "ahu_sat_vs_setpoint"],
  "sections": ["executive_summary", "mechanical_summary"],
  "save_to_volume": true
}
```

Reports save under portfolio/data/reports/ (configurable via OPENFDD\_RCX\_CENTRAL\_DATA).

Legacy endpoint: POST /api/central/rcx/report-legacy

# 99 VAV faults

# 100 VAV faults

Variable air volume terminal codes (sample).

| Code         | Title                             | Severity | Detection summary                  | Likely causes                   | Operator checks                          |
|--------------|-----------------------------------|----------|------------------------------------|---------------------------------|------------------------------------------|
| <b>VAV-A</b> | Zone heating/cooling overlap      | warning  | Heat and cool at terminal          | Reheat valve stuck, SAT too low | Zone temp vs SAT; valve cmd              |
| <b>VAV-B</b> | Poor zone temperature control     | warning  | Persistent deviation from setpoint | Oversized box, bad tuning       | PI loop trends; occupancy                |
| <b>VAV-C</b> | Zone temperature sensor fault     | warning  | Flatline / OOB zone temp           | Failed zone sensor              | Flatline test; compare to neighbor zones |
| <b>VAV-D</b> | Minimum airflow not met           | warning  | Flow below minimum cmd             | Damper stuck, pressure issue    | Airflow vs damper cmd                    |
| <b>VAV-E</b> | Discharge temp deviation from SAT | warning  | DAT far from SAT setpoint          | Coil issue, sensor drift        | DAT vs SAT SP                            |

**Example rule:** oob\_rolling on zone temp → **VAV-C** or comfort bounds per site policy.

## 101 Docker images (GHCR)

## 102 Docker images (GHCR)

Full **Open-FDD edge/operator stack** — not the PyPI library.

### 102.1 Images

| GHCR image                       | Service    | Role                      |
|----------------------------------|------------|---------------------------|
| ghcr.io/bbartling/openfdd-bridge | bridge     | API, dashboard, historian |
|                                  | commission |                           |

| GHCR image                           | Service | Role                      |
|--------------------------------------|---------|---------------------------|
| ghcr.io/bbartling/openfdd-commission |         | BACnet discover/read/poll |
| ghcr.io/bbartling/openfdd-mcp-rag    | mcp-rag | MCP + doc-search sidecar  |

Image name openfdd-mcp-rag is retained for compatibility; the container runs the unified MCP server.

## 102.2 Tags

| Tag    | Meaning                                                    |
|--------|------------------------------------------------------------|
| 3.0.30 | Exact release (preferred for production)                   |
| 3.0    | Minor alias on release tags                                |
| latest | Latest <b>tagged</b> release only                          |
| edge   | Not published by default — use pinned versions on OT hosts |

```
export OPENFDD_IMAGE_TAG=3.0.30
docker pull ghcr.io/bbartling/openfdd-bridge:${OPENFDD_IMAGE_TAG}
docker pull ghcr.io/bbartling/openfdd-commission:${
{OPENFDD_IMAGE_TAG}
docker pull ghcr.io/bbartling/openfdd-mcp-rag:${OPENFDD_IMAGE_TAG}
```

## 102.3 Publish

Triggered by git tag vX.Y.Z → workflow **Publish Docker images to GHCR** (tags X.Y.Z, X.Y, latest).

Manual: Actions → **Publish Docker images to GHCR** (workflow\_dispatch) — reads version from pyproject.toml and publishes the same tags (3.1.3, 3.1, latest). Optional image\_tag input overrides the version.

## 102.4 vs PyPI

| Need                                 | Use                  |
|--------------------------------------|----------------------|
| Embed FDD in your Python/cloud job   | pip install open-fdd |
| BACnet + UI + bridge on an edge host | GHCR images          |

See `Release process({ { "/developer/release-process/" | relative_url })`).

## 103 RTU faults

## 104 RTU faults

Packaged rooftop unit codes (sample).

| Code  | Title                        | Severity | Detection summary           | Likely causes              | Operator checks                 |
|-------|------------------------------|----------|-----------------------------|----------------------------|---------------------------------|
| RTU-A | Compressor short cycling     | warning  | Rapid on/off cycles         | Low charge, oversized unit | Run-time histogram              |
| RTU-B | Economizer / OA damper fault | warning  | OA behavior vs conditions   | Actuator, sensor           | OA damper vs enthalpy/OAT       |
| RTU-C | Supply air temperature fault | warning  | SAT flatline or OOB         | Sensor failure             | SAT trend vs outdoor conditions |
| RTU-D | Simultaneous heat and cool   | critical | Heat + cool stages together | Control board, relay stuck | Stage status trends             |

Bind rules to packaged unit `fdd_input` points (SAT, OAT, compressor cmd, ...).

## 105 Deployment validation

## 106 Deployment validation

For the **live Acme site** after GHCR updates, use the dedicated harness documented in `Acme live validation({ { "/ops/acme-live-validation/" | relative_url })` (`acme_post_deploy_validate.sh`).

Non-destructive checks after deploy or image upgrade. Run from the **edge host** (smoke) or from a **control machine** with the full repo (insurance suite).

## 106.1 Edge host smoke (5 minutes)

```
cd ~/open-fdd

docker compose ps
curl -sf http://127.0.0.1:8765/health | jq . || curl -sf http://
127.0.0.1:8765/health

tail -1 workspace/bacnet/polls/samples.csv
du -sh workspace/data/feather_store/

docker compose logs --since 20m | grep -Ei "error|exception|
traceback|critical" | tail -20 || true
```

Via Caddy (building LAN):

```
curl -sf http://<edge-ip>/health
curl -sf http://<edge-ip>/ | grep -o 'index-[^"]*\.' | head -1
```

## 106.2 Browser checklist

1. Open `http://<edge-ip>/` (Caddy :80, not only loopback :8765)
2. Integrator login (`workspace/auth.env.local` on the edge host)
3. **Host stats** — container image tag / git SHA
4. **BACnet** — driver tree shows devices; last poll timestamp recent
5. **Model & assignments** — commissioning export returns JSON
6. **Rule Lab** — quick-test a saved rule (backend: arrow)

## 106.3 Control-machine insurance check

From a machine with the full open-fdd repo and Ansible inventory:

```
cd infra/ansible/scripts
./post_deploy_check.sh --limit <inventory_host> --full
```

Or by IP:

```
./post_deploy_check.sh --host <edge-ip> --ssh-user <deploy-user>
--full
```

Checks include: Caddy → React dashboard, `/health/stack`, MCP RAG, BACnet tree, BRICK/SPARQL, Docker log scan.

**Do not run on live commissioned sites without approval:** `acme_operational_verify.sh` with BACnet rediscover — use `--skip-discover` for smoke only. See [Examples — GL36 lab]({{ "/examples/acme-gl36-lab/" | relative\_url }}).

## 106.4 LAN security smoke (optional)

From a workstation on the same LAN as the edge:

- Windows: `scripts/security/Run-OpenFddSecurityScan.ps1`
- macOS/Linux: `scripts/security/run_openfdd_security_scan.sh`

Expected findings: [ZAP baseline]({{ "/security/zap-baseline/" | relative\_url }}).

## 106.5 Arrow / FDD rules (3.0+)

On control machine:

```
./scripts/validate_fdd_backends.sh --docker
```

On edge after upgrade:

```
grep -R "apply_faults_arrow" workspace/data/rules_py | wc -l
```

## 106.6 Related

- [Health check]({{ "/quick-start/health-check/" | relative\_url }})
- Security testing cycle({{ "/developer/security-testing/" | relative\_url }}) (maintainers)

# 107 Sensor and data quality

# 108 Sensor and data quality

Sensor faults use cookbook patterns `flatline_1h`, `oob_rolling`, `rate_of_change`, and `mixing_envelope`. Full thresholds: [Expression cookbook — sensor validation]({{ "/rule-cookbook/expression-cookbook/" | relative\_url }}#sensor-validation-bounds-flatline-rate-of-change).

## 108.1 Catalog codes

| Code         | Title                                      | Severity | Pattern                       | Detection summary                       |
|--------------|--------------------------------------------|----------|-------------------------------|-----------------------------------------|
| <b>BLD-B</b> | Outdoor air sensor fault                   | warning  | flatline, oob, rate_of_change | OAT flatline, OOB, or unrealistic spike |
| <b>BLD-D</b> | Stale telemetry                            | critical | stale_points                  | No new samples within threshold         |
| <b>AHU-C</b> | SAT sensor fault                           | warning  | flatline_1h, oob_rolling      | Supply air temp stuck or out of band    |
| <b>AHU-D</b> | MAT inconsistent with OAT/RAT              | warning  | mixing_envelope               | Mixed air outside OAT-RAT envelope      |
| <b>VAV-C</b> | Zone temperature sensor fault              | warning  | flatline_1h, oob_rolling      | Zone temp stuck or 55–90 °F band breach |
| <b>RTU-C</b> | Supply air temperature fault               | warning  | flatline_1h, oob_rolling      | Packaged unit SAT fault                 |
| <b>CH-D</b>  | CHW supply temperature sensor fault        | warning  | flatline_1h, oob_rolling      | Plant CHW sensor fault                  |
| <b>DC-C</b>  | Datacom sensor anomaly                     | warning  | flatline_1h                   | CRAH/room sensor flatline or RH OOB     |
| <b>HP-D</b>  | Discharge/suction temperature sensor fault | warning  | flatline_1h, oob_rolling      | Heat pump refrigerant/air temp sensor   |

## 108.2 Default bounds (imperial, occupied)

| Sensor    | Min   | Max   | Flatline tol | Max $\Delta$ /hr |
|-----------|-------|-------|--------------|------------------|
| Zone temp | 55 °F | 90 °F | 0.10 °F      | 4 °F             |



| Sensor                     | Min     | Max                    | Flatline tol | Max $\Delta$ /hr |
|----------------------------|---------|------------------------|--------------|------------------|
| Supply air temp            | 50 °F   | 110 °F                 | 0.15 °F      | 8 °F             |
| Return air temp            | 55 °F   | 95 °F                  | 0.10 °F      | 3 °F             |
| Duct static                | −0.5    | 3.0 inH <sub>2</sub> O | 0.02         | 0.5              |
| RH                         | 0 %     | 100 %                  | 1.0 %        | 15 %             |
| CHW                        | 40 °F   | 90 °F                  | 0.10 °F      | 4 °F             |
| HW                         | 70 °F   | 200 °F                 | 0.15 °F      | 6 °F             |
| Condenser water            | 50 °F   | 110 °F                 | 0.15 °F      | 5 °F             |
| CO <sub>2</sub> (occupied) | 400 ppm | 1000 ppm               | 5 ppm        | 200 ppm          |

Return-air checks use **narrow bands**; when MAT/OAT/RAT are available, prefer **mixing envelope** over RAT bounds alone.

## 108.3 Communication quality

- Monitor poll cycle timestamps in dashboard.
- Use `stale_points` for dropout detection (**BLD-D**).
- Do not confuse **comm loss** with **equipment fault** — verify BACnet device online before dispatching mechanical.

## 108.4 Operator workflow

1. Confirm **BLD-D** (stale) is resolved before trusting other sensor codes.
2. Cross-check **BLD-B** OAT against weather service or JSON API driver.
3. For **VAV-C** / **AHU-C**, compare neighbor zones / coil behavior before replacing sensor.

# 109 Logging and audit

## 110 Logging and audit

Open-FDD uses a **two-tier logging model** familiar to IT and security teams: structured **audit/error JSONL** for forensics and compliance, plus **service stdout** for engineering troubleshooting. Defaults include **age pruning**, **size rotation in MB**, and **Docker log caps** — no operator cron required.

### 110.1 Log types

| Tier                  | Location                                                         | Format                        | Who reads it                                     |
|-----------------------|------------------------------------------------------------------|-------------------------------|--------------------------------------------------|
| <b>Audit</b>          | workspace/logs/audit.jsonl                                       | JSON Lines                    | Security, MSI, integrator API                    |
| <b>Errors</b>         | workspace/logs/error.jsonl                                       | JSON Lines                    | Support, integrator API                          |
| <b>Service stdout</b> | Docker: docker compose logs ·<br>Dev: workspace/.local-run/*.log | Plain text (uvicorn, workers) | Engineering                                      |
| <b>Historian</b>      | workspace/data/feather_store/                                    | Feather                       | FDD, plots — separate retention (data.env.local) |

Structured audit logs are **not** mixed into uvicorn access logs. That separation matches common practice: SIEM-friendly JSONL for auth and OT commands; unstructured stdout for stack traces and poll worker noise.

### 110.2 Audit record schema

Each line in audit.jsonl / error.jsonl is one JSON object:

| Field      | Example                   | Notes                         |
|------------|---------------------------|-------------------------------|
| @timestamp | 2026-06-07T19:57:21+00:00 | UTC ISO-8601                  |
| event_id   | UUID                      | Unique per event              |
| event_type | auth.login.failure        | Stable taxonomy (see below)   |
| severity   | warning                   | debug ... critical            |
| outcome    | failure                   | success, failure, denied, ... |

| Field      | Example                         | Notes                                                          |
|------------|---------------------------------|----------------------------------------------------------------|
| service    | openfdd-bridge                  |                                                                |
| host       | hostname                        |                                                                |
| action     | login                           | Human-readable verb                                            |
| actor      | {username, role}                | After successful auth                                          |
| client     | {ip, user_agent}                | Respects<br>OFDD_TRUST_X_FORWARDED_FOR<br>behind Caddy         |
| http       | method, path, status            | When HTTP-triggered                                            |
| resource   | BACnet object, model id,<br>... | When applicable                                                |
| detail     | Sanitized map                   | <b>Passwords, tokens,<br/>secrets stripped by key<br/>name</b> |
| request_id | UUID                            | On audited HTTP paths                                          |

### 110.2.1 Auth events (industry-standard login trail)

| event_type         | When                                                   |
|--------------------|--------------------------------------------------------|
| auth.login.success | Valid credentials                                      |
| auth.login.failure | Bad password<br>(generic client<br>message)            |
| auth.login.lockout | Rate limit<br>exceeded (default<br>5 failures / 5 min) |

Login rate limits and lockouts are **audited**; responses never reveal whether the username exists.

### 110.2.2 OT / commissioning events

| event_type      | When                                                |
|-----------------|-----------------------------------------------------|
| bacnet.command  | Supervisory write (allowed, denied, dry-run)        |
| bacnet.discover | Who-Is / point discovery                            |
| model.write     | BRICK model mutations                               |
| agent.tool      | Agent tool calls (prompts hashed/truncated)         |
| api.access      | Sensitive POST/PUT/PATCH/DELETE (BACnet,<br>ingest) |
| error.unhandled | Unhandled exception (also in error.jsonl)           |

**Not audited:** routine GETs (dashboard reads, plot data, health). This reduces noise while keeping **auth and field commands** on the trail.

### 110.2.3 Secret redaction

Never logged: passwords, bearer tokens, `OFDD_AUTH_SECRET`, private keys. Detail objects drop keys containing password, token, secret, authorization. Agent tool arguments use additional hashing for prompts and rule source.

Opt-in full prompt logging (lab only): `OFDD_AUDIT_LOG_PROMPTS=1`.

## 110.3 Integrator API (read logs in the UI stack)

| Method | Path                                     | Role       |
|--------|------------------------------------------|------------|
| GET    | <code>/api/audit/events?limit=100</code> | integrator |
| GET    | <code>/api/audit/errors?limit=100</code> | integrator |
| GET    | <code>/api/audit/summary</code>          | integrator |

Use these for post-incident review without shell access. Forward `workspace/logs/*.jsonl` to SIEM (rsyslog, Filebeat, Wazuh) on production edges.

## 110.4 Retention and rotation (defaults)

Configured in `workspace/data.env.local` (copy from `data.env.example`):

| Variable                               | Default   | Behavior                                                                                                                            |
|----------------------------------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>OFDD_LOG_RETENTION_DAYS</code>   | <b>90</b> | Drop audit/error JSONL lines older than N days; delete aged <code>.local-run</code> and archived <code>*.log / audit/*.jsonl</code> |
| <code>OFDD_LOG_MAX_MB</code>           | <b>50</b> | When <code>audit.jsonl</code> or <code>error.jsonl</code> exceeds N MB, archive to <code>audit.{UTC}.jsonl</code> and start fresh   |
| <code>OFDD_LOCAL_RUN_LOG_MAX_MB</code> | <b>25</b> |                                                                                                                                     |

| Variable                       | Default | Behavior                                                                                                                            |
|--------------------------------|---------|-------------------------------------------------------------------------------------------------------------------------------------|
| OFDD_LOG_ROTATE_INTERVAL_HOURS | 6       | Rotate workspace/.local-run/*.log (bridge, Caddy, FDD loop, ...) Background retention while bridge is running (not only at restart) |

Rotation runs at **bridge startup** and on the **scheduled interval**. No logrotate.d entry is required for audit JSONL.

Optional path overrides:

```
OFDD_AUDIT_LOG_PATH=/var/log/openfdd/audit.jsonl
OFDD_ERROR_LOG_PATH=/var/log/openfdd/error.jsonl
```

## 110.5 Docker edge: json-file caps (not journald for app audit)

Production stacks use **Docker Compose** (docker/compose.edge.yml). Container stdout uses the **json-file driver with size limits**:

```
logging:
  driver: json-file
  options:
    max-size: "25m"
    max-file: "5"
```

That caps each service at roughly **125 MB** of rotated container logs on disk — the same pattern many teams use instead of unbounded docker logs.

### Why audit JSONL stays on disk (not journald)?

- Append-only files under workspace/logs/ survive container recreate and bind-mount to the host.
- JSONL lines import cleanly into SIEM without journal field parsing.
- OT incident response often needs **months** of auth/BACnet write history on the edge box.

**When journald still applies:** native systemd units (Caddy, Ollama on bare metal) — use `journalctl -u <unit>` per your Linux runbook. Bridge audit remains in `workspace/logs/` even when uvicorn stdout goes to Docker or `.local-run/`.

## 110.6 Local dev (run\_local.sh)

| File                                | Service                 |
|-------------------------------------|-------------------------|
| workspace/.local-run/bridge.log     | Bridge / uvicorn        |
| workspace/.local-run/commission.log | BACnet commission agent |
| workspace/.local-run/caddy.log      | Caddy                   |
| workspace/.local-run/fdd_loop.log   | Scheduled FDD           |
| workspace/.local-run/mcp_rag.log    | MCP RAG                 |
| workspace/.local-run/ollama.log     | Ollama                  |

These are **engineering logs**. Structured audit still lands in `workspace/logs/audit.jsonl` when auth is enabled.

## 110.7 Quick checks

```
# Last auth failures
grep 'auth.login' workspace/logs/audit.jsonl | tail -5

# Integrator API (after login)
curl -s -H "Authorization: Bearer $TOKEN" http://127.0.0.1:8765/
api/audit/summary | jq .

# Docker edge
docker compose logs bridge --tail 50
```

## 110.8 Comparison to typical web apps

| Practice                        | Open-FDD                                  |
|---------------------------------|-------------------------------------------|
| Structured auth audit           | Yes — JSONL with actor, IP, outcome       |
| Login rate limit + logout audit | Yes                                       |
| Secret redaction in logs        | Yes — key-based + agent sanitization      |
| Request correlation ID          | Yes — on audited mutation paths           |
| Centralized stdout JSON         | No — stdout is plain text; audit is JSONL |

| Practice                 | Open-FDD                                          |
|--------------------------|---------------------------------------------------|
| Full HTTP access log     | No — mutations and auth only (OT noise reduction) |
| Log rotation / size caps | Yes — defaults above + Docker max-size            |
| SIEM-ready export        | Yes — file tail / Filebeat on workspace/logs/     |

For BACnet write safety and credential generation, see [SECURITY.md](#) and [Secrets and auth](#)({{ "/security/secrets-auth/" | relative\_url }}).

## 110.9 Related

- [JSON API driver](#)({{ "/drivers/json-api/" | relative\_url }}) — OpenWeatherMap showcase, env-based API keys (json\_api.env.local)
- [Live site update](#)({{ "/ops/live\_site\_update/" | relative\_url }}) — docker compose logs after upgrades
- [Data flow](#)({{ "/architecture/data-flow/" | relative\_url }}) — historian retention (OFDD\_FEATHER\_\*) is separate from audit logs

# 111 GHCR image retention

## 112 GHCR image retention

Open-FDD publishes edge images to **GHCR** (ghcr.io/bbartling/openfdd-\*). During rapid development, old package versions accumulate. This retention system prunes **noise** while protecting rollback tags for **Acme**, our live validation building.

### 112.1 Policy (conservative)

**Always keep:**

- latest and edge
- Latest 5 SemVer release tags per image (configurable)
- Tags listed in scripts/ghcr-retention-protected-tags.txt
- Current Acme deployed tag (workflow input or --current-acme-tag)
- Previous Acme known-good tag (--previous-acme-tag)

### Delete candidates (only with `--confirm-delete`):

- Untagged versions older than **7** days
- SHA-only tags older than **30** days
- Feature/dev-style tags older than **30** days
- SemVer releases beyond the latest N window (unless protected)

Images are defined in `docker/images.yaml`:

- `openfdd-bridge`
- `openfdd-commission`
- `openfdd-mcp-rag`
- `openfdd-cloud-exporter`

## 112.2 Dry-run first

The script defaults to **dry-run**. It prints a keep/delete table and optional JSON/Markdown reports. Nothing is deleted until you pass `--confirm-delete`.

There is **no scheduled automatic deletion** yet — only manual runs locally or via GitHub Actions `workflow_dispatch`.

## 112.3 Protect Acme rollback tags

Before cleanup, edit `scripts/ghcr-retention-protected-tags.txt`:

```
latest
edge
v3.0.31
v3.0.30
```

Or pass tags at runtime:

```
./scripts/ghcr_prune_packages.sh --all-images --dry-run \  
  --current-acme-tag v3.0.31 \  
  --previous-acme-tag v3.0.30
```

## 112.4 Run locally

```
gh auth login    # needs read:packages + delete:packages for  
confirm-delete
```

```
./scripts/ghcr_prune_packages.sh --all-images --dry-run  
./scripts/ghcr_prune_packages.sh --image openfdd-bridge --dry-run \  
  --json-out reports/ghcr-prune-plan.json \  
  --confirm-delete
```



```
--markdown-out reports/ghcr-prune-plan.md

# After reviewing the plan:
./scripts/ghcr_prune_packages.sh --all-images --confirm-delete \
--current-acme-tag v3.0.31 --previous-acme-tag v3.0.30
```

## 112.5 GitHub Actions workflow

Workflow: **Prune old GHCR images** (.github/workflows/ghcr-prune.yml)

1. Actions → **Prune old GHCR images** → **Run workflow**
2. Leave **Dry run only** = true for the first run
3. Download the ghcr-prune-report artifact
4. Re-run with **Dry run only** = false only after reviewing the plan
5. Set **Current Acme deployed tag** and **Previous Acme rollback tag** when known

## 112.6 Permissions

Deleting package versions requires:

- Local: gh auth login with delete:packages (and read:packages)
- Actions: permissions.packages: write on the workflow (already set)

If deletion fails, the script prints:

```
Deleting GHCR package versions requires package delete
permissions...
```

## 112.7 Verify images still pull

After pruning:

```
docker pull ghcr.io/bbartling/openfdd-bridge:latest
docker pull ghcr.io/bbartling/openfdd-commission:latest
```

On Acme:

```
export OPENFDD_IMAGE_TAG=v3.0.31
./scripts/upgrade_edge_ghcr.sh --limit acme_vm_bbartling
./scripts/acme_post_deploy_validate.sh --limit acme_vm_bbartling
--quick
```

# 112.8 Rollback example

```
export OPENFDD_IMAGE_TAG=v3.0.30
./scripts/upgrade_edge_ghcr.sh --limit acme_vm_bbartling
```

# 112.9 Related issues

Track open Acme/deploy blockers in [GitHub issue #276](#).

# 113 Fault Codes

# 114 Fault Codes

Open-FDD uses a **check-engine** model: one building-level status (green / yellow / red) with a catalog of fixed fault codes per equipment family.

| Page                                                          | Content         |
|---------------------------------------------------------------|-----------------|
| <a href="#">Convention</a> ({{ "/fault-codes/convention/"     | relative_url }} |
| <a href="#">AHU faults</a> ({{ "/fault-codes/ahu/"            | relative_url }} |
| <a href="#">VAV faults</a> ({{ "/fault-codes/vav/"            | relative_url }} |
| <a href="#">RTU faults</a> ({{ "/fault-codes/rtu/"            | relative_url }} |
| [Sensor & data quality]({{ "/fault-codes/sensor-quality/"     | relative_url }} |
| <a href="#">Short lookup</a> ({{ "/fault-codes/short-lookup/" | relative_url }} |

Live catalog API: GET /api/faults/catalog on the Operator Bridge.

# 115 Operations

## 116 Operations

Runbooks for live edge hosts after the initial `Quick Start({{ "/quick-start/" | relative_url }})` deploy.

| Page                                                                                                           | When to use |
|----------------------------------------------------------------------------------------------------------------|-------------|
| <a href="#">Live site update({{ "/ops/live_site_update/"   relative_url }})</a>                                |             |
| <a href="#">Backup and restore({{ "/ops/backup-restore/"   relative_url }})</a>                                |             |
| <a href="#">Logging and audit({{ "/ops/logging/"   relative_url }})</a>                                        |             |
| <a href="#">Deployment validation({{ "/ops/deployment-validation/"   relative_url }})</a>                      |             |
| <a href="#">ACME live validation({{ "/operations/acme-live-validation/"   relative_url }})</a>                 |             |
| <a href="#">Bench 5007 long FDD smoke({{ "/operations/bench-5007-long-fdd-smoke/"   relative_url }})</a>       |             |
| <a href="#">Bench 5007 dual-source smoke({{ "/operations/bench-5007-dual-source-smoke/"   relative_url }})</a> |             |

Site-specific lab notes (example BACnet scope, GL36 rules):  
[Examples & lab notes\({{ "/examples/" | relative\\_url }}\)](#).

For first-time deploy: `[Quick Start — Docker]({{ "/quick-start/docker/" | relative_url }})`.

# 117 Portfolio

## 118 Portfolio (central desk)

Multi-site analytics on **benserver** — not part of the edge Docker stack.

| Topic                           | Page                                                                     |
|---------------------------------|--------------------------------------------------------------------------|
| Collect rollups from edge sites | <a href="#">Central collection</a> ({{ "/"portfolio/central-collection/" |

Repo layout: portfolio/ (collector, Dash, sites.json). Maintainer tuning API flow: skills/openfdd-edge-deploy-tune/SKILL.md.

## 119 Acme VAV/AHU rule guidance

## 120 Acme VAV/AHU FDD rule bundle

Practical Arrow-native rule set for **acme** (vm-bbartling): one RTU/AHU (acme-vm-bbartling-rtu-01, BACnet **1100**) feeding **JCI VAVs** (instances 1-100, imperial °F at wire) and **Trane VAVs** (11000-13000, metric °C converted to °F before feather via device\_poll\_profiles.csv).

Install / refresh rules:

```
python3 scripts/setup_gl36_fdd.py --site-id acme --building-id vm-bbartling \
--ahu-equipment-id acme-vm-bbartling-rtu-01 --fan-point-id 1100-analog-output-1
```

Validate bundle locally:

```
python3 scripts/acme_validate_fdd_bundle.py
```

## 120.1 Mixed manufacturers and units

| Manufacturer | BACnet instances | Wire units | Open-FDD handling                                                                                     |
|--------------|------------------|------------|-------------------------------------------------------------------------------------------------------|
| JCI VAV      | 1-100            | °F         | Rules use temp_unit: imperial; no poll conversion                                                     |
| Trane VAV    | 11000-13000      | °C         | metric_temp_f in edge_config/acme/vm-bbartling/device_poll_profiles.csv converts to °F before feather |
| RTU AHU      | 1100             | °F         | AHU rules bind to equipment / fan point                                                               |

All zone-temperature rules share **imperial bounds** after poll conversion. If a new Trane box is added without a profile row, zone OOB/flatline will be wrong until the profile is added and data re-ingested.

Command scaling (damper, reheat, fan): rules normalize **0-1 vs 0-100%** with `pc.if_else(greater(c, 1.0), divide(c, 100), c)`.

Poll interval: Acme BACnet poll loop is **60 s**. Flatline windows use `poll_interval_s: 60` and `flatline_minutes: 60` → **60 samples per hour**, not the legacy 5-min × 12 assumption.

## 120.2 Rule matrix

| Phase | Priority | Rule ID (prefix acme-) | Module                  | Fault | Equip |
|-------|----------|------------------------|-------------------------|-------|-------|
| 0     | 1        | oat-bounds             | oat_sensor_bounds.py    | BLD-B | Build |
| 0     | 2        | oat-flatline-1h        | oat_sensor_flatline.py  | BLD-B | Build |
| 0     | 3        | oat-spike              | oat_sensor_spike.py     | BLD-B | Build |
| 0     | 3b       | oat-vs-web-spread      | oat_vs_web_spread_1h.py | BLD-B | Build |

| Phase | Priority | Rule ID<br>(prefix<br>acme-) | Module                             | Fault | Equipment |
|-------|----------|------------------------------|------------------------------------|-------|-----------|
| 0     | 4        | zn-t-oob-occupied            | vav_zone_temp_bounds_occupied.py   | VAV-C | VAV       |
| 0     | 5        | zn-t-flatline-1h             | vav_zone_temp_flatline_occupied.py | VAV-C | VAV       |
| 0     | 6        | da-t-flatline-1h             | flatline_1h.py                     | VAV-E | VAV       |
| 0     | 7        | sat-flatline-1h              | flatline_1h.py                     | AHU-C | AHU       |
| 0     | 8        | sap-flatline-1h              | flatline_1h.py                     | AHU-F | AHU       |
| 1     | 9        | vav-damper-stuck             | vav_damper_stuck_flatline.py       | VAV-D | VAV       |
| 1     | 10       | vav-airflow-low              | vav_airflow_low.py                 | VAV-D | VAV       |
| 1     | 11       | vav-reheat-warm-ambient      | zone_reheat_warm_ambient.py        | VAV-A | VAV       |
| 1     | 12       | vav-reheat-leak              | vav_reheat_dat_vs_sat.py           | VAV-A | VAV       |
| 2     | 13       | ahu-afterhours-runtime       | ahu_afterhours_runtime.py          | BLD-C | AHU       |
| 2     | 14       | mat-oob-economizer           | oob_rolling.py                     | AHU-D | AHU       |
| 2     | 15       | ahu-run-hours                | ahu_run_hours.py                   | —     | AHU       |
| —     | —        | Stale telemetry              | Poll health dashboard              | BLD-D | All       |

### 120.2.1 Phase 2+ (not enabled by default)

Enable only after point coverage on RTU **1100** is verified (OAT, MAT, RAT, SAT, OA damper, cooling cmd):

- **AHU-A** duct static not maintained — needs static SP + fan speed (future rule)
- **AHU-D** mixed air envelope — `mixing_envelope_mask` + fan on
- **AHU-E** economizer trio — 100% OA tracking, mech cooling when free cooling available, OA damper excess open

### 120.2.2 Phase 3 rollups (future)

- Many simultaneous **VAV-D** airflow-low → suggest AHU static / fan issue
- Many **VAV-B/C** comfort faults → AHU scheduling or SAT problem
- Hot-water plant support when HW supply temp is modeled

## 120.3 Data model checks

Before enabling more rules:

1. **No duplicate BACnet device instances** on equipment rows (poll health merges by `bacnet_device_id`).
2. **No duplicate point\_id** rows with different `external_id` / `fdd` aliases (causes `historian_lag` double-count).
3. **Trane devices** in `device_poll_profiles.csv` with `metric_temp_f`.
4. Run `scripts/acme_validate_fdd_bundle.py` after model edits.

## 120.4 Tuning notes

- **acme-zn-t-oob-occupied**: if flag rate > ~70%, use tuning brief / bounds auto-tune (needs ≥85% flagged + analytics on Arrow sweep).
- Disable noisy rules via `enabled: false` in `setup_gl36_fdd.py` rather than deleting modules.
- Economizer and reheat-leak rules stay off until OAT/MAT/SAT columns are confirmed on the 7-day feather window.

## 120.5 JSON API weather + BACnet OAT

- OpenWeather registered on edge JSON API tab (`web-oat-t`, `web-rh`, ~20–30 min poll).
- Local OAT: bind **1100-unknown-2** (RTU outdoor air local) with `external_id: oa-t` — `scripts/acme_patch_oat_column.py`.

- Rule acme-oat-vs-web-spread flags when local vs web spread exceeds 8 °F.

## 120.6 BACnet override scans

Commission agent rotates **one device/hour** through P8 priority-array reads. Status: GET /api/bacnet/overrides/status. See Operator override scans({{ "/bacnet/override-scans/" | relative\_url }}).

## 120.7 Docker backup / recovery

Edge upgrades via scripts/upgrade\_edge\_ghcr.sh preserve workspace/data and feather volumes on the VM. After upgrade:

1. curl /health → expect matching openfdd\_version
2. ./infra/ansible/scripts/acme\_operational\_verify.sh
3. Re-push rules if rules\_store.json was reset: setup\_gl36\_fdd.py --host ... --token ...

Feather backup: snapshot workspace/data/feather/ (or site pack export) before major model changes.

# 121 Bench 5007 dual-source smoke

# 122 Bench 5007 dual-source smoke

Validates BACnet MS/TP device **5007** and Niagara baskStream station **bench9065** on the same physical bench.

## 122.1 Run

```
./scripts/run_local.sh start
./scripts/smoke_bench_5007_dual_source.sh
```

Environment (optional):



| Variable                       | Default                  |
|--------------------------------|--------------------------|
| OPENFDD_BASE_URL               | http://127.0.0.1:8765    |
| OPENFDD_AUTH_ENV               | workspace/auth.env.local |
| OPENFDD_SMOKE_SITE_ID          | demo                     |
| OPENFDD_SMOKE_POLL_SECONDS     | 60                       |
| OPENFDD_SMOKE_DURATION_MINUTES | 10                       |

The script wraps scripts/smoke\_benserver\_bench.py and adds DataFusion SQL lab preview when open-fdd[datafusion] is installed.

## 122.2 Paired semantic points

| Semantic  | BACnet column | Niagara column    |
|-----------|---------------|-------------------|
| oa-t      | oa-t          | niagara-oa-t      |
| oa-h      | oa-h          | niagara-oa-h      |
| duct-t    | duct-t        | niagara-duct-t    |
| stat_zn-t | stat_zn-t     | niagara-stat-zn-t |

Import workspace/data/bench\_dual\_source\_model.json if the model is empty.

## 123 Bench 5007 long FDD smoke

## 124 Bench 5007 long FDD smoke

Validates **data-source-agnostic FDD, PyArrow vs DataFusion SQL equivalence**, and the **fault confirmation window** on BACnet 5007 + Niagara bench9065 (site demo).

**Read-only:** polls devices; only Open-FDD evaluation thresholds change. Never prints credentials.

## 124.1 Commands

| Mode                  | Command                                                                        |
|-----------------------|--------------------------------------------------------------------------------|
| CI synthetic          | <code>python scripts/<br/>smoke_bench_5007_long_fdd.py --<br/>synthetic</code> |
| Dev dry-run (~15 min) | <code>python scripts/<br/>smoke_bench_5007_long_fdd.py --dry-<br/>run</code>   |
| Default (2 h)         | <code>./scripts/<br/>smoke_bench_5007_long_fdd.sh</code>                       |
| Overnight (12 h)      | <code>python scripts/<br/>smoke_bench_5007_long_fdd.py --<br/>overnight</code> |

## 124.2 Key environment

| Variable                                        | Default                                                    |
|-------------------------------------------------|------------------------------------------------------------|
| <code>OPENFDD_BASE_URL</code>                   | <code>http://127.0.0.1:8765</code>                         |
| <code>OPENFDD_SMOKE_DURATION_MINUTES</code>     | 120 (720 if<br><code>OPENFDD_SMOKE_OVERNIGHT=true</code> ) |
| <code>OPENFDD_SMOKE_POLL_SECONDS</code>         | 60                                                         |
| <code>OPENFDD_SMOKE_BASELINE_MINUTES</code>     | 20                                                         |
| <code>OPENFDD_SMOKE_CONFIRMATION_ROWS</code>    | 10                                                         |
| <code>OPENFDD_SMOKE_CONFIRMATION_MINUTES</code> | 10                                                         |
| <code>OPENFDD_SMOKE_PRIMARY_SEMANTIC</code>     | <code>duct-t</code>                                        |
| <code>OPENFDD_SMOKE_FORCED_THRESHOLD_F</code>   | 80.0                                                       |

## 124.3 Fault confirmation window

At 1-minute polling, `min_true_rows = 10` means the raw condition must stay true for about **10 minutes** before a **confirmed fault** is reported. See [fault-confirmation.md](#).

## 124.4 Artifacts

- `workspace/reports/bench_5007_long_fdd_<timestamp>.md`
- `workspace/reports/bench_5007_long_fdd_<timestamp>.json`
- `workspace/reports/bench_5007_long_fdd_<timestamp>_events.csv`

## 124.5 Pass / fail

| Verdict     | Meaning                                                                                                                             |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>PASS</b> | All checks passed;<br>confirmation fully vectorized<br>(not expected today)                                                         |
| <b>WARN</b> | Core FDD uses<br>pyarrow_compute /<br>datafusion_sql; confirmation<br>engine is<br>python_loop_over_arrow_mask<br>(known tech debt) |
| <b>FAIL</b> | python_list core crunching,<br>early confirmed fault, backend<br>mismatch, missing data, or<br>secrets in output                    |

Live mode uses POST /api/bench/long-fdd/evaluate (validation/  
smoke only).

## 124.6 Related

- [Dual-source smoke](#)
- [Fault confirmation](#)

# 125 ACME live validation

# 126 ACME live validation

**ACME** (site\_id: acme, Acme Building) is the **live BACnet HVAC validation site** on the OT LAN. Use it for real VAV/AHU behavior, RCx rule tuning, and overnight FDD runs.

Do **not** confuse ACME with **Bench 5007** dual-source validation (site demo, BACnet 5007 + Niagara bench9065). Bench 5007 proves backend equivalence; ACME proves production HVAC rules on field hardware.

Reference model (no secrets): workspace/data/fixtures/  
acme\_data\_model.json. Validate locally:

```
python scripts/validate_acme_model_context.py  
python scripts/validate_acme_model_context.py --json
```

## 126.1 Model export / import

1. **Export** — GET /api/model/commissioning-export (dashboard **Import / export** tab or API).
2. **Validate** — python scripts/validate\_acme\_model\_context.py on saved JSON; dashboard dry-run before import.
3. **Import** — POST /api/model/commissioning-import only after review. Preserve point id, fdd\_input, metadata.series\_id, and existing fdd\_rule\_ids. Never invent rule IDs.

Someday ACME will be updated from live export again; until the bench harness is stable, treat the committed fixture as the regression baseline.

## 126.2 Run FDD (read-only)

| Goal               | Command / API                                                                                |
|--------------------|----------------------------------------------------------------------------------------------|
| Post-upgrade smoke | ./scripts/acme_post_deploy_validate.sh --limit acme_vm_bbartling --full                      |
| Selected rules     | Rule Lab preview / POST /api/playground/test-rule                                            |
| All enabled rules  | POST /api/rules/batch → workspace/data/fdd_results.json                                      |
| Overnight cycles   | OPENFDD_LIVE_ACME=1 python3 scripts/acme_overnight_fdd_validate.py --limit acme_vm_bbartling |

Normal validation is **read-only**: no BACnet writes, no Pooge reset, no setpoint commands. BACnet writes require commission role + explicit enable — see [BACnet write safety]({{ "/security/bacnet-writes/" | relative\_url }}).

## 126.3 Reports

- Post-deploy JSON: reports/acme-live-validate.json (gitignored)
- Overnight FDD: under reports/ from overnight script
- FDD batch: workspace/data/fdd\_results.json (local edge only)

Redact tokens and hostnames before sharing.

## 126.4 Fault confirmation

Rules may fire **raw** faults before **confirmed** faults. Configure `min_true_rows` and `min_elapsed_minutes` in rule `cfg` (e.g. 10 rows at 1-minute poll  $\approx$  10 minutes). See `Fault confirmation`(`{{ "/rule-cookbook/fault-confirmation/" | relative_url }}`).

## 126.5 Safe rules on ACME

Prefer read-only sensor bounds, flatline, spread (OAT vs weather), damper stuck, and zone temperature rules. Avoid supervisory writes unless explicitly commissioned and guarded.

## 126.6 Related

- `ACME VAV/AHU rule guidance`(`{{ "/operations/acme_vav_ahu_rule_guidance/" | relative_url }}`)
- `Deployment validation`(`{{ "/ops/deployment-validation/" | relative_url }}`)
- `Bench 5007 long FDD smoke`(`{{ "/operations/bench-5007-long-fdd-smoke/" | relative_url }}`) — dual-source bench only

# 127 AI agent context

# 128 AI agent context

Canonical map for humans and MCP/RAG agents operating Open-FDD on the OT edge. **No secrets** in docs, exports, reports, or chat context.

## 128.1 1. Open-FDD in one paragraph

Open-FDD is an **edge FDD platform**: BACnet / Niagara / JSON API drivers ingest telemetry into a **BRICK-style JSON model** and **Feather/Arrow historian**; **PyArrow** (primary) and optional **DataFusion SQL** rules produce boolean fault masks with a shared **confirmation window**; the **Operator Bridge** dashboard and **MCP sidecar** expose commissioning, Rule Lab, batch FDD, and doc search. Deploy via **GHCR Docker** (`openfdd-bridge`, `openfdd-commission`, `openfdd-mcp-rag`) on a trusted LAN — not the public internet.

# 128.2 2. Deployment mental model

| Piece      | Role                                                                                   |
|------------|----------------------------------------------------------------------------------------|
| GHCR stack | openfdd-bridge (API + UI), openfdd-commission (BACnet poll), openfdd-mcp-rag (doc RAG) |
| Host Caddy | :80 / :443 front door to bridge                                                        |
| workspace/ | Model, rules, feather store, auth env — persist across image upgrades                  |
| Auth       | workspace/auth.env.local (gitignored); JWT via POST / api/auth/login                   |
| Update     | Updating the stack({{ “/ quick-start/updating/”                                        |

Rebuild MCP index after doc changes: ./scripts/build\_mcp\_rag\_index.sh.

# 128.3 3. Source / driver modes

| Mode                 | Feather source     | When                                                     |
|----------------------|--------------------|----------------------------------------------------------|
| BACnet direct        | bacnet             | Production OT — commission container polls :47808        |
| Niagara baskStream   | niagara_baskstream | Bench / sites with Niagara HTTP API                      |
| JSON API             | json_api           | Weather, REST sensors, OpenWeather demo                  |
| Platform API profile | scaffold           | Future portfolio central — not production duplicate poll |

Do **not** run legacy BACnet poll alongside Docker **commission** on the same device (double poll). Validation smokes may intentionally dual-source **Bench 5007** only.

## 128.4 4. Data model contract

Commissioning bundle: GET /api/model/commissioning-export → POST /api/model/commissioning-import (dry-run in UI first).

| Field                                       | Rule                                                                        |
|---------------------------------------------|-----------------------------------------------------------------------------|
| sites[], equipment[], points[], fdd_rules[] | Required structure                                                          |
| Point id                                    | <b>Preserve</b> on import — never renumber                                  |
| fdd_input                                   | Rule column role (oa-t, zn-t, sat, ...)                                     |
| metadata.series_id                          | Historian series key                                                        |
| metadata.external_ref                       | Stable feather/BACnet reference                                             |
| fdd_rule_ids                                | Must reference existing fdd_rules[].id — <b>never invent rule</b> on import |
| Column map                                  | open_fdd.arrow_runtime.build_column_map_from_model_points(site_id)          |

Live HVAC reference model: **ACME** (site\_id: acme) — fixture at workspace/data/fixtures/acme\_data\_model.json. Bench dual-source model: site demo / device 5007 — not ACME.

## 128.5 5. Rule execution contract

| Backend                                         | Use when                                                                                |
|-------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>PyArrow</b> (apply_faults_arrow)             | Full HVAC logic, rolling windows, ML prep, cookbook helpers                             |
| <b>DataFusion SQL</b> (backend: datafusion_sql) | Simple threshold / CASE / SQL-readable rules; optional pip install open-fdd[datafusion] |

Both normalize to **Arrow boolean mask** / ArrowRuleResult. SQL runs **server-side only** against registered telemetry; unsafe SQL is rejected. No browser-side SQL. No pandas row-loop runtime on edge.

## 128.6 6. Fault confirmation window

- **Raw fault** — rule mask true on one row.

- **Confirmed fault** — `min_true_rows` consecutive trues and/or `min_elapsed_minutes` elapsed.
- Example: **10 rows** at **1-minute** polling  $\approx$  **10 minutes** confirmed.

See `Fault confirmation({{ "/rule-cookbook/fault-confirmation/" | relative_url }})`.

## 128.7 7. Validation modes

| Mode                 | Purpose                                                                                                                             |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>Bench 5007</b>    | Dual-source BACnet vs Niagara backend equivalence — [long FDD smoke]({{ "/operations/bench-5007-long-fdd-smoke/"   relative_url }}) |
| <b>ACME live</b>     | Real BACnet HVAC, VAV/AHU rules, RCx — <code>ACME live validation({{ "/operations/acme-live-validation/"   relative_url }})</code>  |
| <b>Synthetic CI</b>  | <code>python scripts/smoke_bench_5007_long_fdd.py --synthetic --dry-run</code>                                                      |
| <b>Model fixture</b> | <code>python scripts/validate_acme_model_context.py</code>                                                                          |

## 128.8 8. Security rules

- **No secrets** in docs, logs, exports, MCP bundle, or validation reports.
- Bridge binds **127.0.0.1** by default; LAN exposure requires explicit operator choice.
- **BACnet writes** disabled/guarded — see [BACnet write safety]({{ "/security/bacnet-writes/" | relative\_url }}).
- Niagara / JSON API service accounts: dedicated, read-only where possible.
- Full tracebacks only when `OFDD_DEBUG_TRACEBACKS=1`.

## 128.9 9. Where to look

| Topic   | Doc                                                             |
|---------|-----------------------------------------------------------------|
| Start   | <code>Quick Start({{ "/quick-start/"   relative_url }})</code>  |
| Drivers | <code>Driver framework({{ "/drivers/"   relative_url }})</code> |
| Rules   |                                                                 |



| Topic              | Doc                                                                           |
|--------------------|-------------------------------------------------------------------------------|
|                    | <a href="#">Rule Cookbook</a> ({{ "/rule-cookbook/"                           |
| API                | <a href="#">API routes</a> ({{ "/"                                            |
| Operations         | <a href="#">appendix/bridge_api/"</a>                                         |
|                    | <a href="#">Operations</a> ({{ "/ops/"                                        |
| Architecture       | [ADR: Rust-ready Arrow contract]({{ "/adr/adr-rust-ready-arrow-fdd-contract/" |
| MCP                | <a href="#">MCP server</a> ({{ "/ai/mcp-server/"                              |
| Contributor policy | AGENTS.md (repo root)                                                         |

## 129 LAN hardening

## 130 LAN hardening

Open-FDD targets **trusted building LAN / OT edge** deployment. The Operator Bridge is not intended for direct public internet exposure.

Full mode matrix: [Deployment modes](#)({{ "/architecture/deployment-modes/" | relative\_url }}).

### 130.1 Auth and bind

| Mode               | Bridge bind                    | Auth                                                      |
|--------------------|--------------------------------|-----------------------------------------------------------|
| Local dev          | 127.0.0.1                      | Optional<br>OFDD_AUTH_DISABLED=1<br>on loopback           |
| Lab LAN            | 0.0.0.0                        | Required unless<br>explicit insecure<br>lab flags         |
| Edge<br>production | Loopback + <b>Caddy</b> on LAN | <b>Required</b> —<br>OFDD_AUTH_SECRET +<br>role passwords |

Startup **fails** on non-loopback bind without credentials (unless insecure lab flags are set).

## 130.2 Network exposure

- Put **Caddy** on :80 / :443; keep bridge :8765 on loopback when possible.
- Do not port-forward the Operator Bridge to the public internet without TLS and strong auth.
- Keep BACnet on OT VLANs; management UI on IT VLAN with firewall rules.
- BACnet **writes** are gated by default — [BACnet write guard] (`{{ "/security/bacnet-writes/" | relative_url }}`).

## 130.3 Rule Lab

Rule Lab runs **operator-authored Python** in a sandbox. Restrict integrator accounts; review rules before enable on production sites.

## 130.4 TLS and LAN security scans

| Topic                                | Page                                                                                     |
|--------------------------------------|------------------------------------------------------------------------------------------|
| Certificates (self-signed / site CA) | <a href="#">TLS and certificates</a> ( <code>{{ "/security/tls-and-certs/"</code>        |
| ZAP + Nmap bench smoke               | <a href="#">[ZAP baseline]</a> ( <code>{{ "/security/zap-baseline/"</code>               |
| Ubuntu host + Tenable/Nessus         | <a href="#">Linux host hardening</a> ( <code>{{ "/security/linux-host-hardening/"</code> |
| Release test cycle (maintainers)     | <a href="#">[Security testing]</a> ( <code>{{ "/developer/security-testing/"</code>      |

## 130.5 Ollama on the edge

Ollama has **no built-in auth**. Bind `OLLAMA_HOST=127.0.0.1:11434` on the host; never expose :11434 on the OT LAN. Open-FDD bridge reaches Ollama server-side only. Validation: `scripts/security/check_openfdd_exposure.sh`.

# 131 ZAP baseline (local bench)

# 132 ZAP baseline (local bench)

Passive OWASP ZAP baseline against the **pentest production stack** on benser server is the standard LAN security smoke before Acme edge deploys.

Full per-revision workflow (pytest → PR → GHCR → Acme):  
[Developer — security testing cycle]({{ "/developer/security-testing/" | relative\_url }}).

Packaged scan scripts (Windows + Mac/Linux): scripts/security/README.md.

## 132.1 Run

**On the Open-FDD host:**

```
./scripts/pentest_production_stack.sh start  
./scripts/pentest_production_stack.sh verify
```

**From a LAN workstation** (after verify passes):

```
# Windows – optional integrator auth probe (copy auth.env.local to  
scan PC; never commit)  
.\scripts\security\Run-OpenFddSecurityScan.ps1 -TargetUrl "http://  
192.168.204.18" ` -AuthEnvFile "C:\path\to\auth.env.local"
```

The script writes 40-release-gate-summary.txt with **SECURITY GATE: PASS/FAIL:** - Fails if production JS bundles contain private LAN IPs or bench Niagara defaults - Warns if auth env is missing (Auth loaded: False) — pass -AuthEnvFile or shell vars OFDD\_INTEGRATOR\_USER / OFDD\_INTEGRATOR\_PASSWORD (values are never logged)

```
# macOS / Linux  
./scripts/security/run_openfdd_security_scan.sh --url http://  
192.168.204.18
```

ZAP target (example): `http://192.168.204.18/` — Caddy on `:80` only; bridge `:8765` stays loopback. TLS bench: see [TLS and certificates](#)(`{{ "/security/tls-and-certs/" | relative_url }}`)).

## 132.2 Expected results (HTTP bench mode)

| Finding                                     | Severity | Status                                                                             |
|---------------------------------------------|----------|------------------------------------------------------------------------------------|
| CSP style-src 'unsafe-inline'               | Medium   | <b>Accepted</b> — Vite/React inline styles; TODO to nonce/hash                     |
| Missing COEP                                | Low      | <b>Accepted</b> — require-corp not enabled (breaks some assets)                    |
| Port 80 open (Caddy)                        | Info     | <b>Expected</b> — intentional LAN entry                                            |
| Ports 8765, 5173, 8000, 8080 closed         | —        | <b>Good</b>                                                                        |
| Port 443 filtered                           | —        | <b>Expected</b> in HTTP mode; use <code>OFDD_CADDY_MODE=tls</code> for HTTPS bench |
| Duplicate Referrer-Policy / X-Frame-Options | Low      | <b>Fixed</b> — bridge owns headers; Caddy does not duplicate                       |

## 132.3 Header ownership

| Layer                               | Sets                                                                                                                        |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>Bridge</b> (security_headers.py) | CSP, X-Frame-Options: DENY, frame-ancestors 'none', COOP, CORP, Permissions-Policy, Referrer-Policy, X-Content-Type-Options |
| <b>Caddy HTTP</b>                   | Reverse proxy only; strips Server banners                                                                                   |
| <b>Caddy TLS</b>                    | Adds Strict-Transport-Security only                                                                                         |

# 132.4 Authenticated scan (later)

Baseline scan hits ~4 public endpoints (/health, public agent insight routes). Deep API/dashboard coverage requires ZAP with integrator login — see [Authenticated scanning \(roadmap\)](#)({{ "/security/authenticated-scanning/" | relative\_url }}).

# 132.5 After fixes

Re-run `pentest_production_stack.sh verify`, then the packaged LAN scan from `scripts/security/`. Confirm response headers show **one** `X-Frame-Options: DENY` and no duplicate `Referrer-Policy`.

# 133 TLS and certificates

# 134 TLS and certificates

Who generates, stores, and renews TLS material for Open-FDD on OT LANs — and how that ties into ZAP/Nmap bench testing.

Developer release cycle: [Security testing cycle](#)({{ "/developer/security-testing/" | relative\_url }}).

## 134.1 Ownership

| Environment                         | Who owns certs         | Storage                                                           |
|-------------------------------------|------------------------|-------------------------------------------------------------------|
| Local / benserver bench             | Developer / maintainer | workspace/<br>deploy/caddy/<br>certs/<br>(cert.pem,<br>key.pem)   |
| Ansible edge (Acme, customer sites) | Site integrator        | /etc/openfdd/<br>caddy/ on the VM<br>(playbook-managed or manual) |
| Public internet                     | <b>Not in scope</b>    | Open-FDD targets OT LAN; use site                                 |

| Environment | Who owns certs | Storage            |
|-------------|----------------|--------------------|
|             |                | PKI or self-signed |

Open-FDD does **not** ship Let's Encrypt or public CA automation. OT benches use **self-signed** or customer-provided certs.

## 134.2 Generate bench certs (local)

```
./scripts/setup_caddy_certs.sh
# or: ./scripts/setup_caddy_certs.sh --cn openfdd.local --out
workspace/deploy/caddy/certs
```

Then start with TLS mode:

```
OFDD_CADDY_MODE=tls ./scripts/run_local.sh
# or pentest stack with OFDD_CADDY_MODE=tls
```

Caddy serves :443 and redirects :80 → HTTPS. Caddy adds **Strict-Transport-Security** only; all other security headers remain on the **bridge** ([ZAP baseline]({{ "/security/zap-baseline/" | relative\_url }}#header-ownership)).

## 134.3 ZAP / Nmap with TLS bench

When `OFDD_CADDY_MODE=tls`:

- Nmap: expect **443 open**, 80 may redirect
- ZAP: use `https://<lan-ip>/` (add `-k trust` or `import cert.pem` on the scan workstation)
- PowerShell: `.\Run-OpenFddSecurityScan.ps1 -TargetUrl "https://192.168.204.18"`
- Bash: `./run_openfdd_security_scan.sh --url https://192.168.204.18`

Self-signed certs produce ZAP TLS warnings — expected on OT LAN.

## 134.4 Renewal

Self-signed certs from `setup_caddy_certs.sh` default to **365 days**. Re-run the script before expiry or bake renewal into site runbooks.

Edge Ansible deploys should document whether the integrator rotates certs manually or via site PKI.

## 134.5 Enterprise CA on Caddy (edge)

When IT requires **clean Nessus SSL plugins** (no self-signed / untrusted chain), install a site-provided certificate on the edge VM instead of `setup_caddy_certs.sh` output.

Typical layout (Ansible / manual):

```
/etc/openfdd/caddy/cert.pem # full chain (leaf + intermediates)
/etc/openfdd/caddy/key.pem  # private key (0600, root or caddy
user)
```

Ansible TLS mode (`caddy_mode: tls`) already points Caddy at those paths — see `infra/ansible/templates/Caddyfile.j2`.

### Internal enterprise CA workflow:

1. Generate CSR on the edge host or request cert from site PKI for the operator hostname (e.g. `openfdd.acme.local`).
2. Install `cert.pem` + `key.pem` under `/etc/openfdd/caddy/`.
3. Distribute the **CA root** to admin laptops (trust store) so browsers and ZAP scans validate the chain.
4. Monitor expiry:

```
openssl x509 -in /etc/openfdd/caddy/cert.pem -noout -dates
```

### Caddyfile snippet (TLS, enterprise cert):

```
:443 {
    tls /etc/openfdd/caddy/cert.pem /etc/openfdd/caddy/key.pem
    header {
        Strict-Transport-Security "max-age=31536000;
includeSubDomains"
    }
    reverse_proxy 127.0.0.1:8765
}
```

Lab/self-signed mode still produces expected **Medium** Tenable findings — document as accepted risk or switch to enterprise CA. See [Tenable remediation]({{ "/security/tenable-remediation/" | relative\_url }}).

## 134.6 Caddy internal CA mode (lab)

`./scripts/setup_caddy_certs.sh` generates a **self-signed** pair for bench TLS. Trust `cert.pem` on scan workstations (`-k curl`, ZAP import) or accept browser warnings. Not suitable when IT mandates trusted chains.

## 134.7 Secrets

- **Never commit** `key.pem` or production cert bundles
- `workspace/deploy/caddy/certs/` is for local bench only unless your `.gitignore` already excludes generated keys (verify before push)

## 135 Authenticated scanning (roadmap)

## 136 Authenticated scanning (roadmap)

[ZAP baseline]({{ "/security/zap-baseline/" | relative\_url }}) is **unauthenticated** — it checks public pages, response headers, and a handful of routes reachable without login. That is the right default for every docker image / patch revision smoke on a test LAN.

For **deep** coverage of integrator dashboards, `/api/*` routes, WebSocket sessions, and role-gated writes, plan a separate **authenticated** scan — only on fake/bench stacks with explicit approval.

Release context: `Security testing cycle({{ "/developer/security-testing/" | relative_url }})`.

### 136.1 Current state (3.0.x)

| Capability                                                 | Status                                  |
|------------------------------------------------------------|-----------------------------------------|
| ZAP baseline via <code>scripts/security/</code>            | <b>Shipped</b>                          |
| Pentest creds in <code>workspace/auth.pentest.local</code> | <b>Shipped</b> (host-local, not in git) |
| ZAP full / active scan automation                          | <b>Not shipped</b> — manual only        |
| ZAP authenticated context (login script + session)         | <b>Roadmap</b>                          |
| CI running ZAP against PR previews                         | <b>Not planned</b> (LAN bench only)     |



## 136.2 Why baseline is not enough

After login, the bridge exposes model editing, BACnet/JSON API configuration, Rule Lab, commissioning export, and write-gated commands. Baseline ZAP never exercises:

- Session cookies / JWT persistence
- CSRF or auth bypass on mutating endpoints
- IDOR on site-scoped resources
- Rate limits and audit logging on failed auth

## 136.3 Planned approach (contributors welcome)

1. **Bench-only target** — same pentest stack; never point active scans at live OT controllers
2. **Login hook** — ZAP zap-full-scan.py or Automation Framework with:
  - POST /api/auth/login using integrator creds from env (not committed)
  - Regex or JSON extractor for session token / cookie
3. **Scope file** — include /api/model/\*, /api/commissioning/\*, dashboard static assets; exclude BACnet UDP and unrelated LAN hosts
4. **Report parity** — extend scripts/security/ with opt-in - Authenticated / --zap-full flags and separate report prefix (22-zap-authenticated-\*)
5. **Gate** — document accepted findings; fail release only on High/Critical regressions

## 136.4 Manual authenticated ZAP today

1. Start pentest stack; note creds in workspace/auth.pentest.local
2. Open ZAP desktop or docker with API port exposed
3. Configure **Context** → **Authentication** → script or form-based login to /api/auth/login
4. Run **Spider** then **Active Scan** on http://<lan-ip>/ only
5. Compare with baseline report in openfdd-security-report/

Do **not** run active scans against production Acme without a maintenance window and BACnet write lock.

## 136.5 Related

- [scripts/security/README.md](#) — baseline setup
- [Secrets and auth\({{ "/security/secrets-auth/" | relative\\_url }}\)](#) — env file layout

- [LAN hardening](#)({{ "/security/lan-hardening/" | relative\_url }})  
— bind and auth modes

# 137 Secrets and auth

# 138 Secrets and auth

## 138.1 Files (gitignored)

| File                                       | Contents                                                                                                            |
|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| workspace/<br>auth.env.local               | JWT secret, role passwords                                                                                          |
| workspace/<br>json_api.env.local           | REST API keys for JSON API <code>\${ENV:VAR}</code><br>URLs (e.g. OpenWeather<br><code>OPENWEATHER_API_KEY</code> ) |
| workspace/<br>data.env.local               | Historian + <b>audit log retention</b><br>( <code>OFDD_LOG_*</code> , <code>OFDD_FEATHER_*</code> )                 |
| workspace/<br>ollama.env.local             | Ollama URL, model, RAM tier, GPU<br>mode                                                                            |
| workspace/<br>mcp.env.local                | MCP RAG sidecar                                                                                                     |
| workspace/<br>caddy.env.local              | Reverse proxy TLS/HTTP mode                                                                                         |
| workspace/<br>pentest.env.local            | LAN security scan stack                                                                                             |
| infra/ansible/secrets/<br><host>.env.local | SSH deploy secrets (maintainers)                                                                                    |

Copy from \*.example templates only. Full inventory: [Workspace env files](#)({{ "/appendix/workspace-env-files/" | relative\_url }}).

## 138.2 Required variables (production)

| Variable                        | Purpose                  |
|---------------------------------|--------------------------|
| OFDD_AUTH_SECRET                | HMAC signing (32+ chars) |
| OFDD_INTEGRATOR_USER / PASSWORD | Rule Lab, bindings       |
| OFDD_OPERATOR_USER / PASSWORD   | Operations               |

## 138.3 GHCR pulls

Use read-only registry credentials on edge if images are private; rotate tokens periodically.

## 138.4 Backup

Back up workspace/data/ and auth env in secure operator vault — not in git.

Audit JSONL under workspace/logs/ may contain client IPs and usernames — treat like security logs. See `Logging and audit({{ "/ops/logging/" | relative_url }})`.

# 139 BACnet write allowlists

# 140 BACnet write allowlists

Supervisory writes require:

1. **Feature flag** enabled in environment (off by default).
2. **Allowlist** file at workspace/bacnet/write\_allowlist.json (copy from `write_allowlist.example.json`). If the flag is on but the allowlist is missing, writes are **denied**.
3. **Integrator** role on API calls.
4. **Audit** log review after commissioning sessions.

Allowlist entries support `device_instance`, `object_identifier`, `property_identifier`, optional `priority_min/priority_max`, `value_min/value_max`, and `allowed_values` for binary/multistate targets.

Lab-only override: `OFDD_BACNET_WRITE_ALLOW_ANY=1` (audited; not for production edges).

See operator guide: `[BACnet write safety]({{ "/bacnet/write-safety/" | relative_url }})`.

Never enable blanket write access for agent or integrator automation without human approval per site.

# 141 Linux host hardening

## 142 Linux host hardening

Generic **Ubuntu edge VM** guidance for Open-FDD deployments scanned by IT tools (Tenable/Nessus, Qualys, etc.). This complements application-level [LAN hardening](#)(`{{ "/security/lan-hardening/" | relative_url }}`) and [\[TLS\]](#)(`{{ "/security/tls-and-certs/" | relative_url }}`).

Nessus-specific finding tables: [\[Tenable remediation\]](#)(`{{ "/security/tenable-remediation/" | relative_url }}`).

### 142.1 Scope

| Layer                                | Owner                       | Repo support                                                                             |
|--------------------------------------|-----------------------------|------------------------------------------------------------------------------------------|
| Ubuntu kernel, OpenSSH, apt packages | <b>Site IT / integrator</b> | <code>scripts/security/remediate_ubuntu_host.sh</code>                                   |
| Docker engine, compose, Caddy        | <b>Integrator</b>           | Ansible + <code>check_openfdd_exposure.sh</code>                                         |
| Open-FDD containers (pip/pyOpenSSL)  | <b>Open-FDD maintainers</b> | <code>docker/python-security-constraints.txt</code> , GHCR images                        |
| Self-signed TLS on OT LAN            | <b>Expected lab finding</b> | Enterprise CA path in <a href="#">[TLS]</a> ( <code>{{ "/security/tls-and-certs/"</code> |

### 142.2 Quick operator workflow

On the edge VM (SSH):

```
cd ~/open-fdd
./scripts/security/check_host_security.sh
./scripts/security/check_openfdd_exposure.sh --edge-ip "$
(hostname -I | awk '{print $1}')
```

Remediate host OS (maintenance window):

```
./scripts/security/remediate_ubuntu_host.sh # dry-run
sudo ./scripts/security/remediate_ubuntu_host.sh --apply
# if /var/run/reboot-required exists:
sudo reboot
./scripts/security/check_host_security.sh
```

Pull new Open-FDD images after a release that pins container pip/pyOpenSSL:

```
OPENFDD_IMAGE_TAG=latest ./scripts/upgrade_edge_ghcr.sh --limit
<inventory_host>
```

## 142.3 Ubuntu version and EOL

Supported targets: **Ubuntu 22.04 LTS** or **24.04 LTS** (64-bit).  
Rebuild hosts still on **16.04** or **18.04** — apt upgrade cannot clear an SEoL finding.

Validate a surprising 16.04 Nessus hit:

```
lsb_release -a
cat /etc/os-release
# Scan credential / stale DNS / container banner mismatch?
```

## 142.4 Kernel and package updates

```
sudo apt update
sudo apt full-upgrade -y
uname -r
test -f /var/run/reboot-required && echo "reboot required"
sudo reboot # maintenance window
```

Optional status tools:

```
ubuntu-security-status # or: pro status
apt list --upgradable | grep -i security
```

## 142.5 Host Python (pip, wheel, pyOpenSSL)

Tenable often reports **host** pip, wheel, and pyOpenSSL versions installed by Ubuntu packages or admin pip install. Prefer **apt** on the host:

```
python3 -m pip show pip setuptools wheel pyOpenSSL
sudo apt install --only-upgrade python3-pip python3-openssl
python3-wheel
```

Open-FDD **containers** pin toolchain versions at image build — see `docker/python-security-constraints.txt`. Rescan after `docker compose pull`.

## 142.6 Ollama — no LAN exposure

Ollama has **no built-in authentication** in typical deployments. Treat LAN exposure as **Critical**.

| Rule                            | Implementation                                                                            |
|---------------------------------|-------------------------------------------------------------------------------------------|
| Bind loopback only              | OLLAMA_HOST=127.0.0.1:11434 in systemd (ollama.service) or dev bootstrap                  |
| Bridge calls server-side        | OFDD_OLLAMA_BASE_URL=http://127.0.0.1:11434 or host.docker.internal:11434 from containers |
| Never publish :11434 on 0.0.0.0 | Compose uses 127.0.0.1:11434:11434 when Docker Ollama is enabled                          |

Validation:

```
ss -tulpn | grep 11434
curl -sf http://127.0.0.1:11434/api/tags | head
curl -sf --max-time 3 http://<edge-lan-ip>:11434/api/tags
# must FAIL from another host
```

Firewall (if management host needs GPU Ollama — rare):

```
# Example: block LAN → 11434; allow loopback only (adjust
interface names)
sudo ufw deny 11434/tcp
sudo ufw allow from 127.0.0.0/8 to any port 11434 proto tcp
```

## 142.7 OpenSSH Terrapin (CVE-2023-48795)

Apply Ubuntu security updates first (openssh-server, openssh-client). Verify:

```
ssh -V
sudo sshd -T | grep -Ei '^(ciphers|macs|kexalgorithms)'
```

Manual algorithm hardening is **optional** and can lock out clients — document rollback (`sshd_config.bak`) before changes. Do not automate cipher stripping in repo scripts.

## 142.8 ICMP timestamp (Low)

Nessus may report ICMP timestamp disclosure. Optional hardening at host firewall or upstream network — low priority for isolated OT VLANs.

Example (nftables — site-specific):

```
# Block ICMP timestamp / type 13 – verify with IT before applying
sudo nft add rule inet filter input icmp type timestamp-request
drop
```

## 142.9 TLS and certificate scans

Self-signed or internal CA certificates produce **Medium** Nessus SSL findings (untrusted, self-signed, expiry). That is **expected** for lab/OT encryption-in-transit. For clean SSL plugin results, install a **site enterprise CA** certificate on Caddy — [TLS and certificates](#)({ { “/security/tls-and-certs/” | relative\_url } } #enterprise-ca-production).

## 142.10 Maintainer security audits (CI parity)

```
./scripts/security/run_maintainer_audits.sh
```

Matches GitHub Actions operator-bridge-security job where possible.

# 143 Tenable / Nessus remediation

# 144 Tenable / Nessus remediation

Remediation plan for **IT/Tenable.io Nessus** findings on an Open-FDD **OT/LAN edge** deployment (e.g. Acme GL36 lab VM). Open-FDD runs behind a building firewall on Ubuntu with Docker Compose, Caddy, and optional host Ollama.

Related: [Linux host hardening](#)({{ "/security/linux-host-hardening/" | relative\_url }}), [TLS and certificates](#)({{ "/security/tls-and-certs/" | relative\_url }}), [LAN hardening](#)({{ "/security/lan-hardening/" | relative\_url }}), [Security testing cycle](#)({{ "/developer/security-testing/" | relative\_url }}).

### 144.1 Prioritized action plan

| Priority | Action                                                                | Owner           |
|----------|-----------------------------------------------------------------------|-----------------|
| P0       | Confirm Ubuntu is <b>not</b> 16.04/18.04; rebuild if EOL              | IT / integrator |
| P0       | Bind Ollama to <b>127.0.0.1</b> only; verify LAN cannot reach :11434  | Integrator      |
| P0       | apt full-upgrade + <b>reboot</b> for kernel USNs (8254, 8278, 8373)   | IT / integrator |
| P1       | Upgrade host pip/wheel/pyOpenSSL via apt or documented pip path       | IT / integrator |
| P1       | Pull GHCR images built with docker/python-security-constraints.txt    | Integrator      |
| P2       | OpenSSH packages current; review Terrapin after patch                 | IT              |
| P2       | TLS: accept self-signed findings <b>or</b> install enterprise CA cert | IT decision     |
| P3       | ICMP timestamp: optional firewall rule                                | Network / IT    |

### 144.2 Finding classification table

| Nessus finding | Severity | Class | Remediation                                       |
|----------------|----------|-------|---------------------------------------------------|
|                | Critical |       | lsb_release -a — rebuild VM on 24.04 LTS if truly |



| Nessus finding                            | Severity | Class                                   | Remediation                                                                               |
|-------------------------------------------|----------|-----------------------------------------|-------------------------------------------------------------------------------------------|
| Canonical Ubuntu Linux SEoL (16.04.x)     |          | Host OS / <b>false positive?</b>        | 16.04; if host is 22.04/24.04, check scan target IP, credentials, stale asset record      |
| Ollama Unauthenticated Access             | Critical | Deployment config                       | OLLAMA_HOST=127.0.0.1:11434; no 0.0.0.0:11434; firewall deny; see <a href="#">Ollama</a>  |
| Ubuntu kernel USN-8254-1                  | Critical | Host OS                                 | sudo apt update && sudo apt full-upgrade -y && sudo reboot                                |
| pyOpenSSL 22.0.x < 26.0.0 Buffer Overflow | Critical | Host pip <b>and</b> container           | Host: apt upgrade python3-openssl or pip ≥26; Container: GHCR image with constraints file |
| Ubuntu pip USN-8344-1                     | High     | Host OS + container build               | apt full-upgrade; image build upgrades pip≥26                                             |
| SSL Certificate Cannot Be Trusted         | Medium   | <b>Accepted risk</b> (lab) / deployment | Self-signed Caddy — use enterprise CA for clean scan                                      |
| SSL Self-Signed Certificate               | Medium   | <b>Accepted risk</b> (lab)              | Same as above                                                                             |
| SSL Certificate Expiry                    | Medium   | Deployment / monitoring                 | openssl x509 -dates; renew setup_caddy_certs.sh or site PKI                               |
| SSH Terrapin (CVE-2023-48795)             | Medium   | Host OS                                 | Upgrade openssh-server; validate ssh -V                                                   |
| Ubuntu wheel USN-8221-1                   | Medium   | Host OS                                 | apt full-upgrade                                                                          |
| Ubuntu kernel USN-8278-1, USN-8373-1      | Medium   | Host OS                                 | apt full-upgrade + reboot                                                                 |
| pyOpenSSL 0.14.x Security Bypass          | Medium   | Host + container                        | Same as pyOpenSSL Critical — upgrade to ≥26                                               |
| ICMP Timestamp Request                    | Low      | Network / optional host                 | Firewall drop; often accepted on isolated OT VLAN                                         |

**Class legend:** (1) repo-owned fix · (2) Docker base/image · (3) host OS · (4) deployment/config · (5) accepted risk · (6) validate false positive

## 144.3 Operator commands (edge VM)

### 144.3.1 Validate (read-only)

```
cd ~/open-fdd
./scripts/security/check_host_security.sh
./scripts/security/check_openfdd_exposure.sh --edge-ip "$
(hostname -I | awk '{print $1}')"
lsb_release -a
uname -r
test -f /var/run/reboot-required && cat /var/run/reboot-
required.pkgs
python3 -m pip show pip setuptools wheel pyOpenSSL 2>/dev/null |
grep -E '^(Name|Version):'
ss -tulpn | grep -E ':80|:443|:8765|:11434'
curl -sf http://127.0.0.1:11434/api/tags | head -3
```

From a **different LAN workstation** (must fail):

```
curl -sf --max-time 3 http://<edge-ip>:11434/api/tags
```

### 144.3.2 Remediate host (maintenance window)

```
./scripts/security/remediate_ubuntu_host.sh # dry-run
./scripts/security/remediate_ubuntu_host.sh --apply
sudo reboot # if reboot-required
./scripts/security/check_host_security.sh
```

### 144.3.3 Remediate Open-FDD containers

```
OPENFDD_IMAGE_TAG=latest ./scripts/upgrade_edge_ghcr.sh --limit
<inventory_host>
docker compose pull && docker compose up -d
```

### 144.3.4 Rescan

After reboot and image pull, request IT **Nessus rescan** of the edge IP. Compare plugin output to the table above.

## 144.4 Ollama unauthenticated access (Critical)

**Not an Open-FDD code bug** — Ollama's API is unauthenticated by design. Risk is **network exposure**.

Repo defaults (3.0.5+):

- Systemd: Environment=OLLAMA\_HOST=127.0.0.1:11434 in infra/ansible/templates/ollama.service.j2
- Compose Ollama (optional profile): 127.0.0.1:11434:11434 only
- Acme-style edge: host Ollama + bridge via host.docker.internal:11434 (container → host loopback path)
- post\_deploy\_check.sh warns if :11434 listens on 0.0.0.0

Re-apply Ansible Ollama unit after upgrade:

```
ansible-playbook -i inventory.yml ollama_bootstrap.yml --limit  
<host> -e enable_ollama=true
```

## 144.5 pyOpenSSL and pip in containers

Docker build installs docker/python-security-constraints.txt before application requirements:

- pip>=26.0, setuptools>=75.0, wheel>=0.45
- pyOpenSSL>=26.0.0, cryptography>=44.0.0

CI pip-audit runs against bridge + bacnet requirements with the same constraints.

## 144.6 TLS — expected vs production

| Mode                              | Nessus SSL plugins                         | When to use                 |
|-----------------------------------|--------------------------------------------|-----------------------------|
| Caddy<br>HTTP :80                 | No TLS plugins                             | Dev / HTTP-only<br>VLAN     |
| Caddy<br>self-<br>signed :<br>443 | Medium: untrusted, self-<br>signed, expiry | OT LAN encryption,<br>lab   |
| Enterprise<br>CA cert<br>on Caddy | Clean SSL plugins (if chain<br>trusted)    | IT mandates clean<br>Nessus |

See [TLS — enterprise CA]({{ "/security/tls-and-certs/" |  
relative\_url }}#enterprise-ca-on-caddy-edge).

# 144.7 Residual findings after remediation

Even with a patched host and loopback-only Ollama, IT may still see:

- **SSL** Medium on self-signed TLS (accepted for lab; document in risk register)
- **ICMP** Low (optional network block)
- **Container** plugins if Nessus scans Docker overlay networks — clarify scan scope with IT

# 144.8 Notes for IT / security team

1. Scan the **edge VM IP**, not individual container IPs, unless container CVE correlation is required.
2. Open-FDD **bridge :8765** should be **loopback-only**; LAN entry is **Caddy :80/:443**.
3. **BACnet UDP 47808** on commission is expected OT traffic — out of scope for web/plugin remediation.
4. Do not expose Ollama **11434** to the building LAN; agent chat is server-side only.
5. Kernel CVEs require **host reboot** — schedule with operations.
6. For authenticated web/API depth, see roadmap [Authenticated scanning]({{ "/security/authenticated-scanning/" | relative\_url }}).

# 144.9 Repo-owned fixes (this document's release)

| Item                         | Location                                                  |
|------------------------------|-----------------------------------------------------------|
| Container pip/pyOpenSSL pins | docker/python-security-constraints.txt, docker/Dockerfile |
| Ollama loopback systemd      | infra/ansible/templates/ollama.service.j2                 |
| Host/exposure check scripts  | scripts/security/check_*.sh, remediate_ubuntu_host.sh     |
| Post-deploy Ollama bind warn | infra/ansible/scripts/post_deploy_check.sh                |
| CI pip-audit + constraints   | .github/workflows/ci.yml                                  |

# 145 AI agents

# 146 AI agents and MCP

- [AI agent context](#)({{ "/ai-agent-context/" | relative\_url }}) — canonical map for deploy, drivers, model, rules, validation (**start here**)
- [MCP server](#)({{ "/ai/mcp-server/" | relative\_url }}) — FastMCP tools over the operator bridge
- Skills under [skills/openfdd-\\*-agent/](#) and [skills/openfdd-mcp-server/](#)
- Operator memory: [workspace/MEMORY.md](#) (see [workspace/MEMORY.md.example](#))

Internal runbooks (not primary nav): [\[agent-skills/\]\({{ "/agent-skills/bench-validation-agent/" | relative\\_url }}\)](#).

# 147 Security

# 148 Security

LAN deployment security for the Operator Bridge and BACnet write guard. Open-FDD is for **trusted edge / OT LAN** — not direct public internet exposure.

| Page                                                                                                        | Topic           |
|-------------------------------------------------------------------------------------------------------------|-----------------|
| <a href="#">LAN hardening</a> ({{ "/security/lan-hardening/"   relative_url }})                             | relative_url }} |
| <a href="#">ZAP baseline (local bench)</a><br>({{ "/security/zap-baseline/"   relative_url }})              | relative_url }} |
| <a href="#">TLS and certificates</a> ({{ "/security/tls-and-certs/"   relative_url }})                      | relative_url }} |
| <a href="#">Linux host hardening</a> ({{ "/security/linux-host-hardening/"   relative_url }})               | relative_url }} |
| <a href="#">Tenable / Nessus remediation</a> ({{ "/security/tenable-remediation/"   relative_url }})        | relative_url }} |
| <a href="#">Authenticated scanning (roadmap)</a> ({{ "/security/authenticated-scanning/"   relative_url }}) | relative_url }} |

| Page                                                                     | Topic           |
|--------------------------------------------------------------------------|-----------------|
| <a href="#">Secrets and auth</a> ({{ "/" security/secrets-auth/"         | relative_url }} |
| <a href="#">BACnet write allowlists</a> ({{ "/" security/bacnet-writes/" | relative_url }} |
| [Release & LAN scan cycle]<br>({{ "/" developer/security-testing/"       | relative_url }} |
| <a href="#">Logging and audit</a> ({{ "/" ops/logging/"                  | relative_url }} |

## 149 API routes

## 150 API routes

REST API served by **openfdd-bridge** (default <http://127.0.0.1:8765>).  
Production: Caddy on :80 proxies to the bridge.

**OpenAPI:** GET /docs and GET /redoc when the bridge runs.

**Auth:** JWT via POST /api/auth/login. Most routes require Authorization: Bearer <token>. Roles: viewer, operator, integrator, commission, agent, admin. BACnet writes require commission role + [write safety](#)({{ "/" bacnet/write-safety/" | relative\_url })). Dev: OFDD\_AUTH\_DISABLED on loopback only.

**WebSocket:** WS /ws/dashboard — POST /api/auth/ws-ticket, then pass the ticket via Sec-WebSocket-Protocol: ofdd.ws, <ticket>. Query ?ticket= is dev-only when OFDD\_WS\_ALLOW\_QUERY\_TICKET=1. Tickets are short-lived and not interchangeable with the Bearer token.

### 150.1 Health & audit

| Method | Path          | Role   | Description                                                         |
|--------|---------------|--------|---------------------------------------------------------------------|
| GET    | /health       | public | Minimal liveness (ok, service, version, auth_required)              |
| GET    | /health/stack | auth   | Stack traffic-light (verbose URLs/bind with OFDD_DEBUG_DIAGNOSTICS) |

| Method | Path                | Role | Description                  |
|--------|---------------------|------|------------------------------|
| POST   | /api/auth/ws-ticket | auth | Short-lived WebSocket ticket |
| GET    | /api/audit/summary  | auth | Audit counters               |
| GET    | /api/audit/events   | auth | Audit log (query params)     |
| GET    | /api/audit/errors   | auth | Recent API errors            |

\*Exact public set depends on OFDD\_AUTH\_DISABLED and middleware.

## 150.2 Auth

| Method | Path             | Description               |
|--------|------------------|---------------------------|
| POST   | /api/auth/login  | Username/password → JWT   |
| GET    | /api/auth/me     | Current user              |
| GET    | /api/auth/status | Auth mode / lockout hints |

## 150.3 Rules & FDD

| Method | Path                         | Role        | Description                 |
|--------|------------------------------|-------------|-----------------------------|
| GET    | /api/rules/saved             | user        | Rule index                  |
| POST   | /api/rules/save              | operator+   | Create/update rule metadata |
| GET    | /api/rules/saved/{id}/source | user        | Python source               |
| PUT    | /api/rules/saved/{id}/source | operator+   | Write .py file              |
| GET    | /api/rules/assignments       | user        | Point bindings              |
| POST   | /api/rules/bind              | integrator+ | Bind rule to point          |
| POST   | /api/rules/bindings          | integrator+ | Bulk bindings               |

| Method   | Path                  | Role      | Description                                 |
|----------|-----------------------|-----------|---------------------------------------------|
| POST     | /api/rules/<br>batch  | operator+ | Run all enabled rules<br>→ fdd_results.json |
| GET/POST | /api/rules/<br>drafts | user      | Draft rules                                 |

## 150.4 Rule Lab playground

| Method | Path                                 | Description                                           |
|--------|--------------------------------------|-------------------------------------------------------|
| POST   | /api/<br>playground/<br>lint         | AST lint Python rule                                  |
| POST   | /api/<br>playground/<br>test-rule    | Run apply_faults_arrow() on<br>feather PyArrow window |
| POST   | /api/<br>playground/<br>run-script   | Execute script mode                                   |
| GET    | /api/<br>playground/<br>sample-frame | Sample DataFrame for editor                           |

## 150.5 BRICK / data model

| Method | Path                                    | Role        | Description                                                            |
|--------|-----------------------------------------|-------------|------------------------------------------------------------------------|
| GET    | /api/model/<br>sites                    | user        | Site list                                                              |
| GET    | /api/model/<br>tree                     | user        | Equipment tree<br>(SPARQL)                                             |
| GET    | /api/model/<br>graph                    | user        | RDF graph fragment                                                     |
| GET    | /api/model/<br>scope                    | user        | Scoped query                                                           |
| GET    | /api/model/<br>export                   | user        | Export TTL/JSON                                                        |
| GET    | /api/model/<br>commissioning-<br>export | user        | Full commissioning<br>bundle (points +<br>fdd_rule_ids +<br>fdd_rules) |
| POST   |                                         | integrator+ |                                                                        |



| Method   | Path                            | Role        | Description                               |
|----------|---------------------------------|-------------|-------------------------------------------|
|          | /api/model/commissioning-import |             | Bulk import; validate in UI dry-run first |
| GET      | /api/model/health               | user        | Model point/equipment counts              |
| GET      | /api/model/ttl                  | user        | TTL metadata                              |
| POST     | /api/model/sync-ttl             | integrator+ | Regenerate TTL from JSON                  |
| POST     | /api/model/import               | integrator+ | Import model payload                      |
| POST     | /api/model/sites                | integrator+ | Create site                               |
| GET/POST | /api/model/bacnet-sync          | integrator+ | BACnet ↔ model sync state                 |
| GET      | /api/model/fdd-query-presets    | user        | Saved FDD query presets                   |
| GET      | /api/model/fdd-coverage         | user        | Rule coverage summary                     |
| POST     | /api/model/sparql               | user        | SPARQL query (scoped)                     |

## 150.6 Rule Lab (dashboard)

| Method | Path                    | Role      | Description                      |
|--------|-------------------------|-----------|----------------------------------|
| POST   | /api/rules/lab/validate | operator+ | Validate rule source             |
| POST   | /api/rules/lab/preview  | operator+ | Preview rule on historian window |
| POST   | /api/rules/lab/compare  | operator+ | Compare two rule versions        |
| POST   | /api/rules/lab/lint-sql | operator+ | DataFusion SQL safety lint       |
| POST   | /api/rules/lab/save     | operator+ | Save Rule Lab draft              |

Mounted at /api/rules/lab — see OpenAPI /docs.

## 150.7 Bench validation (read-only)

| Method | Path                                                  | Role      | Description                                         |
|--------|-------------------------------------------------------|-----------|-----------------------------------------------------|
| GET    | /api/<br>bench/<br>health                             | operator+ | Bench harness health                                |
| GET    | /api/<br>bench/<br>mapping                            | operator+ | Semantic cross-source<br>mapping                    |
| GET    | /api/<br>bench/<br>poll-<br>status                    | operator+ | BACnet + Niagara poll<br>heartbeat                  |
| POST   | /api/<br>bench/<br>validate/<br>bacnet-vs-<br>niagara | operator+ | Dual-source<br>equivalence check                    |
| POST   | /api/<br>bench/<br>long-fdd/<br>evaluate              | operator+ | Long FDD smoke<br>evaluate (PyArrow/<br>DataFusion) |

See [Bench 5007 long FDD smoke](#)({{ "/operations/bench-5007-long-fdd-smoke/" | relative\_url }}).

## 150.8 Niagara baskStream

| Method | Path                                                 | Role | Description         |
|--------|------------------------------------------------------|------|---------------------|
| GET    | /api/<br>niagara/<br>health                          | read | Connector health    |
| GET    | /api/<br>niagara/<br>stations                        | read | Configured stations |
| POST   | /api/<br>niagara/<br>stations/<br>{id}/<br>discover  | read | Discover points     |
| POST   | /api/<br>niagara/<br>stations/<br>{id}/poll/<br>once | poll | One poll cycle      |

| Method | Path                                   | Role | Description               |
|--------|----------------------------------------|------|---------------------------|
| GET    | /api/niagara/stations/{id}/poll/status | read | Poll worker status        |
| GET    | /api/niagara/driver/tree               | read | Driver commissioning tree |

Read-only by default. See `Niagara baskStream connector({{ “/niagara-baskstream-connector/” | relative_url }})`.

## 150.9 BACnet

| Method | Path                          | Role       | Description               |
|--------|-------------------------------|------------|---------------------------|
| GET    | /config/bacnet                | read       | Commission URL / config   |
| GET    | /api/bacnet/commission/status | read       | Commission agent health   |
| GET    | /api/bacnet/server/points     | read       | Server point table        |
| GET    | /api/bacnet/inventory         | read       | Discovered inventory      |
| POST   | /api/bacnet/whois             | commission | Who-Is                    |
| POST   | /api/bacnet/discover          | commission | Discover devices          |
| POST   | /api/bacnet/read              | commission | Read property             |
| POST   | /api/bacnet/read-multiple     | commission | RPM read                  |
| POST   | /api/bacnet/write             | write      | Supervisory write (gated) |

| Method | Path                              | Role       | Description                 |
|--------|-----------------------------------|------------|-----------------------------|
| POST   | /api/bacnet/import-to-model       | integrator | Inventory → BRICK           |
| POST   | /api/bacnet/driver/sync-discovery | commission | Sync driver tree            |
| POST   | /api/bacnet/poll/once             | commission | Trigger poll cycle          |
| GET    | /api/bacnet/poll/status           | read       | Poll worker status          |
| POST   | /ingest/bacnet                    | read       | Ingest samples into feather |

Bind address: BACNET\_BIND in commission env — see [BACnet network setup]({{ "/bacnet/network-setup/" | relative\_url }}).  
 Operator guide: [Discover and read](#)({{ "/bacnet/discover-read/" | relative\_url }}).

PATCH | /api/bacnet/driver/point | commission | Enable poll + interval on point |  
 PATCH | /api/bacnet/driver/device | commission | Enable poll for all points on device |  
 PATCH | /api/bacnet/driver/device/remap | commission | Change instance and/or address |  
 DELETE | /api/bacnet/driver/point/{point\_id} | commission | Remove point from registry |  
 DELETE | /api/bacnet/driver/device/{device\_instance} | commission | Remove device |  
 DELETE | /api/bacnet/driver/registry | commission | Clear CSVs + model sync |

## 150.10 Faults (check-engine)

| Method | Path                | Description                |
|--------|---------------------|----------------------------|
| GET    | /api/faults/catalog | Fixed fault codes          |
| GET    | /api/faults/status  | GREEN/YELLOW/RED aggregate |
| GET    | /api/faults/tree    | Fault tree by equipment    |
| GET    |                     | Code metadata              |

| Method | Path                    | Description |
|--------|-------------------------|-------------|
|        | /api/faults/code/{code} |             |

## 150.11 Timeseries & sites

| Method | Path                       | Description         |
|--------|----------------------------|---------------------|
| GET    | /api/timeseries/sites      | Feather sites       |
| GET    | /api/timeseries/series     | Series metadata     |
| GET    | /api/timeseries/plot       | Plot payload        |
| GET    | /api/timeseries/readings   | Raw readings        |
| GET    | /api/sites                 | Site list           |
| GET    | /api/sites/{site_id}/frame | Wide frame for site |

## 150.12 Building & host

| Method | Path                 | Description      |
|--------|----------------------|------------------|
| GET    | /api/building/status | Building summary |
| GET    | /api/building/alerts | Active alerts    |
| GET    | /api/host/stats      | CPU/mem/disk     |

## 150.13 Local agent (Ollama)

Prefix `/openfdd-agent`. Role `agent` for tool execution.

| Method | Path                         | Description                                  |
|--------|------------------------------|----------------------------------------------|
| GET    | /openfdd-agent/context       | Composed prompt context                      |
| GET    | /openfdd-agent/tools         | Tool manifest                                |
| POST   | /openfdd-agent/tool          | Execute tool (e.g. <code>rules.save</code> ) |
| GET    | /openfdd-agent/ollama/health | Ollama reachability                          |
| GET    |                              |                                              |

| Method | Path                            | Description                     |
|--------|---------------------------------|---------------------------------|
| GET    | /openfdd-agent/building-insight | Check-engine narrative context  |
|        | /openfdd-agent/zone-temps       | Zone temperature insight        |
|        | /openfdd-agent/chat             | Chat completion (catalog-bound) |

Optional host Ollama for building insight — not required for core FDD. Configure via compose profile or host service; see [Architecture overview]({{ "/architecture/overview/" | relative\_url }}).

## 150.14 Modbus

| Method | Path                       | Role        | Description                          |
|--------|----------------------------|-------------|--------------------------------------|
| GET    | /api/modbus/driver/tree    | operator+   | Commissioning tree                   |
| GET    | /api/modbus/poll/status    | operator+   | Poll worker status                   |
| POST   | /api/modbus/poll/once      | integrator+ | Force poll cycle                     |
| POST   | /api/modbus/read_registers | operator+   | One-shot register read               |
| POST   | /api/modbus/read_and_store | operator+   | Read + CSV + feather (source=modbus) |
| POST   | /api/modbus/refresh        | operator+   | Re-read one register                 |
| PATCH  | /api/modbus/register/poll  | integrator+ | Enable poll interval                 |

See [Driver framework](#)({{ "/drivers/" | relative\_url }}).

## 150.15 JSON API (HTTP/HTTPS)

| Method | Path                     | Role      | Description |
|--------|--------------------------|-----------|-------------|
| GET    | /api/json-api/env/status | operator+ |             |

| Method | Path                              | Role        | Description                                         |
|--------|-----------------------------------|-------------|-----------------------------------------------------|
|        |                                   |             | Env var configured flags (no secret values)         |
| POST   | /api/json-api/presets/openweather | integrator+ | Register 3-point OpenWeather bundle + optional poll |
| GET    | /api/json-api/driver/tree         | operator+   | Endpoints grouped by host                           |
| GET    | /api/json-api/poll/status         | operator+   | Poll worker status                                  |
| POST   | /api/json-api/poll/once           | integrator+ | Force poll cycle                                    |
| POST   | /api/json-api/request             | operator+   | One-shot GET/POST (no store)                        |
| POST   | /api/json-api/read_and_store      | operator+   | Request + CSV + feather (source=json_api)           |
| POST   | /api/json-api/refresh             | operator+   | Re-fetch endpoint                                   |
| PATCH  | /api/json-api/endpoint/poll       | integrator+ | Enable poll interval                                |
| DELETE | /api/json-api/endpoint/{point_id} | integrator+ | Remove endpoint                                     |

**Outbound auth:** auth\_type = none | bearer | basic; optional verify\_tls (default true). See [JSON API driver](#)({ { "/drivers/json-api/" | relative\_url } }).

## 150.16 PyPI package (separate)

Library-only HTTP is **not** bundled. See [Python package](#)({ { "/appendix/python-package/" | relative\_url } }) for open\_fdd.arrow\_runtime (Arrow rules offline).

## 150.17 Errors

JSON error bodies; stack traces hidden unless OFDD\_DEBUG\_TRACEBACKS=1. See [LAN hardening](#)({ { "/security/lan-hardening/" | relative\_url } }).

# 151 Python package

## 152 Python package (open-fdd)

Install from PyPI (embeddable FDD runtime — not the Docker edge stack):

```
pip install open-fdd
pip install "open-fdd[analytics]"      # optional NumPy helpers
pip install "open-fdd[ml]"            # optional sklearn for
offline experiments
```

### 152.1 Modules

| Module                 | Purpose                                                                            |
|------------------------|------------------------------------------------------------------------------------|
| open_fdd.arrow_runtime | <b>Default</b> — apply_faults_arrow on PyArrow Tables, cookbook masks, column maps |
| open_fdd.playground    | Rule Lab lint/compile helpers for Arrow rules                                      |

Retired in 3.0.1+: open\_fdd.engine (YAML/pandas RuleRunner) is **not** shipped on PyPI. Use Operator Bridge Rule Lab or import rules from workspace/data/rules\_py/.

### 152.2 When to use package-only

| Scenario                       | Use                                           |
|--------------------------------|-----------------------------------------------|
| Lint/test Arrow rules offline  | open_fdd.arrow_runtime + open_fdd.playground  |
| Graph ML experiments (Layer B) | open-fdd[ml] in experiments/ (see issue #211) |
| Full building operator stack   | <b>Docker</b> Operator Bridge                 |

### 152.3 Versioning

Publish: git tag vX.Y.Z (or legacy open-fdd-vX.Y.Z) → GitHub Actions **Publish open-fdd to PyPI**. See [Release process](#)({{ "/" developer/release-process/" | relative\_url }}).



```
import open_fdd
print(open_fdd.__version__)
```

## 152.4 Offline Arrow example

```
import pyarrow as pa
import pyarrow.compute as pc
from open_fdd.arrow_runtime import run_arrow_rule

code = '''
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(table["SAT"], float(cfg["high"]))
'''

table = pa.table({"SAT": [70.0, 90.0, 88.0]})
result = run_arrow_rule(code, table, {"high": 85})
print(result.flagged_count)
```

API surface: open\_fdd/arrow\_runtime docstrings and [Arrow recipes] (`{{ "/rule-cookbook/arrow-recipes/" | relative_url }}`).

# 153 Configuration reference

# 154 Configuration reference

## 154.1 Bridge / auth

| Variable              | Default   | Notes                       |
|-----------------------|-----------|-----------------------------|
| OFDD_BRIDGE_HOST      | 127.0.0.1 | 0.0.0.0 for LAN             |
| OFDD_AUTH_SECRET      | —         | Required for LAN            |
| OFDD_AUTH_DISABLED    | 0         | Localhost dev only          |
| OFDD_INSECURE_LAN_DEV | 0         | Lab only with auth disabled |

## 154.2 BACnet

| Variable                  | Notes                           |
|---------------------------|---------------------------------|
| BACNET_BIND               | ip/mask:47808 in commission.env |
| OFDD_BACNET_WRITE_ENABLED |                                 |

| Variable | Notes                                     |
|----------|-------------------------------------------|
|          | Default off — site-specific name may vary |

## 154.3 Docker edge

| Variable                      | Notes                       |
|-------------------------------|-----------------------------|
| OPENFDD_IMAGE_TAG             | GHCR tag for Ansible deploy |
| openfdd_docker_pull_from_ghcr | Ansible host_var            |

Rule Lab Python rules use `rules_store.json + workspace/data/rules_py/*.py`. Legacy YAML schema (`docs/config_schema.json`) is archived — not used by Operator Bridge 3.0.1+.

## 155 Glossary

## 156 Glossary

| Term                   | Definition                                                                  |
|------------------------|-----------------------------------------------------------------------------|
| <b>Operator Bridge</b> | FastAPI service serving API + static dashboard                              |
| <b>Rule Lab</b>        | UI and APIs for Arrow <code>apply_faults_arrow()</code> rules               |
| <b>Feather store</b>   | On-disk columnar historian under <code>workspace/data/feather_store/</code> |
| <b>fdd_input</b>       | Brick tag linking a point to a rule binding role                            |
| <b>Check-engine</b>    | Building-level GREEN/YELLOW/RED fault summary                               |
| <b>Commission</b>      | BACnet discovery/read (and optional write) agent container                  |
| <b>Poll driver</b>     | Scheduled BACnet RPM reads into historian                                   |
| <b>GHCR</b>            | GitHub Container Registry — <code>ghcr.io/bbartling/openfdd-*</code>        |

# 157 Workspace env files

# 158 Workspace env files

All live under workspace/. Copy from \*.example templates;  
committed \*.local files are gitignored.

| File               | Loaded by                              | Purpose                                                                                                                                       |
|--------------------|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| auth.env.local     | run_local.sh, Docker compose, Ansible  | JWT secret (OFDD_AUTH_SECRET), integrator/operator/agent passwords, OFDD_AUTH_DISABLED for localhost dev only                                 |
| data.env.local     | run_local.sh, bridge, feather maintain | Feather historian caps (OFDD_FEATHER_*), audit/error log rotation (OFDD_LOG_*), BACnet discovery mutation toggle                              |
| json_api.env.local | bridge, Ansible edge sync              | API keys for JSON API driver \${ENV:VAR} substitution (e.g. OPENWEATHER_API_KEY)                                                              |
| ollama.env.local   | run_local.sh, bridge                   | Ollama base URL, model name, RAM tier, GPU mode, timeouts — see <a href="#">Ollama and analytics</a> ({{ "/operator-bridge/ollama-analytics/" |
| mcp.env.local      | run_local.sh, MCP sidecar              | Enable MCP RAG REST (OFDD_MCP_*), index path                                                                                                  |
| caddy.env.local    | run_local.sh, Ansible edge             | Reverse proxy mode (http / tls / off), TLS CN, port overrides                                                                                 |
| pentest.env.local  | pentest_production_stack.sh            | Production-like LAN scan stack: CORS, auth, passive BACnet defaults                                                                           |

## 158.1 Ansible / edge extras

| Path                                       | Purpose                                                                                      |
|--------------------------------------------|----------------------------------------------------------------------------------------------|
| infra/ansible/secrets/<br><host>.env.local | SSH deploy credentials, GHCR<br>pull tokens, site-specific BACnet<br>bind — maintainers only |

## 158.2 Related docs

- [Secrets and auth](#)({{ "/security/secrets-auth/" | relative\_url }}) — required production variables
- [Logging and audit](#)({{ "/ops/logging/" | relative\_url }}) — OFDD\_LOG \* detail
- [JSON API driver](#)({{ "/drivers/json-api/" | relative\_url }}) — \${ENV:VAR} pattern
- [Configuration reference](#)({{ "/appendix/configuration/" | relative\_url }}) — bridge runtime flags not in env files

# 159 Appendix

# 160 Appendix

Reference material for integrators and library users.

| Page                                                                                        | Content         |
|---------------------------------------------------------------------------------------------|-----------------|
| <a href="#">API routes</a> ({{ "/appendix/bridge_api/"   relative_url }})                   | relative_url }} |
| <a href="#">Python package</a> ({{ "/appendix/python-package/"   relative_url }})           | relative_url }} |
| <a href="#">Configuration reference</a> ({{ "/appendix/configuration/"   relative_url }})   | relative_url }} |
| <a href="#">Workspace env files</a> ({{ "/appendix/workspace-env-files/"   relative_url }}) | relative_url }} |
| <a href="#">Glossary</a> ({{ "/appendix/glossary/"   relative_url }})                       | relative_url }} |

# 161 GL36 lab site (Acme)

# 162 GL36 lab site (Acme)

**Lab note only.** This documents a maintainer **example edge host** (site acme, building vm-bbartling) used for Guideline 36 supervisory FDD development. Replace inventory host names, IPs, and BACnet ranges for your site.

Reference for **ASHRAE Guideline 36** supervisory monitoring on a live lab edge. Rule code lives in `workspace/data/rules_py/`; commissioning uses **Arrow constants** (no `config.json`).

Trim & respond Niagara reference: [README\\_TRIM\\_RESPOND](#).

## 162.1 BACnet poll scope

Device filter (`infra/ansible/scripts/acme_trim_devices.sh`):

| Range       | Equipment       |
|-------------|-----------------|
| 1-100       | JCI VMA/VAV     |
| 1100        | RTU-01 (AHU)    |
| 1002        | Hot water plant |
| 11000-13000 | Trane UC210 VAV |

Poll interval: **60 s**. Commission from control machine:

```
cd infra/ansible
POLL_INTERVAL=60 ./scripts/acme_commission_gl36.sh
```

Points land in `edge_backup/local/acme/vm-bbartling/points.csv` for Ansible sync.

## 162.2 Example FDD rules (Arrow)

### 162.2.1 Duct static pressure high (AHU)

```
""AHU duct static high – GL36 trim advisory.""

import pyarrow.compute as pc

VALUE_COLUMN = "duct-static"
HIGH_INWC = 1.20
LOOKBACK_HOURS = 3
```

```

def _kit_lookback_stats(table, *, hours=None):
    h = hours if hours is not None else LOOKBACK_HOURS
    ts = pc.cast(table["timestamp"], "timestamp[us, UTC]")
    tmin, tmax = pc.min(ts).as_py(), pc.max(ts).as_py()
    span_h = (tmax - tmin).total_seconds() / 3600.0 if tmin and
tmax else 0.0
    print(f"lookback={h}h rows={table.num_rows} start={tmin}
stop={tmax} span={span_h:.2f}h")

def apply_faults_arrow(table, cfg, context=None):
    _kit_lookback_stats(table)
    return pc.greater(pc.cast(table[VALUE_COLUMN], "float64"),
HIGH_INWC)

```

## 162.2.2 SAT too cold vs setpoint

```

"""AHU SAT too cold vs setpoint."""

import pyarrow.compute as pc

SAT_COLUMN = "sa-t"
SP_COLUMN = "sa-t-sp"
COLD_DELTA_F = 5.0

def apply_faults_arrow(table, cfg, context=None):
    sat = pc.cast(table[SAT_COLUMN], "float64")
    sp = pc.cast(table[SP_COLUMN], "float64")
    return pc.less(pc.subtract(sat, sp), -COLD_DELTA_F)

```

## 162.2.3 Site rule stubs in the repo

| Rule file                                           | Use                                    |
|-----------------------------------------------------|----------------------------------------|
| bench_stat_zn-t_flatline_1h.py                      | Zone temp flatline — bind ZN-T per VAV |
| acme_duct_static_flatline_1h_gl36_duct_t_r_input.py | RTU duct static flatline               |
| oat_vs_web_spread_1h.py                             | Local OAT vs OpenWeather JSON API      |

## 162.2.4 Local OA-T vs web weather (JSON API)

1. Control machine: workspace/json\_api.env.local with  
OPENWEATHER\_API\_KEY
2. JSON API tab → **OpenWeatherMap** preset → **Register**  
(default poll **30 min**)
3. Bind BACnet local OAT in the model (oa-t historian column)
4. Enable oat\_vs\_web\_spread\_1h.py in Rule Lab — flags when |oa-t  
- web-oat-t| > 8 °F

## 162.3 Deploy / upgrade (Ansible, this lab only)

```
cd infra/ansible
OPENFDD_IMAGE_TAG=latest ./deploy.sh docker --limit
<inventory_host> -e enable_bacnet_poll_driver=true
```

Full UI + image upgrade from control machine:

```
OPENFDD_IMAGE_TAG=latest ./scripts/upgrade_edge_full.sh --limit
<inventory_host>
infra/ansible/scripts/post_deploy_check.sh --limit
<inventory_host> --full
```

Do **not** rsync dev auth.env.local to the edge (deploy\_sync\_auth: false in private host\_vars). Regenerate credentials on the edge host if login fails after upgrade.

## 162.4 AI-assisted data modeling (lab workflow)

1. BACnet poll running (~340 GL36 points @ 60 s on this lab host).
2. **Model & assignments** → export commissioning JSON for LLM-assisted binding.
3. Import returned JSON via commissioning import.
4. Pin FDD rules on ZN-T, DA-T, duct-static, OAT, etc. — export shows readable names in fdd\_rules\_linked.

Model seed: workspace/data/acme\_gl36\_model.json.

# 163 Examples & lab notes

# 164 Examples & lab notes

Site-specific commissioning notes, GL36 examples, and maintainer lab workflows. These pages are **not** required for a generic edge deployment.

| Page                                                                                  | Description |
|---------------------------------------------------------------------------------------|-------------|
| <a href="#">GL36 lab site (Acme)({{ "/examples/acme-gl36-lab/"   relative_url }})</a> |             |

For production IT operators, start with [Quick Start\({{ "/quick-start/" | relative\\_url }}\)](#) and [Operations\({{ "/ops/" | relative\\_url }}\)](#).

## 165 Paired FDD smoke (bench + Acme)

## 166 Paired FDD smoke

Hardcoded cross-site validation: **bensserver bench 5007** and **Acme** run the **same schedule** with mid-test fault-parameter toggles. PyArrow and DataFusion SQL rules must agree on flagged row counts.

### 166.1 What is tested

| Site         | Rule                                                | Normal phase | Blatant phase |
|--------------|-----------------------------------------------------|--------------|---------------|
| Bench (demo) | stat_zn-t bounds on BACnet 5007 + Niagara bench9065 | 65-75 °F     | 99-100 °F     |
| Acme         |                                                     | ≤ 10 °F      | ≤ 0.001 °F    |



| Site | Rule                                  | Normal phase | Blatant phase |
|------|---------------------------------------|--------------|---------------|
|      | Local oa-t<br>vs web-oat-<br>t spread |              |               |

Rule IDs are fixed in `open_fdd/validation/paired_fdd_contract.py`.  
Each site has **arrow** and **datafusion\_sql** variants.

## 166.2 Fault confirmation (5 minutes)

All paired smoke rules use:

```
        {"min_elapsed_minutes": 5, "min_true_rows": 5, "poll_interval_s":  
60}
```

A condition must stay true for **~5 minutes** before it counts as a confirmed fault. See [Fault confirmation](#).

## 166.3 Modes (hardcoded)

| Mode      | Duration | Toggle interval |
|-----------|----------|-----------------|
| tryout    | 6 min    | 3 min           |
| short     | 30 min   | 15 min          |
| standard  | 2 h      | every 15 min    |
| overnight | 12 h     | every 15 min    |

Toggles alternate **normal** ↔ **blatant** fault parameters via the rules API (same as an operator or AI agent adjusting thresholds mid-run).

## 166.4 Run

Prereqs: local stack on :8765, Acme edge on latest image,  
`OPENFDD_LIVE_ACME=1`.

```
        OFDD_SKIP_UI_BUILD=1 ./scripts/run_local.sh start  
        OPENFDD_IMAGE_TAG=3.1.3 ./scripts/upgrade_edge_site.sh --limit  
acme_vm_bbartling  
  
        OPENFDD_LIVE_ACME=1 ./scripts/smoke_paired_fdd_harness.sh --  
tryout      # dev  
        OPENFDD_LIVE_ACME=1 ./scripts/smoke_paired_fdd_harness.sh --  
short       # 30 min  
        OPENFDD_LIVE_ACME=1 ./scripts/smoke_paired_fdd_harness.sh --  
standard    # 2 h
```

```
OPENFDD_LIVE_ACME=1 ./scripts/smoke_paired_fdd_harness.sh --  
overnight # 12 h
```

scripts/smoke\_sites\_parity.sh delegates to this harness.

## 166.5 Reports

- reports/paired\_fdd\_smoke\_validation.md
- reports/paired\_fdd\_smoke\_validation.json
- reports/site\_parity\_smoke.json (UI bundle + API revision parity)

## 166.6 Pass criteria

- Site parity smoke passes (matching UI hash, no 401 on health endpoints).
- No PyArrow vs SQL flagged-count mismatch per cycle.
- No save/batch API errors in either site loop.

Blatant-phase fault counts may be zero on Acme when the historian is still on demo fallback — check feather ingest and live OAT columns before overnight runs.

# 167 Arrow-native runtime

# 168 Arrow-native runtime

Open-FDD 3.0 executes Rule Lab rules on **PyArrow tables** end to end.

```
feather_store.read_site_table()  
→ fdd_runner / playground.run_arrow_table()  
→ apply_faults_arrow(table, cfg, context)  
→ pyarrow.compute.masks
```

## 168.1 Authoring contract

```
import pyarrow.compute as pc  
  
def apply_faults_arrow(table, cfg, context=None):  
    return pc.greater(table["zone_temp"], cfg["max_zone_temp"])
```

Cookbook helpers: `open_fdd.arrow_runtime.cookbook` (flatline, spread, OOB, after-hours fan).

Script-mode analytics rules receive `table` (PyArrow) and `cfg` in the sandbox — not pandas DataFrames.

## 168.2 PyArrow-only policy (edge + Rule Lab)

| Allowed                                                                       | Forbidden                                             |
|-------------------------------------------------------------------------------|-------------------------------------------------------|
| <code>pyarrow (pa)</code> , <code>pyarrow.compute (pc)</code>                 | <code>import pandas, pd.DataFrame, to_pandas()</code> |
| <code>open_fdd.arrow_runtime.cookbook</code> helpers                          | <code>import numpy</code> for rule logic              |
| Script mode globals: <code>table</code> , <code>cfg</code> , <code>out</code> | Legacy <code>df DataFrame</code> scripts              |

Rule Lab **lint** (`POST /api/playground/lint`) and rule **save** reject pandas patterns with agent-facing errors, e.g. *use `table` (PyArrow), not `df`*.

Fault rules: `apply_faults_arrow(table, cfg, context=None)`. Analytics scripts: top-level code on `table/cfg`, set `out = {"events": [...], "metrics": {...}}`.

## 168.3 Package layout

| Module                                           | Role                              |
|--------------------------------------------------|-----------------------------------|
| <code>open_fdd.arrow_runtime.backend</code>      | Execute rule code, batch chunks   |
| <code>open_fdd.arrow_runtime.cookbook</code>     | Shared fault masks                |
| <code>open_fdd.arrow_runtime.windows</code>      | Rolling min/max, consecutive-true |
| <code>open_fdd.playground.arrow_templates</code> | Rule Lab starter templates        |

# 169 Release process

# 170 Release process

Open-FDD has **two publish paths**:

| Artifact                 | Trigger                                    | Registry                                                                  |
|--------------------------|--------------------------------------------|---------------------------------------------------------------------------|
| <b>PyPI</b> open-fdd     | Git tag vX.Y.Z (or legacy open-fdd-vX.Y.Z) | <a href="https://pypi.org/project/open-fdd">pypi.org/project/open-fdd</a> |
| <b>Docker</b> edge stack | Same release tag (or manual workflow)      | ghcr.io/bbartling/openfdd-*                                               |

Do **not** publish from ordinary branch pushes. Merge to master, bump version, tag, push tag.

## 170.1 1. Prepare version

```
git checkout master
git pull --ff-only
# Edit pyproject.toml + open_fdd/__init__.py → same X.Y.Z
python scripts/release/check_version.py
```

## 170.2 2. Merge and tag

```
git tag -a v3.0.30 -m "open-fdd 3.0.30"
git push origin v3.0.30
```

Tag push runs:

- [.github/workflows/publish-open-fdd.yml](#) — build, test, twine check, PyPI via **Trusted Publishing**
- [.github/workflows/docker-publish.yml](#) — GHCR images with tags 3.0.30, 3.0, latest

Legacy tag open-fdd-v3.0.30 is still accepted during transition.

## 170.3 3. PyPI Trusted Publishing setup (one-time)

On [pypi.org](https://pypi.org) → project **open-fdd** → Publishing → **Add a new trusted publisher**:

| Field             | Value                    |
|-------------------|--------------------------|
| PyPI project name | open-fdd                 |
| Owner             | bbartling                |
| Repository        | open-fdd                 |
| Workflow name     | Publish open-fdd to PyPI |

| Field       | Value                                  |
|-------------|----------------------------------------|
| Environment | pypi (optional; enables approval gate) |

No long-lived API token in the repo.

## 170.4 4. Verify release

```

pip install open-fdd==3.0.30
python -c "import open_fdd; print(open_fdd.__version__)"

docker pull ghcr.io/bbartling/openfdd-bridge:3.0.30
docker manifest inspect ghcr.io/bbartling/openfdd-bridge:3.0.30

```

## 170.5 5. Manual workflow\_dispatch

- **PyPI:** Publish open-fdd to PyPI with dry\_run=true (default) — build/test only
- **Docker:** Publish Docker images to GHCR — publishes pyproject.toml version + minor alias + latest (optional image\_tag override)

## 170.6 Package vs Docker

See `PyPI package`(`{{ "/developer/pypi-package/" | relative_url }}`) and `[Docker images]`(`{{ "/ops/docker-images/" | relative_url }}`).

# 171 PyPI package

## 172 PyPI package (open-fdd)

### 172.1 What it is

Embeddable **Arrow-native FDD compute** for consultants, cloud pipelines, and test benches:

- Lint and test `apply_faults_arrow(table, cfg, context)` rules
- Run rules on PyArrow Tables, Feather, or Parquet
- Optional NumPy analytics helpers (`pip install "open-fdd[analytics]"`)

## 172.2 What it is not

The PyPI wheel does **not** include:

- Operator Bridge / FastAPI app
- React dashboard
- BACnet commission/poll service
- MCP server
- Ansible / Tailscale deploy tooling

Use **GHCR Docker images** for the full edge stack.

## 172.3 Install

```
pip install open-fdd
pip install "open-fdd[analytics]" # optional NumPy helpers
pip install "open-fdd[ml]"       # offline sklearn experiments
```

## 172.4 CLI

```
open-fdd version
open-fdd lint-rule path/to/rule.py
open-fdd test-rule path/to/rule.py --input sample.feather --
config cfg.json
open-fdd run-arrow path/to/rule.py --input in.feather --output
out.feather
open-fdd validate-rule-pack ./rules
```

## 172.5 Rule contract

```
def apply_faults_arrow(table, cfg, context=None):
    # return PyArrow bool mask, length == table.num_rows
    ...
```

Use `open_fdd.arrow_runtime.run_arrow_rule` for structured ArrowRuleResult (counts, errors, preview).

## 172.6 Modules shipped on PyPI

| Module                              | Purpose                                     |
|-------------------------------------|---------------------------------------------|
| <code>open_fdd.arrow_runtime</code> | Rule execution, cookbook masks, column maps |
| <code>open_fdd.playground</code>    | Lint/compile helpers                        |
| <code>open_fdd.faults</code>        | Fault catalog YAML                          |

| Module          | Purpose        |
|-----------------|----------------|
| open_fdd.schema | Result schemas |

open\_fdd.engine (pandas/YAML) is **not** in the wheel.

## 172.7 Examples

Repo examples/ — [arrow\\_minimal](#), Feather batch, generic IoT pipeline.

## 172.8 Local validation

```

pip install -e ".[test,dev,analytics]"
python scripts/release/check_version.py
pytest open_fdd/tests -q
python -m build && twine check dist/*

```

# 173 MCP server

# 174 Open-FDD MCP server

FastMCP agent interface over the **operator bridge** — not a parallel FDD stack.

## 174.1 Modes

| Mode      | OFDD_MCP_MODE | Site resolution                                            |
|-----------|---------------|------------------------------------------------------------|
| Edge      | edge          | OPENFDD_BRIDGE_BASE_URL<br>(default http://127.0.0.1:8765) |
| Portfolio | portfolio     | portfolio/sites.json +<br>optional site_id per tool        |

**RCx Central** (:8050 Dash, :8060 API) is a **separate HTTP stack** — not the MCP server. Agents doing RCx overview, FDD preset tables, or DOCX reports should call Central API (/api/central/\*) or use the Dash; see [RCx Central agent guide](#).

Acme production edge runs **without** MCP (enable\_mcp: false). Run MCP centrally and call http://100.122.106.124 via Tailscale, or use RCx Central with Edge Connections registry.

## 174.2 Transport

| Transport       | Env                                                    | Use                       |
|-----------------|--------------------------------------------------------|---------------------------|
| Streamable HTTP | OFDD_MCP_TRANSPORT=streamable-http (default in Docker) | http://127.0.0.1:8090/mcp |
| Stdio           | OFDD_MCP_TRANSPORT=stdio                               | Cursor / Claude Desktop   |

Legacy bridge doc search: POST /tools/search\_docs on :8090 (unchanged).

## 174.3 Human approval

These tools require human\_approved=true:

- save\_rule
- apply\_fdd\_tuning
- run\_fdd\_batch

MCP does not command BACnet writes — only Open-FDD bridge APIs.

## 174.4 Client examples

**Cursor** (~/.cursor/mcp.json):

```
{
  "mcpServers": {
    "open-fdd": {
      "command": "python",
      "args": ["-m", "mcp_server.run"],
      "env": {
        "PYTHONPATH": "/path/to/open-fdd/workspace:/path/to/open-fdd/workspace/api",
        "OPENFDD_REPO_ROOT": "/path/to/open-fdd",
        "OFDD_MCP_TRANSPORT": "stdio",
        "OFDD_MCP_MODE": "portfolio",
        "OFDD_PORTFOLIO_SITES_PATH": "/path/to/open-fdd/portfolio/sites.json"
      }
    }
  }
}
```



```
}  
}
```

**HTTP** (bensserver dev stack):

```
docker compose -f docker/compose.dev.yml up -d mcp-rag  
curl -fsS http://127.0.0.1:8090/health
```

## 174.5 Site registry example

```
{  
  "sites": [  
    {  
      "site_id": "acme",  
      "name": "Acme GL36 Lab",  
      "base_url": "http://100.122.106.124",  
      "username": "integrator"  
    }  
  ]  
}
```

Store passwords in env or OS keychain — not in git.

## 174.6 RCx Central (not MCP)

| Service     | Default URL           | Agent docs                                |
|-------------|-----------------------|-------------------------------------------|
| Central API | http://127.0.0.1:8060 | <a href="#">central-api.md</a>            |
| RCx Dash    | http://127.0.0.1:8050 | <a href="#">rcx-central-dash-agent.md</a> |

MCP portfolio\_rollup hits Edge /api/building/portfolio-rollup per site. RCx GET /api/central/overview/{site\_id} adds Dash KPIs, fault pie (with [short fault descriptions](#)), and mechanical narrative from FDD presets.

## 174.7 Fault code labels (agents)

Use fixed lookup — do not paraphrase codes in UI copy:

- Docs: [fault-codes/short-lookup.md](#)
- Python: portfolio/central/fault\_code\_lookup.py
- Edge API: GET /api/faults/catalog

## 174.8 Doc RAG refresh

After editing docs/ consumed by MCP search:

```
./scripts/build_mcp_rag_index.sh
```

- workspace/mcp\_server/ — FastMCP tools, resources, prompts
- workspace/mcp\_rag/retrieval.py — RAG index (rebuild separately)
- Docker image openfdd-mcp-rag — compatibility name; runs unified server

## 174.9 Module layout

# 175 Adr Datafusion Rust Ready Rule Backend

## 176 ADR: DataFusion as optional Rust-ready rule backend

### 176.1 Status

Accepted — Open-FDD 3.1.0

### 176.2 Context

Open-FDD v3 executes FDD rules over **Apache Arrow** tables. The primary contract is:

```
apply_faults_arrow(table, cfg, context=None) -> BooleanArray |  
ChunkedArray
```

Operators and integrators author rules in Python/PyArrow. BACnet polling, bridge APIs, dashboard, MCP, and Docker deployment all consume normalized ArrowRuleResult payloads.

We want a migration path toward Rust/DataFusion components **without** rewriting the project in Rust or weakening the PyArrow contract.

## 176.3 Decision

Add an **optional** second rule backend:

| Backend        | Entry                                          |
|----------------|------------------------------------------------|
| arrow          | apply_faults_arrow(...) (default, first-class) |
| datafusion_sql | Restricted SELECT over registered telemetry    |

DataFusion is Apache Arrow-native and implemented in Rust. The Python datafusion package gives Open-FDD a clean seam where a future Rust service can replace or accelerate the wrapper while preserving the same result schema.

## 176.4 Why

- Same Arrow memory model end-to-end
- SQL expressions are easy to read for simple threshold/CASE rules
- Compare mode can prove SQL masks match PyArrow before migration
- Optional extra — normal pip install open-fdd unchanged

## 176.5 Non-goals (this milestone)

- No Rust rewrite
- No replacing PyArrow rules
- No pandas on edge runtime
- No arbitrary SQL console (injection-safe subset only)
- No changes to BACnet, bridge auth, or MCP unless required for tests

## 176.6 Future path

**Phase A** — PyArrow + optional DataFusion Python backend (this ADR)

**Phase B** — Shared Arrow rule result contract across batch, Rules Lab, and reporting

**Phase C** — Optional Rust DataFusion crate/service replaces Python wrapper

**Phase D** — ML/graph analytics consume Arrow/Parquet outputs without changing the FDD rule contract

## 176.7 Consequences

- CI adds a job with `pip install -e ".[test,dev,datafusion]"` for backend validation
- Rule store persists `sql` and `fault_column` separately from Python code
- Rules Lab gains backend selector; SQL executes server-side only

## 177 Adr Rust Ready Arrow Fdd Contract

## 178 ADR: Rust-ready Arrow FDD contract

### 178.1 Status

Accepted

### 178.2 Context

Open-FDD runs supervisory FDD over historian telemetry from BACnet, Niagara, JSON API, and future connectors. PyArrow is the primary rule path; DataFusion SQL is an optional backend for SQL-authored rules and Rust migration prep.

### 178.3 Decision

1. **PyArrow remains primary.** Rules implement `apply_faults_arrow(table, cfg, context=None)` and return Arrow boolean masks normalized to `ArrowRuleResult`.
2. **DataFusion SQL is optional.** Install via `open-fdd[datafusion]`. Rules declare `backend: datafusion_sql`, execute server-side only, read the registered telemetry Arrow table, and normalize to the same `ArrowRuleResult` shape.

3. **Common historian contract.** Drivers ingest timestamp, site\_id, point columns, and metadata.source into Feather shards. Point metadata (fdd\_input, cross\_source\_semantic, equipment\_id) drives rule binding — not vendor-specific code in rule engines.
4. **Future Rust.** A Rust execution service may replace the Python Arrow/DataFusion wrappers behind the same table-in, boolean-mask-out contract. UI, commissioning, and driver orchestration stay in Python until migrated.
5. **Drivers must not bypass the contract.** No connector may emit a custom FDD result shape or row-loop rule path for production batch evaluation.
6. **No browser-side SQL.** DataFusion runs in the bridge process only.
7. **No pandas row-loop runtime on edge.** PyArrow columnar masks only for production FDD.
8. **Confirmation.** May use correctness-first Python streak loops today; future vectorized/window implementations must keep the same raw vs confirmed mask contract.

## 178.4 Consequences

- Rule Lab, batch FDD, and validation smokes share one mask contract.
- Confirmation window filters apply after raw masks for all backends.
- Vectorizing confirmation (window functions / Arrow kernels) is planned; correctness-first Python streak loops are acceptable for bench scale today.

## 179 Bench Validation Agent

## 180 Bench validation agent (local benservert)

**Local bench only. Read-only.**

## 180.1 Topology

| Item                  | Value                                                       |
|-----------------------|-------------------------------------------------------------|
| Open-FDD host         | benserver                                                   |
| BACnet device         | instance <b>5007</b> , router 2000:7 (MS/TP)                |
| Niagara station       | https://192.168.204.11                                      |
| Niagara BACnet folder | slot:/Drivers/BacnetNetwork/BENS\$20BENCHTEST\$20BOX/points |
| Password env          | OPENFDD_NIAGARA_ADMIN_PASSWORD (never commit)               |

/stream/health returns **302** before login (reachability). **200** after SCRAM.

## 180.2 One-command validation

```
export OPENFDD_NIAGARA_ADMIN_PASSWORD='...'  
python3 scripts/bootstrap_bench_dual_source.py  
python3 scripts/bench_validate_bacnet_vs_niagara.py --write-report
```

## 180.3 API tools (bridge must be running)

| Action                  | Endpoint                                      |
|-------------------------|-----------------------------------------------|
| Bench health            | GET /api/bench/health                         |
| Cross-source validate   | POST /api/bench/validate/bacnet-vs-niagara    |
| Poll status             | GET /api/bench/poll-status                    |
| Poll cadence            | GET /api/bench/poll-cadence                   |
| Niagara test            | POST /api/niagara/stations/bench9065/test     |
| Niagara discover        | POST /api/niagara/stations/bench9065/discover |
| Start/stop Niagara poll | POST .../poll/start / .../poll/stop           |
| BACnet driver tree      | GET /api/bacnet/driver/tree                   |

## 180.4 MCP / agent notes

- Browser → Open-FDD API only (no direct Niagara WebSocket from React).

- Preserve Niagara ORDs exactly (\$20, \$2d, \$23). PowerShell: single-quote ORDs.
- Default Niagara discovery excludes proxyExt (10 points, not 20).
- Branch: fix/3.0.34-bugs; push and watch PR #300 CI + CodeRabbit.

## 180.5 Overnight

```
python3 scripts/run_overnight_bench_smoke.py
```

See [overnight-bench-smoke.md](#) and [bench-bacnet-vs-niagara.md](#).

# 181 Rcx Central Dash Agent

## 182 RCx Central Dash — agent guide

For Cursor/Codex agents working on **OpenFDD RCx Central** (portfolio/) — analyst Dash + Central API over read-only Edge REST.

### 182.1 Runtime stack (benserver / analyst workstation)

| Service          | Script                          | Port | Bind                                  |
|------------------|---------------------------------|------|---------------------------------------|
| RCx Central API  | ./scripts/run_central_api.sh    | 8060 | 0.0.0.0<br>(OPENFDD_CENTRAL_API_HOST) |
| RCx Central Dash | ./scripts/run_portfolio_dash.sh | 8050 | 0.0.0.0<br>(OPENFDD_RCX_DASH_HOST)    |

Env: OPENFDD\_CENTRAL\_API\_URL=http://127.0.0.1:8060 (Dash → API).  
LAN access: sudo ./scripts/open\_rcx\_central\_lan\_ports.sh (UFW 8050+8060).

**Not** the operator bridge (:8765) — RCx Central polls Edge sites via Tailscale/LAN URLs in portfolio/config/sites.json.

## 182.2 Dash UI (2026-06)

Single **Dashboard** tab (faults, building summary, FDD rules, preset buttons, RCx report) + **Edge Connections** tab.

| Area               | Data source                                                                                  |
|--------------------|----------------------------------------------------------------------------------------------|
| Fault pie + legend | <code>fault_code_lookup.py</code> + live/CSV faults                                          |
| Building summary   | <code>build_mechanical_narrative()</code> → Edge FDD presets                                 |
| FDD rules table    | <code>GET /api/central/fdd-analytics/{site_id}</code>                                        |
| FDD preset buttons | <code>GET /api/central/fdd-preset/{site_id}/{preset_id}</code> → same as Edge Data Model tab |
| P8 overrides       | KPI count always; chart only when overrides present                                          |
| RCx DOCX           | <code>POST /api/central/rcx/report</code>                                                    |

## 182.3 Edge parity (Data Model tab)

RCx Central must **not** scrape React. Use Edge REST:

- `GET /api/model/fdd-query-presets` + `GET /api/model/fdd-query-presets/{id}`
- `GET /api/model/sparql/predefined` + `POST /api/model/sparql`
- `GET /api/model/health`, `GET /api/model/tree`

Preset ids: `rules_to_equipment`, `rules_to_sensors`, `ahus_vavs_zones`, `missing_rule_bindings`, `orphan_points`, `rule_coverage_by_equipment_type`, etc. — see `portfolio/central/fdd_preset_catalog.py`.

## 182.4 BRICK equipment typing (common bug)

Acme and many sites store types in **brick\_type** (AHU, VAV), not `equipment_type`.

- Count/classify with `portfolio/central/equipment_classify.py` (mirrors `workspace/api/openfdd_bridge/equipment_classify.py`).
- TTL sync maps short types → Brick classes (AHU → `Air_Handling_Unit`, VAV → `Variable_Air_Volume_Box`) in `ttl_service.py`.
- After model.json edits on Edge: **sync TTL** before SPARQL HVAC buttons return counts.
- Packaged rooftop serving VAVs: prefer **brick\_type: AHU** and display name AHU 01 (not Rtu 01) when it is a full air handler.



## 182.5 Fault codes (stable labels)

| Layer        | Path                                                                               |
|--------------|------------------------------------------------------------------------------------|
| Docs table   | docs/fault-codes/short-lookup.md                                                   |
| Dash lookup  | portfolio/central/fault_code_lookup.py                                             |
| Edge catalog | GET /api/faults/catalog ·<br>workspace/api/<br>openfdd_bridge/<br>fault_catalog.py |

Never invent codes. Letter suffix only (VAV-C, not VAV-03).

## 182.6 Central API routes (agent smoke)

```
curl -s http://127.0.0.1:8060/health
curl -s http://127.0.0.1:8060/api/central/overview/acme
curl -s "http://127.0.0.1:8060/api/central/fdd-analytics/acme?
hours=24"
curl -s http://127.0.0.1:8060/api/central/fdd-preset/acme/
ahus_vavs_zones
```

## 182.7 Tests

```
pytest tests/portfolio/ tests/scripts/
test_rcx_overnight_patch_cycle.py
pytest tests/workspace_bridge/test_fdd_query_presets.py
```

## 182.8 PRs (typical split)

- **Edge bridge + TTL + model fixes** → fix/edge-dashboard-bridge (#299)
- **RCx Dash / portfolio only** → feature/rcx-central-overview-ui (#298)

## 182.9 Related

- Overnight patch cycle
- Model queries
- MCP server — portfolio mode vs RCx Central HTTP API

# 183 Rcx Central Overnight Patch Cycle

## 184 RCx Central overnight patch-cycle (agent runbook)

See also: [RCx Central Dash agent guide](#) for UI/API modules and BRICK typing.

Mission: validate **OpenFDD RCx Central** against local services and (optionally) live **ACME Edge** read-only, up to **10 patch cycles** with **2-hour** spacing.

### 184.1 Safety (non-negotiable)

- **Read-only** on BACnet and live BAS equipment.
- Never commit secrets, .env, tokens, or raw building dumps.
- Merge, publish, and deploy only when RCX\_ALLOW\_MERGE=1, RCX\_ALLOW\_PUBLISH=1, or RCX\_ALLOW\_DEPLOY=1 and the owner authorizes.

### 184.2 One patch cycle

1. `git pull --ff-only` on `feature/rcx-central-overview-ui` (or follow-up branch).
2. `gh pr checks 298` and `gh run view <id> --log-failed` for failures.
3. Classify CodeRabbit comments: `must-fix` / `nice-to-have` / `false positive`.
4. Run tests: `python3 -m pytest tests/portfolio tests/workspace_bridge -q`.
5. Start RCx Central: `./scripts/run_central_api.sh`, `./scripts/run_portfolio_dash.sh` (bind `0.0.0.0`; LAN: `sudo ./scripts/open_rcx_central_lan_ports.sh`).
6. Run `python3 scripts/rcx_central_overnight_patch_cycle.py` (try-out or overnight env).
7. Patch bugs, commit, push, confirm CI green.
8. Write `reports/rcx_central_overnight_logs/cycle_NN.md`.

## 184.3 GH Actions

```
gh pr checks 298
gh run list --branch feature/rcx-central-overview-ui --limit 10
gh run view <RUN_ID> --log-failed
```

Fix order: unit tests → operator-bridge → docs → Docker supervisor.

## 184.4 RCx Central local

```
./scripts/run_central_api.sh # :8060
OPENFDD_CENTRAL_API_URL=http://127.0.0.1:8060 ./scripts/
run_portfolio_dash.sh # :8050
curl -s http://127.0.0.1:8060/health
curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:8050/
```

Edge connections: save URL + credentials on **Edge Connections** tab (local portfolio/config/).

## 184.5 Live ACME (gated)

```
export OPENFDD_LIVE_ACME=1
export RCX_EDGE_SITE=acme
export RCX_CENTRAL_API_URL=http://127.0.0.1:8060
```

Requires ACME credentials in local registry — never log passwords.

## 184.6 Try-out vs overnight

| Mode      | RCX_PATCH_CYCLES | RCX_CYCLE_SLEEP_MINUTES |
|-----------|------------------|-------------------------|
| Try-out   | 2                | 0                       |
| Overnight | 10               | 120                     |

## 184.7 Rev bump / deploy

Only when tests and CI pass and owner sets RCX\_ALLOW\_PUBLISH=1 / RCX\_ALLOW\_DEPLOY=1. Use existing scripts/upgrade\_edge\_full.sh — do not invent new deploy paths.

## 184.8 Branch cleanup after merge

```
git checkout master && git pull --ff-only
git branch -d feature/rcx-central-overview-ui
git push origin --delete feature/rcx-central-overview-ui
```

## 185 Bench Bacnet Vs Niagara

## 186 BACnet direct vs Niagara bench validation

Local **benserver** read-only cross-check: same physical BACnet device via two paths.

### 186.1 Topology

```
BENS BENCHTEST BOX (BACnet device 5007, MS/TP 2000:7)
├─ Open-FDD native BACnet poll (commission agent → samples.csv →
feather source=bacnet)
└─ Niagara Windows station → baskStream WebSocket → Open-FDD
Niagara driver (source=niagara_baskstream)
```

### 186.2 Quick start

```
export OPENFDD_NIAGARA_ADMIN_PASSWORD='...' # never commit
python3 scripts/bootstrap_bench_dual_source.py
python3 scripts/bench_validate_bacnet_vs_niagara.py --write-report
```

Or via API (bridge running):

```
curl -X POST http://127.0.0.1:8765/api/bench/validate/bacnet-vs-
niagara \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/
json" \
-d '{"write_report": true}'
```

## 186.3 Mapping config

Edit workspace/data/bench\_bacnet\_vs\_niagara.yaml for semantic point mapping, tolerances, and Niagara ORDs (\$20 / \$2d preserved).

## 186.4 Brick model

Import dual-source semantics:

```
# Included in bootstrap; or manually:
python3 -c "
import json, os, sys
from pathlib import Path
REPO = Path('.').resolve()
sys.path.insert(0, str(REPO/'workspace/api'))
os.environ['OFDD_DESKTOP_DATA_DIR'] = str(REPO/'workspace/data')
from openfdd_bridge.model_service import ModelService
from openfdd_bridge.ttl_service import TtlService
ModelService().import_json(json.loads((REPO/'workspace/data/
bench_dual_source_model.json').read_text()), replace=False)
TtlService().sync()
"
```

## 186.5 Normalized data model

All drivers normalize through driver\_point\_contract.py:

- **bacnet** → canonical bacnet\_direct
- **niagara\_baskstream, modbus, json\_api**

Semantic identity (fdd\_input / cross\_source\_semantic) links both sources to one Brick point role.

## 186.6 Poll intervals

- BACnet: poll\_interval\_s in points.csv (standard: 60, 300, ...)
- Niagara: poll\_interval\_seconds on station config; worker re-reads each cycle (no restart required)
- Lab minimum Niagara interval: 15s (API validates  $\geq 15$ )

Validate cadence: GET /api/bench/poll-cadence?  
source=bacnet\_direct&expected\_interval\_s=60

## 186.7 Overnight smoke

```
python3 scripts/run_overnight_bench_smoke.py --dry-run # one
checkpoint
python3 scripts/run_overnight_bench_smoke.py # 12h,
checkpoint every 2h
```

Reports: reports/overnight\_bench/<timestamp>/

## 186.8 Read-only rules

No BACnet writes, no Niagara overrides, no alarm ack. Relay booleans are read-only observations only.

# 187 Dashboard Analytics

## 188 Dashboard analytics

Engineering analytics pages live under **Analytics** in the operator dashboard (/analytics).

### 188.1 Pages

| Route                | Purpose                                            |
|----------------------|----------------------------------------------------|
| /analytics           | Overview KPIs, fault-hour charts, top faults table |
| /analytics/faults    | Time-range fault analytics                         |
| /analytics/equipment | AHU / VAV fault-hour summary                       |
| /analytics/health    | BACnet / BRICK model health                        |
| /analytics/rcx       | RCx Report Builder (preview + DOCX download)       |

### 188.2 API (read-only)

- GET /api/analytics/overview
- GET /api/analytics/faults?hours=24
- GET /api/analytics/model-health
- POST /api/reports/rcx/preview

- POST /api/reports/rcx/generate → DOCX download

## 188.3 Fault hours

Elapsed fault hours are estimated from FDD batch run analytics.estimated\_fault\_duration\_sec when present, otherwise from flagged/total sample ratio. See open\_fdd/reports/fault\_hours.py.

## 188.4 Theme

All Plotly charts use getPlotlyThemeLayout(theme) from workspace/dashboard/src/lib/plotly-theme.ts.

## 188.5 Tests

```
pytest tests/workspace_bridge/test_dashboard_analytics.py
open_fdd/tests/test_fault_hours.py
cd workspace/dashboard && npm test
```

Live BACnet is not required. Gate live ACME validation with OPENFDD\_LIVE\_ACME=1.

# 189 Datafusion Sql Rules

## 190 DataFusion SQL rules (optional)

Open-FDD v3 remains **PyArrow-first**. Rules implement apply\_faults\_arrow(table, cfg, context=None) and return a boolean Arrow mask.

The optional **DataFusion SQL** backend (backend: datafusion\_sql) is a future-Rust-prep seam:

- Input: PyArrow Table registered as telemetry
- Rule body: a restricted SELECT that adds a boolean fault column
- Output: the same ArrowRuleResult shape as PyArrow rules

Install:

```
pip install 'open-fdd[datafusion]'
```

## 190.1 When to use SQL vs PyArrow

| Use DataFusion SQL                          | Use PyArrow                                   |
|---------------------------------------------|-----------------------------------------------|
| Simple thresholds, CASE WHEN, boolean logic | Custom HVAC helpers, rolling windows, ML prep |
| SQL-readable expressions for operators      | Complex graph/feature work                    |
| Proving parity before Rust migration        | Primary supported authoring path              |

## 190.2 Rule metadata

```
id: zone_temp_high_sql
name: Zone temperature high - SQL
backend: datafusion_sql
fault_column: fault
sql: |
    SELECT
        *,
        zone_temp > 75.0 AS fault
    FROM telemetry
```

## 190.3 Safety

SQL rules are **not** a database console. The linter rejects:

- Multiple statements, DDL/DML
- Reads other than FROM telemetry
- File paths and external URLs
- Missing or non-boolean fault column
- Row counts that differ from input telemetry
- Wrong fault column alias (e.g. not\_fault when fault is required)

Server-side execution only — the Rules Lab browser editor never runs SQL locally.

## 190.4 Rules Lab

In **Rule Lab**, choose **PyArrow** or **DataFusion SQL**:

- **Validate SQL** — lint + optional server preview on latest historian sample



- **Run Preview** — bounded sample against site telemetry
- **Compare backends** — prove an equivalent SQL rule matches a PyArrow rule mask

See [ADR: DataFusion Rust-ready backend](#).

# 191 Dashboard Rcx Makeover Inventory

## 192 Dashboard + RCx Makeover — Inventory

Branch: feature/dashboard-rcx-mega-makeover  
Date: 2026-05-30

### 192.1 What exists

| Area                     | Location                                                   | Notes                                                           |
|--------------------------|------------------------------------------------------------|-----------------------------------------------------------------|
| React operator dashboard | workspace/dashboard/                                       | Vite + React 19, theme via contexts/theme-context.tsx, CSS vars |
| Plotly charts            | lib/plot-chart.ts, PlotPage.tsx, HostStatsPage.tsx         | Partial theme handling; not centralized                         |
| Building check-engine    | building_status.py, FaultsPage, BuildingInsightDashboard   | Single source for alerts                                        |
| FDD batch + analytics    | fdd_results.py, fdd_fault_analytics.py                     | Per-run sample analytics, equipment enrichment                  |
| Poll / model health      | poll_throughput.py, model_health.py, device_poll_health.py | Used by agent, not dashboard analytics pages                    |
| Zone temp analytics      | zone_temp_analytics.py                                     | VAV comfort metrics via agent routes                            |
| Central RCx DOCX         |                                                            | Central-only (POST /api/                                        |

| Area                     | Location                                              | Notes                                         |
|--------------------------|-------------------------------------------------------|-----------------------------------------------|
|                          | portfolio/central/rcx_report.py,<br>fault_hours.py    | central/rcx/report)                           |
| Generic open_fdd reports | open_fdd/reports/docx_generator.py,<br>fault_viz.py   | Equipment FDD reports, optional deps          |
| Portfolio desk           | portfolio/dash/app.py,<br>portfolio/central/api.py    | Separate Dash + FastAPI stack                 |
| Frontend unit tests      | 14 Vitest files under<br>workspace/dashboard/src/lib/ | Not in CI                                     |
| Bridge Python tests      | tests/workspace_bridge/,<br>tests/portfolio/          | Strong coverage for building insight, central |

## 192.2 What partially exists

- **Theme toggle** — app shell updates; Plotly uses inline colors in plot-chart.ts / HostStatsPage.tsx (inconsistent).
- **Fault display** — equipment names in check-engine alerts when model maps columns; catalog page is not analytics-focused.
- **RCx DOCX** — portfolio central builder with basic sections; not exposed on edge bridge; no chart/section checkboxes.
- **Fault hours** — portfolio/central/fault\_hours.py estimates from rollups; FDD run analytics has duration estimates but no REST aggregation.
- **Demo data** — workspace/data/acme\_gl36\_model.json, fdd\_results.json, samples/demo\_site.csv; no unified analytics fixture.

## 192.3 What is missing (this branch targets)

- Shared getPlotlyThemeLayout() helper + all charts using it
- Analytics pages: Overview, Fault Analytics, Equipment, BACnet/Model Health, RCx Report Builder
- Bridge REST: /api/analytics/overview, /faults, /equipment/{id}, /model-health, /reports/rcx/preview, /reports/rcx/generate
- Edge RCx DOCX with selectable sections/charts (open\_fdd/reports/rcx\_docx.py)
- Fault-hour aggregation module with tests
- Demo analytics fixture for CI
- Docs: docs/dashboard-analytics.md, docs/rcx-report-builder.md

## 192.4 Files inspected

AGENTS.md, README.md, pyproject.toml  
workspace/dashboard/src/App.tsx, AppLayout.tsx, theme-context.tsx,  
plot-chart.ts  
workspace/api/openfdd\_bridge/main.py, building\_status.py,  
fdd\_results.py, fdd\_fault\_analytics.py  
workspace/api/openfdd\_bridge/routes/analytics\_routes.py  
portfolio/central/rcx\_report.py, fault\_hours.py, api.py  
open\_fdd/reports/  
tests/workspace\_bridge/, tests/portfolio/test\_central.py

## 192.5 Recommended implementation path

1. **Foundation** — open\_fdd/reports/fault\_hours.py, charts.py, rcx\_docx.py; bridge dashboard\_analytics.py + routes; deps in bridge requirements.
2. **Theme** — plotly-theme.ts; refactor plot-chart.ts + HostStatsPage.tsx.
3. **UI** — Analytics nav section + pages wired to new APIs; RCx Report Builder with preview/generate.
4. **Fixtures + tests** — demo JSON, backend pytest, Vitest for theme + report builder.
5. **Docs** — operator-facing how-to.

## 192.6 Known risks

- **Scope** — full prompt is multi-PR; this branch delivers edge dashboard + bridge RCx; Central Dash remains separate.
- **Live BACnet** — analytics read persisted FDD/model/poll data only (read-only).
- **Trend charts in RCx** — need timeseries API + point roles; may be partial when historian empty.
- **ACME coupling** — use demo fixture + model-driven roles, not hard-coded point names.

# 193 Rcx Central Current State

## 194 OpenFDD RCx Central — current state

Updated: 2026-05-30  
Branches: feature/rcx-central-overview-ui (#298)  
**Status:** Local TTL mirror + SPARQL on Central; benserver sign-off before Docker GHCR publish.

### 194.1 Product names

| User-facing                | Internal path                    |
|----------------------------|----------------------------------|
| <b>OpenFDD Edge</b>        | workspace/, docker/, GHCR images |
| <b>OpenFDD RCx Central</b> | portfolio/ (Dash + Central API)  |

### 194.2 Entrypoints

| Service          | Command                                   | Port          | Bind default |
|------------------|-------------------------------------------|---------------|--------------|
| RCx Central Dash | ./scripts/run_portfolio_dash.sh           | 8050          | 0.0.0.0      |
| RCx Central API  | ./scripts/run_central_api.sh              | 8060          | 0.0.0.0      |
| Docker (both)    | docker/rcx-central/<br>docker-compose.yml | 8050,<br>8060 |              |

LAN firewall (benserver): sudo ./scripts/  
open\_rcx\_central\_lan\_ports.sh

### 194.3 Central API

- GET/POST/PUT/DELETE /api/central/edges, POST /api/central/edges/test
- GET /api/central/overview/{site\_id} — KPIs, fault pie, mechanical narrative, P8
- GET /api/central/fdd-analytics/{site\_id} — rules table + descriptions

- GET /api/central/fdd-preset/{site\_id}/{preset\_id} — Edge FDD preset proxy + Central enrichment
- POST /api/central/model/remediate/{site\_id} — equipment types, BACnet metadata, Edge TTL sync, Central mirror
- POST /api/central/model/sync-ttl/{site\_id} — pull data\_model.ttl to portfolio/data/sites/{site\_id}/model/
- GET /api/central/model/sparql/validate/{site\_id} — local AHU/VAV/site SPARQL counts
- POST /api/central/model/sparql/{site\_id} — read-only SPARQL on mirrored TTL
- GET /api/central/mechanical-summary/{site\_id}
- GET /api/central/model-tree/{site\_id} — on-demand BRICK tree (lazy; not on auto-refresh)
- GET /api/central/rcx/points/{site\_id} — BACnet point catalog for Report Builder
- POST /api/central/rcx/preview, /charts/preview, /report

## 194.4 Edge REST (read-only, shared with Data Model tab)

Model: /api/model/tree, /api/model/health, SPARQL, /api/model/fdd-query-presets/{id}

Trends: /api/timeseries/readings

Analytics: /api/analytics/faults

Faults: /api/faults/status, /api/faults/catalog

## 194.5 Dash UI tabs

### Dashboard · Report Builder · Edge Connections

Dashboard: fault mix + legend, building summary, P8 KPI, FDD rules (lazy load), FDD/BRICK preset buttons, **local SPARQL** (TTL mirror).

Report Builder: 3-step wizard — building/time → charts & sections → custom points → preview DOCX.

Edge data fetched on demand (not with dashboard load).

## 194.6 Pre-Docker owner checklist (benserver :8050)

1. Dashboard loads in under 5s without hanging on model tree.
2. **Load full BRICK model** and **Load FDD rules** buttons work when clicked.
3. Report Builder: load catalog → preview charts → generate DOCX for Acme.
4. Fault overlays visible on trend previews when enabled.

5. Sign off here before docker/rcx-central image rebuild or  
RCX\_ALLOW\_PUBLISH=1.

## 194.7 BRICK model notes (agents)

- Classify equipment with brick\_type **and** equipment\_type — see equipment\_classify.py
- TTL maps AHU/VAV → Brick schema classes for SPARQL HVAC queries
- Acme roof unit: brick\_type: AHU, name AHU 01 (not Rtu 01 in new models)

## 194.8 Fault codes

- Stable short labels: docs/fault-codes/short-lookup.md, portfolio/central/fault\_code\_lookup.py
- Full catalog: GET /api/faults/catalog on Edge

## 194.9 Agent docs

- [RCx Central Dash agent guide](#)
- [AI agent workflow](#)
- [MCP server](#)

## 194.10 Tests

```
pytest tests/portfolio/ · tests/workspace_bridge/  
test_fdd_query_presets.py
```

## 194.11 Remaining gaps

- **Owner Dash sign-off** on benserver before Docker GHCR publish
- GHCR publish for openfdd-rcx-central image (gated: RCX\_ALLOW\_PUBLISH=1)
- Optional OpenAI insights hook (templates used today)
- True Edge start/end bounded timeseries (hours lookback used for now)
- Summarize HVAC SPARQL buttons on Dash — **done** (local TTL mirror on Central)

# 195 Short Lookup

## 196 Fault code short descriptions

Stable lookup for pie charts, tables, and RCx reports. Titles match `workspace/api/openfdd_bridge/fault_catalog.py`.

| Code  | Short description                               |
|-------|-------------------------------------------------|
| AHU-A | Supply fan performance degradation              |
| AHU-B | Simultaneous heating and cooling                |
| AHU-C | Supply air temperature sensor fault             |
| AHU-D | Mixed-air temperature inconsistent with OAT/RAT |
| AHU-E | Economizer not economizing                      |
| AHU-F | Damper/valve command vs feedback mismatch       |
| AHU-G | PID hunting / excessive control oscillation     |
| VAV-A | Reheat active during cooling demand             |
| VAV-B | Airflow not meeting setpoint                    |
| VAV-C | Zone temperature sensor fault                   |
| VAV-D | Damper command vs airflow mismatch              |
| VAV-E | Rogue zone (chronic reheat/overcooling)         |
| VAV-F | VAV actuator PID hunting                        |
| BLD-A | Whole-building energy deviation                 |
| BLD-B | Outdoor-air temperature sensor fault            |
| BLD-C | Equipment running outside occupancy schedule    |
| BLD-D | Point data dropout (stale points)               |
| RTU-A | Supply fan performance degradation              |
| RTU-B | Simultaneous heating and cooling                |
| RTU-C | Discharge air temperature sensor fault          |
| RTU-D | Damper/valve command vs feedback mismatch       |
| RTU-E | Cooling capacity PID hunting                    |
| CH-G  | Pump / variable-speed PID hunting               |

Full catalog: [index.md](#) and live GET `/api/faults/catalog` on Edge.

Python mirror: `portfolio/central/fault_code_lookup.py` (RCx Central Dash).

# 197 Niagara Baskstream Connector

## 198 Niagara baskStream connector (read-only)

Open-FDD integrates with **Niagara Framework** stations through the vendor **baskStream** module. The bridge API performs SCRAM login, opens a persistent WebSocket, and reads point values via MessagePack — the browser never connects directly to Niagara.

### 198.1 Read-only posture

This connector is **read-only**. It does not implement writes, overrides, alarm acknowledge, or emergency actions. Use Niagara Workbench for OT commands.

### 198.2 Niagara station setup

1. Install the **baskStream** module on the Niagara station (same module validated by `niagara-baskstream-python-tools` on Linux).
2. Add **BASkStreamService** (or equivalent `baskStream` service) on the station.
3. Ensure HTTPS (port **443**) is reachable from the Open-FDD bridge host.
4. Firewall: allow the bridge server → station on 443.

#### 198.2.1 Health check behavior

- **Before login:** GET `/stream/health` returns **302** redirect to `/login`.
- **After SCRAM login:** GET `/stream/health` returns **200** with JSON including `authenticatedUser`.

From Linux bench:

```
curl -k -i https://192.168.204.11/stream/health  
# Expect 302 before login
```



## 198.3 Open-FDD configuration

1. Copy workspace/niagara.env.example → workspace/niagara.env.local (gitignored).
2. Set OPENFDD\_NIAGARA\_ADMIN\_PASSWORD (or your custom env name).
3. In the dashboard **Niagara** tab, add a station:
  - URL: https://<station-ip>
  - Username: Niagara web user
  - **password\_env**: OPENFDD\_NIAGARA\_ADMIN\_PASSWORD
  - **verify\_tls**: off for self-signed bench certs
  - **default\_points\_root**: e.g. slot:/Drivers/BacnetNetwork/BENS\$20BENCHTEST\$20BOX/points

### 198.3.1 ORD encoding

Niagara ORDs use encoded characters (\$20 = space, \$2d = hyphen, \$23 = #). **Preserve these strings exactly** in config, API, and UI. Do not URL-decode or shell-expand them.

PowerShell users: use **single-quoted** strings for ORDs so \$20 is not treated as a variable.

### 198.3.2 Allowed Origins

The Open-FDD **backend** connector does not require Niagara “Allowed Origins” for browser CORS — traffic is **Browser** → **Open-FDD** /api/niagara → **Niagara**. Direct browser-to-Niagara WebSocket is discouraged.

## 198.4 Dependencies

Bridge API requires aiohttp and msgpack (see workspace/api/requirements.txt). If missing, the app still boots; /api/niagara/health reports dependencies\_ok: false.

## 198.5 Polling

Enable poll per station in the UI. The worker uses one persistent WebSocket per active station, batched reads, cached discovery metadata, and historian ingest with source=niagara\_baskstream.

Disable background worker: OFDD\_DISABLE\_NIAGARA\_POLL\_WORKER=1.

## 198.6 Bench reference (benserver)

| Item                 | Value                                                           |
|----------------------|-----------------------------------------------------------------|
| Station URL          | https://192.168.204.11                                          |
| BACnet device folder | slot:/Drivers/BacnetNetwork/<br>BENS\$20BENCHTEST\$20BOX/points |
| Expected points      | ~10 BACnet components (proxy<br>extensions excluded by default) |

Validate: **Test connection** → **Discover** → **Read selected** / **Poll once**. Values should match the standalone `baskstream_cli.py` values output (OA-T, DUCT-T, STAT ZN-T, etc.).

## 199 Data Model

## 200 Data Model — sync and FDD queries

### 200.1 Explorer (BACnet poll sync)

The **Explorer** tab focuses on **BACnet poll CSV** ↔ **model.json** sync status and **Sync poll** → **model**. Write TTL lives under **Advanced** (or SPARQL RDF tools).

### 200.2 FDD query presets

The **SPARQL** tab includes **FDD / BRICK query presets** — composed queries across `model.json`, rule bindings, and BACnet metadata:

- Rules → Equipment / Sensors / BACnet devices
- Equipment → Points, AHUs/VAVs/Zones
- Missing rule bindings, orphan points, sensor classes, coverage by equipment type

API: GET `/api/model/fdd-query-presets`, GET `/api/model/fdd-query-presets/{id}`.

# 201 Host Resources

## 202 Host Resources — Easy Pooge reset

Integrators see a **Danger Zone** on the Host Resources tab for job-site redeploys.

- Typed confirmation: POOGE THIS EDGE
- Dry-run preview by default
- Allowlisted path cleanup (historian, BACnet scratch, exports)
- Optional Linux apt upgrade and GHCR image pull

API: POST /api/host/pooge/preview, POST /api/host/pooge/run (integrator only).

## 203 Trend Plots

## 204 Trend plots

The **Trend plot** tab reads feather historian data via GET /api/timeseries/readings.

### 204.1 HTTP / Tailscale

Charts work over plain HTTP on localhost, LAN, and Tailscale — COOP/CORP security headers are omitted on non-HTTPS origins so Plotly is not blocked.

### 204.2 Troubleshooting

If the chart is blank, open **Trend plot debug** at the bottom of the tab:

- Endpoint called
- Site, equipment, selected keys
- Series and timestamp row counts
- Last error message

Common fixes: enable BACnet polling, pick points with feather columns, reduce FDD overlay scope.

## 205 Cloud Exporter

## 206 Cloud exporter sidecar

Optional openfdd-cloud-exporter container polls the bridge API and POSTs JSON to a webhook-compatible URL (PostBin, webhook.site, etc.).

### 206.1 Enable (edge compose)

```
export OPENFDD_EXPORT_ENDPOINT=https://webhook.site/your-uuid
export OPENFDD_EXPORT_DRY_RUN=1
docker compose --profile cloud-exporter up -d cloud-exporter
```

Set OPENFDD\_EXPORT\_DRY\_RUN=0 only when you intend live export.

### 206.2 Security

- Disabled by default (OPENFDD\_EXPORT\_DRY\_RUN=1).
- Does not export operator passwords or auth env files.
- Tokens are redacted in logs.

See workspace/cloud\_exporter/README.md for environment variables.

## 207 Overnight Bench Smoke

## 208 Overnight bench smoke test

12-hour local validation on **benserver** only. Read-only.

## 208.1 Command

```
export OPENFDD_NIAGARA_ADMIN_PASSWORD='...'  
python3 scripts/run_overnight_bench_smoke.py
```

Options:

| Flag               | Default | Purpose                              |
|--------------------|---------|--------------------------------------|
| --duration-hours   | 12      | Total run length                     |
| --checkpoint-hours | 2       | Wake interval                        |
| --dry-run          | off     | Single checkpoint then exit          |
| --skip-bootstrap   | off     | Skip BACnet/Niagara setup            |
| --test-poll-freq   | off     | Run poll interval change test at t=0 |

## 208.2 Each checkpoint

1. Cross-source validate (bench\_validate\_bacnet\_vs\_niagara)
2. Poll cadence report (BACnet + Niagara)
3. Targeted pytest (contract, validator, Niagara)
4. JSON + Markdown report under reports/overnight\_bench/<run\_id>/

## 208.3 Final deliverable

final\_report.md / final\_report.json with PASS/FAIL and next actions.

## 208.4 GH Actions loop

After code fixes during a wake cycle:

```
git add ... && git commit -m "... " && git push origin fix/3.0.34-bugs  
gh pr checks 300 --watch
```

No empty commits when validation-only cycles pass.

# 209 Ai Agent Workflow

## 210 AI agent workflow vs runtime use

### 210.1 Three agent surfaces

| Surface                         | Port                   | Use when                                               |
|---------------------------------|------------------------|--------------------------------------------------------|
| <b>OpenFDD Edge</b><br>(bridge) | 8765 / :80<br>Caddy    | Rule Lab, BACnet, Data Model SPARQL, live FDD          |
| <b>OpenFDD RCx Central</b>      | 8050 Dash,<br>8060 API | Multi-site analyst dashboard, RCx DOCX, Edge registry  |
| <b>OpenFDD MCP</b>              | 8090                   | Cursor/Codex tools over bridge or portfolio/sites.json |

Acme production edge runs **without** MCP (enable\_mcp: false). Use **RCx Central API** or **portfolio MCP** against http://100.122.106.124 (Tailscale).

### 210.2 Runtime analyst (no git clone)

1. docker compose -f docker/rcx-central/docker-compose.yml up  
    **or** ./scripts/run\_central\_api.sh + ./scripts/run\_portfolio\_dash.sh
2. Open http://localhost:8050 (LAN: sudo ./scripts/open\_rcx\_central\_lan\_ports.sh on benserver)
3. **Edge Connections** → add Acme URL → Test → Save
4. **Dashboard** → select building → verify building summary (AHU/VAV counts), FDD preset buttons, fault pie descriptions
5. **RCx report** → Preview → Generate DOCX

Kill containers when done. Registry in Docker volume rcx-central-config.

## 210.3 Developer / coding agent (git working tree)

Use Cursor or Codex against open-fdd. Read:

- [RCx Central Dash agent guide](#)
- [Overnight patch cycle](#)

Key modules: portfolio/dash/overview\_tab.py, portfolio/central/mechanical\_narrative.py, portfolio/central/fault\_code\_lookup.py, workspace/api/openfdd\_bridge/fdd\_query\_presets.py, ttl\_service.py.

After doc changes consumed by MCP RAG: ./scripts/build\_mcp\_rag\_index.sh (optional on benserver).

## 210.4 Model / SPARQL validation (Acme)

1. Confirm model.json equipment uses brick\_type (AHU, VAV) — not only legacy equipment\_type
2. Sync TTL on Edge (Data Model tab or bridge restart)
3. Edge UI: **Summarize HVAC → AHUs / VAV boxes** must return non-zero for Acme lab
4. RCx Dash: **AHUs / VAVs / Zones** preset → same row counts as Edge

## 210.5 Secrets

- Never commit portfolio/config/credentials.json or passwords in sites.json
- Env: OFDD\_AGENT\_PASSWORD, per-site credentials in config store
- Live ACME tests: OPENFDD\_LIVE\_ACME=1 only when explicitly requested

## 210.6 Model routing (test triage)

See AGENTS.md — classify CI failures as SIMPLE vs COMPLEX before processing.

# 211 Docker Desktop Windows

## 212 RCx Central on Docker Desktop (Windows)

### 212.1 Prerequisites

- Docker Desktop for Windows
- Edge reachable from the host (e.g. Tailscale)

### 212.2 Run

```
cd open-fdd
docker compose -f docker/rcx-central/docker-compose.yml up --build
```

Open:

- Dash: <http://localhost:8050>
- API health: <http://localhost:8060/health>

No git clone required at runtime if you pull [ghcr.io/bbartling/openfdd-rcx-central](https://github.com/bbartling/openfdd-rcx-central) (when published). Build locally with `OPENFDD_IMAGE_TAG=local`.

### 212.3 Volumes

| Volume             | Purpose                                    |
|--------------------|--------------------------------------------|
| rcx-central-data   | Rollups, reports (portfolio/data/reports/) |
| rcx-central-config | sites.json, masked credentials             |

### 212.4 Tailscale networking

If Edge is reachable on the Windows host but not from inside the Linux container:

1. Test from host: `curl https://<tailscale-ip>:8765/health`
2. Test from container: `docker compose exec rcx-central-api curl -s http://<tailscale-ip>:8765/health`



3. If container cannot route, use the host's Tailscale IP (Docker Desktop often routes LAN correctly) or document `host.docker.internal` proxy for dev.

## 212.5 Smoke test

```
./scripts/test_rcx_central_docker.sh
```

Windows PowerShell variant: run `compose up`, then `curl` health URLs manually per steps above.

## 213 Edge Auth

## 214 Edge authentication (RCx Central)

### 214.1 Browser flow

1. Open RCx Central → **Edge Connections**
2. Enter `site_id`, name, Edge base URL (`https://<tailscale-ip>:8765`)
3. Enter username/password (or bearer token via API)
4. **Test Connection** — probes `/health`, `/api/model/health`, `/api/faults/status`
5. **Save Edge** — writes to `portfolio/config/sites.json` + `credentials.json` (Docker volume)

### 214.2 API

```
GET /api/central/edges
POST /api/central/edges
POST /api/central/edges/test
PUT /api/central/edges/{site_id}
DELETE /api/central/edges/{site_id}
```

Secrets are **never** returned in API responses or committed to git. Passwords are masked in UI after save.

### 214.3 Auth types

- password — `POST /api/auth/login` on Edge

- bearer — static token in credentials store
- none — public Edge endpoints only (dev)

## 214.4 Rejected URLs

file://, ftp://, and hosts without scheme are rejected.

## 215 Index

## 216 OpenFDD RCx Central

**OpenFDD RCx Central** is the local analyst / RCx engineer dashboard. It runs on an analyst workstation or **Docker Desktop (Windows/Linux)** — not on the OT edge by default.

| Product                    | Role                                                        |
|----------------------------|-------------------------------------------------------------|
| <b>OpenFDD Edge</b>        | Arrow-native, no-pandas OT-LAN FDD machine at the building  |
| <b>OpenFDD RCx Central</b> | Read-only multi-edge analytics, chart preview, DOCX reports |

Internal source path: portfolio/ (package name unchanged).

### 216.1 Quick start (host Python)

```
pip install -r portfolio/requirements.txt
./scripts/run_central_api.sh # :8060
./scripts/run_portfolio_dash.sh # :8050
```

### 216.2 Docker Desktop

```
./scripts/run_rcx_central_docker.sh
# Dash http://localhost:8050 API http://localhost:8060/health
```

See [docker-desktop-windows.md](#).

## 216.3 Pages

- Overview — collected rollup trends (Plotly, light/dark theme)
- Edge Connections — add/test/save Edge URLs (secrets in local volume)
- Mechanical Summary — BRICK equipment counts from Edge APIs
- FDD Rules & Analytics — configured rules, chart packs
- Trend Explorer — pointer to Overview Plotly charts (local collect CSVs)
- RCx Report Builder — matplotlib preview, fault overlays, DOCX download
- Validation Runs — one-off read-only Edge probes
- Settings — API URL and volume paths

Model query details: [model-queries.md](#)

## 216.4 Safety

RCx Central is **read-only** toward Edge and BACnet. No writes, commands, or setpoint changes.

# 217 Model Queries

## 218 Model queries (RCx Central → OpenFDD Edge + local TTL mirror)

OpenFDD RCx Central reuses the **same read-only Edge REST APIs** as the Edge operator React **Data model** tab. RCx Central does not scrape the React UI.

**Local BRICK SPARQL:** POST /api/central/model/sync-ttl/{site\_id} fetches Edge model.json, builds TTL with the same ttl\_service as Edge, and stores it under portfolio/data/sites/{site\_id}/model/data\_model.ttl. Central runs SPARQL locally (rdflib). Optionally calls Edge POST /api/model/sync-ttl first so on-site TTL stays aligned when the Edge image is current.

## 218.1 Primary endpoints

| Method | Edge path                         | Purpose                                              |
|--------|-----------------------------------|------------------------------------------------------|
| GET    | /api/model/tree                   | Equipment + points (BRICK roles, timeseries columns) |
| GET    | /api/model/health                 | Point/equipment counts, model issues                 |
| GET    | /api/model/sparql/predefined      | Prebuilt SPARQL query list                           |
| POST   | /api/model/sparql                 | Run SPARQL ( <code>{"query": "..."} </code> )        |
| GET    | /api/model/fdd-query-presets      | FDD commissioning query presets                      |
| GET    | /api/model/fdd-query-presets/{id} | Run preset by id                                     |
| GET    | /api/model/ttl?save=false         | Turtle text (used by Central TTL mirror)             |
| POST   | /api/model/sync-ttl               | Regenerate TTL on Edge from <code>model.json</code>  |

## 218.2 Trends (matplotlib preview + fault overlays)

| Method | Edge path                                         | Purpose                                  |
|--------|---------------------------------------------------|------------------------------------------|
| GET    | /api/timeseries/series?site_id=                   | Plottable columns per site               |
| GET    | /api/timeseries/readings?site_id=&columns=&hours= | Trend samples + optional FDD fault flags |

RCx Central maps BRICK **point roles** (e.g. `supply_air_temperature`) to historian columns via `/api/model/tree`, then calls `/api/timeseries/readings` with `include_faults=true` for overlay bands.

## 218.3 Analytics

| Method | Edge path                    | Purpose                                 |
|--------|------------------------------|-----------------------------------------|
| GET    | /api/analytics/overview      | KPIs, fault hours by severity/equipment |
| GET    | /api/analytics/faults?hours= | Fault rows for selected window          |
| GET    | /api/analytics/model-health  | Stale/missing roles                     |

## 218.4 RCx Central API wrappers

| Method | Central path                                   |
|--------|------------------------------------------------|
| GET    | /api/central/overview/{site_id}                |
| GET    | /api/central/mechanical-summary/{site_id}      |
| GET    | /api/central/fdd-analytics/{site_id}           |
| GET    | /api/central/fdd-preset/{site_id}/{preset_id}  |
| POST   | /api/central/model/remediate/{site_id}         |
| POST   | /api/central/model/sync-ttl/{site_id}          |
| GET    | /api/central/model/ttl/{site_id}               |
| GET    | /api/central/model/sparql/predefined/{site_id} |
| POST   | /api/central/model/sparql/{site_id}            |
| GET    | /api/central/model/sparql/validate/{site_id}   |
| POST   | /api/central/rcx/preview                       |

Edge model mutations use integrator auth via remediate; SPARQL on Central is read-only.

## 218.5 FDD preset ids (Edge parity)

Same as React DataModelSparqlPanel FDD buttons:

rules\_to\_equipment, rules\_to\_sensors, rules\_to\_bacnet\_devices,  
equipment\_to\_points, ahus\_vavs\_zones, missing\_rule\_bindings,  
points\_by\_bacnet\_device, sensor\_classes\_used\_by\_fdd,  
rule\_coverage\_by\_equipment\_type, orphan\_points

Dash **Local BRICK SPARQL** buttons call Central paths above (mirrored TTL).

## 218.6 BRICK typing (do not mis-count AHUs/VAVs)

Many sites (including Acme) set **brick\_type** (AHU, VAV) without **equipment\_type**. Presets and mechanical narrative use **equipment\_classify** — not **equipment\_type** alone.

After `model.json` changes on Edge, **sync TTL** (POST `/api/central/model/sync-ttl/{site_id}` or `remediate`) so SPARQL predefined queries (`Air_Handling_Unit`, `Variable_Air_Volume_Box`) match equipment counts.

Packaged rooftop air handlers feeding VAVs should be modeled as **brick\_type: AHU** even when the legacy BACnet name contains “RTU”.

## 219 Overnight Acme Validation

## 220 Overnight ACME validation via RCx Central

RCx Central drives read-only validation of the ACME OpenFDD Edge over Tailscale.

### 220.1 Prerequisites

- Local RCx Central API (:8060) and Dash (:8050).
- ACME Edge registered under **Edge Connections** with credentials in `portfolio/config/` (gitignored).
- gh CLI authenticated for PR/CI inspection.

### 220.2 Try-out (2 cycles, no sleep)

```
OPENFDD_LIVE_ACME=1 \  
RCX_CENTRAL_API_URL=http://127.0.0.1:8060 \  
RCX_CENTRAL_DASH_URL=http://127.0.0.1:8050 \  
RCX_EDGE_SITE=acme \  
RCX_PATCH_CYCLES=2 \  

```

```
RCX_CYCLE_SLEEP_MINUTES=0 \
python3 scripts/rcx_central_overnight_patch_cycle.py
```

## 220.3 Overnight (10 cycles, 2 h spacing)

Set `RCX_PATCH_CYCLES=10` and `RCX_CYCLE_SLEEP_MINUTES=120`.

Reports land in `reports/rcx_central_overnight_logs/cycle_NN.md`.

## 220.4 Read-only scope

Collects health, model tree counts, fault status, and overview data. No BACnet writes.

# 221 Patch Cycle Release Process

## 222 Patch-cycle release process

A **patch cycle** completes when CI is green, local smoke passes, optional ACME read-only validation passes, changes are committed, and the cycle log is written.

### 222.1 Authorization flags

| Flag                                     | Action                  |
|------------------------------------------|-------------------------|
| <code>RCX_ALLOW_MERGE=0</code> (default) | Do not merge PRs        |
| <code>RCX_ALLOW_PUBLISH=0</code>         | Do not push GHCR images |
| <code>RCX_ALLOW_DEPLOY=0</code>          | Do not deploy to ACME   |

Set to 1 only when the repo owner authorizes.

### 222.2 Image bump (when authorized)

1. Tests and `gh pr` checks green.
2. Bump patch tag only.

3. `./scripts/docker_build.sh` and publish via existing GHCR workflow.
4. `./scripts/upgrade_edge_full.sh --limit <host>` when deploy authorized.
5. Re-run overnight validator with `OPENFDD_LIVE_ACME=1`.

## 223 Report Builder

## 224 RCx Report Builder (RCx Central)

### 224.1 Flow

1. Select saved Edge site
2. Choose date range (2h / 24h / 7d)
3. **Collect Data / Preview** — `POST /api/central/rcx/preview`
4. **Preview Charts** — matplotlib PNG as base64 in browser
5. Select chart/section checkboxes
6. **Generate DOCX** — `POST /api/central/rcx/report`

Reports save to `portfolio/data/reports/` when `save_to_volume=true`.

### 224.2 Fault overlays

Preview charts include severity-colored bands when fault rows exist and overlay toggle is on.

### 224.3 DOCX

Uses `open_fdd/reports/rcx_docx.py` when available (selectable sections/charts). Falls back to `portfolio/central/rcx_report.py`.

Read-only safety note is included in every report.



# 225 Rcx Report Builder

## 226 RCx Report Builder

The **RCx Report Builder** (/analytics/rcx) collects a read-only snapshot from the edge bridge and generates a Word document.

### 226.1 Workflow

1. Choose report window (2 h / 24 h / 7 d).
2. Click **Collect Data / Preview** — calls `POST /api/reports/rcx/preview`.
3. Review available vs disabled charts (disabled charts show missing point roles or no fault data).
4. Select sections and charts.
5. Click **Generate DOCX** — `POST /api/reports/rcx/generate` returns `application/vnd.openxmlformats-officedocument.wordprocessingml.document`.

### 226.2 Implementation

- Preview/readiness: `workspace/api/openfdd_bridge/dashboard_analytics.py`
- DOCX builder: `open_fdd/reports/rcx_docx.py`
- Charts: `open_fdd/reports/charts.py` (matplotlib → BytesIO, no temp PNGs)

### 226.3 Safety

Report generation is **read-only**. No BACnet writes, commands, overrides, or setpoint changes.

### 226.4 Dependencies

Bridge API requires `python-docx` and `matplotlib` (see `workspace/api/requirements.txt`).

## 226.5 Central vs edge

Portfolio Central (portfolio/central/rcx\_report.py, port 8060) supports multi-site rollups. The edge builder uses local FDD results and building status for single-site RCx exports from the operator UI.

## 227 3.1.2

## 228 Open-FDD 3.1.2

Release date: 2026-06-15

### 228.1 Highlights

- **DataFusion / Rust-prep validation** — Bench long FDD smoke exercises PyArrow and DataFusion SQL backends with execution evidence and backend equivalence checks.
- **Bench 5007 long FDD smoke** — Model-driven dual-source validation (BACnet MS/TP device 5007 + Niagara bench9065) with 10-minute fault confirmation window.
- **Fault confirmation window** — Confirmed faults require consecutive true samples; reports include raw vs confirmed timing.
- **GHCR tags** — ghcr.io/bbartling/openfdd-bridge:3.1.2, openfdd-commission:3.1.2, openfdd-mcp-rag:3.1.2.

### 228.2 Known warnings

- `confirmation_engine=python_loop_over_arrow_mask` — confirmation still uses a Python loop over the Arrow fault mask; vectorized confirmation is planned.

### 228.3 Validation

```
python scripts/smoke_bench_5007_long_fdd.py --synthetic --dry-run
python scripts/smoke_bench_5007_long_fdd.py --live --duration-
minutes 35 --baseline-minutes 10 \
    --confirmation-rows 10 --confirmation-minutes 10 --poll-seconds
60 --strict-live-freshness
```

```
python scripts/inspect_bench_5007_long_fdd_report.py workspace/
reports/bench_5007_long_fdd_*.json
```

Reports now classify **historian\_mode** (live\_recent, historical\_replay, synthetic, dry\_run) and refuse to claim live polling was validated when sample timestamps are outside the wall-clock run window. Use --allow-historical-replay only for debugging replay data.

For live bench sign-off, the inspector must show data\_is\_fresh: true and historian\_mode: live\_recent, plus non-zero BACnet/Niagara rows.

## 229 3.1.3

## 230 Open-FDD 3.1.3

Release date: 2026-06-15

### 230.1 Patch fixes

- **Bench long FDD evaluate** — when wall-clock run\_started\_at / lookback filtering would empty a non-empty historian table (demo CSV fallback or stale feather replay), evaluate now uses the full loaded table and sets time\_filter\_relaxed: true. Strict smoke freshness gates still FAIL/WARN honestly on non-fresh samples.
- **Live smoke lookback** — final matrix evaluate uses the same 24h minimum lookback as cycle evaluates.

### 230.2 GHCR tags

- ghcr.io/bbartling/openfdd-bridge:3.1.3
- ghcr.io/bbartling/openfdd-commission:3.1.3
- ghcr.io/bbartling/openfdd-mcp-rag:3.1.3

### 230.3 Validation

```
python scripts/smoke_bench_5007_long_fdd.py --synthetic --dry-run
python scripts/smoke_bench_5007_long_fdd.py --live --duration-
minutes 35 \
    --baseline-minutes 10 --confirmation-rows 10 --confirmation-
minutes 10 \
```

```
--poll-seconds 60 --strict-live-freshness  
python scripts/inspect_bench_5007_long_fdd_report.py workspace/  
reports/bench_5007_long_fdd_*.json
```